Content Guide

Changing and Adding Vehicles

Overview

This document will detail the steps needed to change the current vehicles or add a new vehicle into the game No One Lives Forever 2 (NOLF2). This document will go into changing the source code so proficiency in C++ and Microsoft Visual C++ 6.0 will be needed for most of the changes and additions. In NOLF2 there were two types of vehicles, the snowmobile and the PlayerLure that was used for the bicycle chase. For this document we will only get into adding vehicles like the snowmobile. Within the document, any place 'new vehicle' or 'NewVehicle' is mentioned you should replace with the name of your new vehicle such as, Truck, ATV, or Tricycle. When asked to add code you will only need to add the code that is in bold type. The other code is there as reference so it is easier to see where to add the new code.

PlayerVehicle

If you want to add a new vehicle to NOLF2 you will first need to add the ability to place the new vehicle in the levels. The object that level designers use to place vehicles in the levels in PlayerVehicle. The source for PlayerVehicle is in the files \Game\ObjectDLL\ObjectShared\PlaeryVehicle.cpp/.h
In order to get your new vehicle to show up in the drop down list of the VehicleType property on the PlayerVehicle object you will first need to add a new type to the PlayerPhysicsModel enum and to the list of PlayerPhysicsModel types that are considered vehicles in the function IsVehicleModel() found in Game\Shared\SharedMovement.h. Add these two lines:

```
enum PlayerPhysicsModel
{
        PPM_FIRST=0,
        PPM_NORMAL=0,
        PPM_SNOWMOBILE,
        PPM_LURE,
        PPM_LIGHTCYCLE,
        PPM_NEWVEHICLE,
        PPM_NUM_MODELS
};

inline LTBOOL IsVehicleModel(PlayerPhysicsModel eModel)
{
        switch (eModel)
        {
                case PPM_SNOWMOBILE :
                case PPM_LURE :
                case PPM_LIGHTCYCLE :
                case PPM_NEWVEHICLE:
        return LTTRUE;
                break;

                default : break;
        }

    return LTFALSE;
}
```

Next, you will need to add the name of your new vehicle to the array of vehicle names found in Game\Shared\SharedMovement.cpp. Add the one line:

```
char* aPPMStrings[] =
{
        "Normal",
        "Snowmobile",
        "Lure",
        "LightCycle",
        "NewVehicle"
};
```

Now you need to specify the model filename, skins and renderstyles to use for your vehicle. These are set within the

PlayerVehicle::PostPropRead() found in \Game\ObjectDLL\ObjectShared\PlaeryVehicle.cpp. The current filenames for the snowmobile are defined in the player section of serverbutes.txt. You will want to create new entries for your new vehicle in this section. Open the file \Game\ObjectDLL\ObjectShared\PlayerButes.h and add the following #defines:

```
#define PLAYER_BUTE_SNOWMOBILETRANSLUCENT          "SnowmobileTranslucent"
#define PLAYER_BUTE_NEWVEHICLEMODEL          "NewVehicleModel"
#define PLAYER_BUTE_NEWVEHICLESKIN           "NewVehicleSkin"
#define PLAYER_BUTE_NEWVEHICLERS             "NewVehicleRS"
#define PLAYER_BUTE_NEWVEHICLETRANSLUCENT  "NewVehicleTranslucent"
#define PLAYER_BUTE_WALKVOLUME               "WalkFootstepVolume"
```

You will now need to add these entries to the player section in serverbutes.txt. You will probably want to create a new model for your vehicle but for now, we will just use the tricycle:

```
SnowmobileTranslucent      = 1
NewVehicleModel            = "Props\Models\InTricycle.ltb"
NewVehicleSkin             = "Props\Skins\InTricycle.dtx"
NewVehicleRS               = "RS\default.ltb"
NewVehicleRS2              = "RS\transparent.ltb"
NewVehicleTranslucent      = 1
WalkFootstepVolume         = 0.0
```

Next, you need to setup the PlayerVehicle object to use your new filenames for your new vehicle. All of the file names are set in PlayerVehicle::PostReadProp(). Add the new PlayerPhysicsModel you created to the switch statement there and set the appropriate char pointers for the serverbute entries you made:

```
switch (m_ePPhysicsModel)
{
    case PPM_SNOWMOBILE :
    {
        pModelAttribute     = PLAYER_BUTE_SNOWMOBILEMODEL;
        pSkin1Attribute     = PLAYER_BUTE_SNOWMOBILESKIN;
        pSkin2Attribute     = PLAYER_BUTE_SNOWMOBILESKIN2;
        pSkin3Attribute     = PLAYER_BUTE_SNOWMOBILESKIN3;
        pRenderStyle1Bute   = PLAYER_BUTE_SNOWMOBILERS;
        pRenderStyle2Bute   = PLAYER_BUTE_SNOWMOBILERS2;
        pRenderStyle3Bute   = PLAYER_BUTE_SNOWMOBILERS3;
        pModelTranslucent   = PLAYER_BUTE_SNOWMOBILETRANSLUCENT;
    }
    break;

    case PPM_NEWVEHICLE :
    {
        pModelAttribute     = PLAYER_BUTE_NEWVEHICLEMODEL;
        pSkin1Attribute     = PLAYER_BUTE_NEWVEHICLESKIN;
        pRenderStyle1Bute   = PLAYER_BUTE_NEWVEHICLERS;
        pRenderStyle2Bute   = PLAYER_BUTE_NEWVEHICLERS2;
        pModelTranslucent   = PLAYER_BUTE_SNOWMOBILETRANSLUCENT;
    }
    break;

    default :
    break;
}
```

You will now be able to place your new vehicle in your levels by adding a PlayerView object and selecting NewVehicle in the VehicleType dropdpwn list property. However if you try to activate your new vehicle you will notice that you are unable to actually ride it. This is because you have not yet added the physics for your new vehicle. Adding physics is discussed in the next section.

Vehicle Physics

Once you have your vehicle in the level you will now need to create the player view model for the vehicle and setup the physics. Most of the necessary code you will need to add will be to the class CVehicleMgr located in the source files \Game\ClientShellDLL\ClientShellShared\VehicleMgr.cpp/h. CVehicleMgr is where you will need to add code for getting on your new vehicle, physics for your new vehicle, creating and updating the player view model, and dismounting from your vehicle and switching to normal player physics.

The first thing you will want to do is setup the player view model for your vehicle. The code for creating the model is

in the function, CVehicleMgr::CreateVehicleModel(). You will probably want to create your own player view model that more closely resembles the vehicle you are adding. You will need to add the following code to CVehicleMgr::CreateVehicleModel():

```
case PPM_NEWVEHICLE:
{
        createStruct.m_ObjectType = OT_MODEL;
        createStruct.m_Flags     = FLAG_VISIBLE | FLAG_REALLYCLOSE;
        createStruct.m_Flags2   = FLAG2_FORCETRANSLUCENT | FLAG2_DYNAMICDIRLIGHT;

        SAFE_STRCPY(createStruct.m_Filename, "Guns\\Models_PV\\ NewVehicle.ltb");
        SAFE_STRCPY(createStruct.m_SkinNames[1], "Guns\\Skins_PV\\ NewVehicle.dtx");
        SAFE_STRCPY(createStruct.m_RenderStyleNames[0], "RS\\default.ltb");

        CCharacterFX* pCharFX = g_pMoveMgr->GetCharacterFX();

        if (pCharFX  )
        {
           SAFE_STRCPY(createStruct.m_SkinNames[0], g_pModelButeMgr->GetHandsSkinFilename
(pCharFX->GetModelId()));
        }
}
break;
```

Once your player view model is created you will need to make sure that it will get updated properly. Updating of the vehicle model happens in the function CVehicleMgr::UpdateVehicleModel(). You will need to change the following line:

```
if (!m_hVehicleModel || (m_ePPhysicsModel != PPM_SNOWMOBILE) ) return;
```

To:

```
if (!m_hVehicleModel || ((m_ePPhysicsModel != PPM_SNOWMOBILE) && (m_ePPhysicsModel !=
PPM_NEWVEHICLE)) ) return;
```

That's it for creating and updating your player view model. You will need to add a call to CreateVehicleModel() when you set your new vehicle physics model which is discussed a little later in this document.

The next thing you will need to do is create some console variables for your new vehicle that control a lot of the behavior. Console variables are used so you will be able to change their value while in the game, allowing you to tweak the variables until the vehicle feels the way you want it to behave.
In VehicleMgr.cpp there is already a long list of VarTrack variables, you will need to add the following lines to the list of VarTracks:

```
VarTrack        g_vtNewVehicleInfoTrack;
 VarTrack       g_vtNewVehicleVel;
 VarTrack       g_vtNewVehicleAccel;
 VarTrack       g_vtNewVehicleAccelTime;
 VarTrack       g_vtNewVehicleMaxDecel;
 VarTrack       g_vtNewVehicleDecel;
 VarTrack       g_vtNewVehicleDecelTime;
 VarTrack       g_vtNewVehicleMaxBrake;
 VarTrack       g_vtNewVehicleBrakeTime;
 VarTrack       g_vtNewVehicleOffsetX;
 VarTrack       g_vtNewVehicleOffsetY;
 VarTrack       g_vtNewVehicleOffsetZ;
 VarTrack       g_vtNewVehicleStoppedPercent;
 VarTrack       g_vtNewVehicleStoppedTurnPercent;
 VarTrack       g_vtNewVehicleMaxSpeedPercent;
 VarTrack       g_vtNewVehicleAccelGearChange;
 VarTrack       g_vtNewVehicleMinTurnAccel;
```

You will need to initialize these variables so add the following lines to CVehicleMgr::Init():

```
g_vtNewVehicleInfoTrack.Init( g_pLTClient, "NewVehicleInfo", NULL, 0.0f );
g_vtNewVehicleVel.Init( g_pLTClient, "NewVehicleVel", NULL, 600.0f );
g_vtNewVehicleAccel.Init( g_pLTClient, "NewVehicleAccel", NULL, 3000.0f );
g_vtNewVehicleAccelTime.Init( g_pLTClient, "NewVehicleAccelTime", NULL, 3.0f );
g_vtNewVehicleMaxDecel.Init( g_pLTClient, "NewVehicleDecelMax", LTNULL, 22.5f );
```

```
g_vtNewVehicleDecel.Init( g_pLTClient, "NewVehicleDecel", LTNULL, 37.5f );
g_vtNewVehicleDecelTime.Init( g_pLTClient, "NewVehicleDecelTime", LTNULL, 0.5f );
g_vtNewVehicleMaxBrake.Init( g_pLTClient, "NewVehicleBrakeMax", LTNULL, 112.5f );
g_vtNewVehicleBrakeTime.Init( g_pLTClient, "NewVehicleBrakeTime", LTNULL, 1.0f );
g_vtNewVehicleMinTurnAccel.Init( g_pLTClient, "NewVehicleMinTurnAccel", LTNULL, 0.0f );

 g_vtNewVehicleOffsetX.Init( g_pLTClient, "NewVehicleOffsetX", LTNULL, 0.0f );
g_vtNewVehicleOffsetY.Init( g_pLTClient, "NewVehicleOffsetY", LTNULL, -0.8f );
g_vtNewVehicleOffsetZ.Init( g_pLTClient, "NewVehicleOffsetZ", LTNULL, 0.6f );

 g_vtNewVehicleStoppedPercent.Init( g_pLTClient, "NewVehicleStoppedPercent", LTNULL, 0.05f );
g_vtNewVehicleMaxSpeedPercent.Init( g_pLTClient, "NewVehicleMaxSpeedPercent", LTNULL, 0.95f );
g_vtNewVehicleAccelGearChange.Init( g_pLTClient, "NewVehicleAccelGearChange", LTNULL, 1.0f );
```

Once all of your console variables are set up you can add the code that will make sure your vehicle will use the proper values for things like velocity and acceleration. A lot of the functions that are looking for physics related values are set up as switch statements based on the PlayerPhysicsModel so adding your vehicle requires you to add your NewVehicle PhysicsModel, PPM_NEWVEHICLE, to these switch statements and that the proper values are set to the console variable values. To begin you should add your velocity console variable to the function CVehicleMgr::GetMaxVelMag(). Add these lines to that function:

```
case PPM_SNOWMOBILE :
{
	fMaxVel = g_vtSnowmobileVel.GetFloat();
}
break;

case PPM_NEWVEHICLE:
{
	fMaxVel = g_vtNewVehicleVel.GetFloat();
}
break;
```

Now do the same for CVehicleMgr::GetMaxAccelMag():

```
case PPM_SNOWMOBILE :
{
	fMaxVel = g_vtSnowmobileAccel.GetFloat();
}
break;

case PPM_NEWVEHICLE:
{
	fMaxVel = g_vtNewVehicleAccel.GetFloat();
}
break;
```

In the function CVehicleMgr::GetVehicleAccelTime() add:

```
case PPM_SNOWMOBILE :
	fTime = g_vtSnowmobileAccelTime.GetFloat();
break;

case PPM_NEWVEHICLE:
	fTime = g_vtNewVehicleAccelTime.GetFloat();
break;
```

In the function CVehicleMgr::GetVehicleMaxDecel() add:

```
case PPM_SNOWMOBILE :
	fValue = g_vtSnowmobileMaxDecel.GetFloat();
break;

case PPM_NEWVEHICLE:
	fValue = g_vtNewVehicleMaxDecel.GetFloat();
break;
```

In the function CVehicleMgr::GetVehicleDecelTime() add:

```
case PPM_SNOWMOBILE :
        fValue = g_vtSnowmobileDecelTime.GetFloat();
break;

case PPM_NEWVEHICLE:
        fValue = g_vtNewVehicleDecelTime.GetFloat();
break;
```

In the function CVehicleMgr::GetVehicleMaxBrake() add:

```
 case PPM_SNOWMOBILE :
        fValue = g_vtSnowmobileMaxBrake.GetFloat();
break;

case PPM_NEWVEHICLE:
        fValue = g_vtNewVehicleMaxBrake.GetFloat();
break;
```

In the function CVehicleMgr::GetVehicleBrakeTime() add:

```
case PPM_SNOWMOBILE :
        fValue = g_vtSnowmobileBrakeTime.GetFloat();
break;

case PPM_NEWVEHICLE:
        fValue = g_vtNewVehicleBrakeTime.GetFloat();
break;
```

In the function CVehicleMgr::GetVehicleMinTurnAccel() add:

```
case PPM_SNOWMOBILE :
        fValue = g_vtSnowmobileMinTurnAccel.GetFloat();
break;

case PPM_NEWVEHICLE:
        fValue = g_vtNewVehicleMinTurnAccel.GetFloat();
break;
```

That's it for adding code for getting the values specified in console variables. Now you should add code to the functions that are looking for specific sounds. You will need to add the following code.

In the function CVehicleMgr::GetIdleSnd() add:

```
case PPM_SNOWMOBILE :
        return "Snd\\Vehicle\\Snowmobile\\idle.wav";
break;

case PPM_NEWVEHICLE:
        return "Snd\\Vehicle\\ NewVehicle \\idle.wav";
break;
```

In the function CVehicleMgr::GetAccelSnd() add:

```
case PPM_SNOWMOBILE :
        return "Snd\\Vehicle\\Snowmobile\\accel.wav";
break;

case PPM_NEWVEHICLE:
        return "Snd\\Vehicle\\ NewVehicle \\accel.wav";
break;
```

In the function CVehicleMgr::GetDecelSnd() add:

```
case PPM_SNOWMOBILE :
        return "Snd\\Vehicle\\Snowmobile\\decel.wav";
break;

case PPM_NEWVEHICLE :
        return "Snd\\Vehicle\\ NewVehicle \\decel.wav";
```

```
	break;
```

In the function CVehicleMgr::GetBrakeSnd() add:

```
	case PPM_SNOWMOBILE :
		return "Snd\\Vehicle\\Snowmobile\\brake.wav";
	break;

	case PPM_NEWVEHICLE :
		return "Snd\\Vehicle\\NewVehicle\\brake.wav";
	break;
```

In the function CVehicleMgr::GetImpactSnd() add:

```
	case PPM_SNOWMOBILE :
	{
		if (fCurVelocityPercent > 0.1f && fCurVelocityPercent < 0.4f)
		{
			pSound = "Snd\\Vehicle\\Snowmobile\\slowimpact.wav";
		}
		else if (fCurVelocityPercent < 0.7f)
		{
			pSound = "Snd\\Vehicle\\Snowmobile\\medimpact.wav";
		}
		else
		{
			pSound = "Snd\\Vehicle\\Snowmobile\\fastimpact.wav";
		}
	}
	break;

	case PPM_NEWVEHICLE :
	{
		if (fCurVelocityPercent > 0.1f && fCurVelocityPercent < 0.4f)
		{
			pSound = "Snd\\Vehicle\\NewVehicle\\slowimpact.wav";
		}
		else if (fCurVelocityPercent < 0.7f)
		{
			pSound = "Snd\\Vehicle\\NewVehicle\\medimpact.wav";
		}
		else
		{
			pSound = "Snd\\Vehicle\\NewVehicle\\fastimpact.wav";
		}
	}
	break;
```

Now you should have all of the sound functions that are dependent on which vehicle the player is riding up to date. You will want to make sure that the sounds actually get used and the correct ones play at the correct time. To do this add the following line to CVehicleMgr::UpdateSound():

```
	case PPM_SNOWMOBILE :
	case PPM_NEWVEHICLE :
		UpdateVehicleSounds();
	break;
```

The physics for your new vehicle should be added next. You will first need to add code for switching to and from your PPM_NEWVEHICLE PlayerPhysicsModel, which is setting up what happens when the player gets on and off your vehicle. The function, CVehicleMgr::SetPhysicsModel() gets called when the player wants to switch their PlayerPhysicsModel, such as getting on or off a vehicle. CVehicleMgr::SetPhysicsModel() proceeds to call CVehicleMgr::PreSetPhysicsModel() first to do some preliminary setup for the vehicles. You should look at this function first and add the following code for when the player first gets on the vehicle:

```
	// Check if we should disable the weapon.
	switch (eModel)
	{
		case PPM_SNOWMOBILE :
			EnableWeapon( false );
```

```
            break;

            case PPM_NEWVEHICLE:
                EnableWeapon( false );
            break;

            case PPM_LURE :
            case PPM_NORMAL :
            default :
            break;
        }
```

The above code will disable the player's weapon when getting on your vehicle. Now you will also need to add the following block of code to the same function for when the player gets off your vehicle:

```
         case PPM_SNOWMOBILE :
        {
            g_pClientSoundMgr->PlaySoundLocal("Snd\\Vehicle\\Snowmobile\\turnoff.wav",
SOUNDPRIORITY_PLAYER_HIGH, PLAYSOUND_CLIENT);
            PlayVehicleAni("Deselect");

            // Can't move until deselect is done...
            g_pMoveMgr->AllowMovement(LTFALSE);

            // Wait to change modes...
            bRet = LTFALSE;
        }
        break;

        case PPM_NEWVEHICLE :
        {
            g_pClientSoundMgr->PlaySoundLocal("Snd\\Vehicle\\NewVehicle\\turnoff.wav",
SOUNDPRIORITY_PLAYER_HIGH, PLAYSOUND_CLIENT);

            PlayVehicleAni("Deselect");

            // Can't move until deselect is done...
            g_pMoveMgr->AllowMovement(LTFALSE);

            // Wait to change modes...
            bRet = LTFALSE;
        }
        break;
```

The above code will play a sound for turning off the engine of your vehicle and will play the deselect animation on the player view model showing the player they are getting off the vehicle. Getting back to CVehicleMgr::SetPhysicsModel() you will need to add code for actually setting the physics model for your new vehicle. Add the following code to CVehicleMgr::SetPhysicsModel():

```
        case PPM_SNOWMOBILE :
        {
            SetSnowmobilePhysicsModel();

            if (g_vtSnowmobileInfoTrack.GetFloat())
            {
                g_pLTClient->CPrint("Snowmobile Physics Mode: ON");
            }
        }
        break;

        case PPM_NEWVEHICLE:
        {
            SetNewVehiclePhysicsModel();
        }
        break;
```

Note that you are now making a call to a function that does not yet exist. You will need to write this function next. Open the header file for the class CVehicleMgr, \Game\ClientShellDLL\ClientShellShared\VehicleMgr.h and add the following line to the CVehicleMgr's protected function definitions:

```
void            SetSnowmobilePhysicsModel();
void            SetPlayerLurePhysicsModel( );
void            SetNormalPhysicsModel();
void            SetNewVehiclePhysicsModel();
```

Now you will need to go back to \Game\ClientShellDLL\ClientShellShared\VehicleMgr.cpp and write the actual function CVehicleMgr::SetNewVehicelPhysicsModel(). Add the following function:

```
void CVehicleMgr::SetNewVehiclePhysicsModel()
{
        if (!m_hVehicleStartSnd)
        {
                uint32 dwFlags = PLAYSOUND_GETHANDLE | PLAYSOUND_CLIENT;
                m_hVehicleStartSnd = g_pClientSoundMgr->PlaySoundLocal
("Snd\\Vehicle\\NewVehicle\\startup.wav", SOUNDPRIORITY_PLAYER_HIGH, dwFlags);
        }

        CreateVehicleModel();

        m_vVehicleOffset.x = g_vtNewVehicleOffsetX.GetFloat();
        m_vVehicleOffset.y = g_vtNewVehicleOffsetY.GetFloat();
        m_vVehicleOffset.z = g_vtNewVehicleOffsetZ.GetFloat();

        // Reset acceleration so we don't start off moving...

        m_fVehicleBaseMoveAccel = 0.0f;

        ShowVehicleModel(LTTRUE);

        PlayVehicleAni("Select");

        // Can't move until select ani is done...

        g_pMoveMgr->AllowMovement(LTFALSE);

        m_bSetLastAngles = false;
        m_vLastPlayerAngles.Init();
        m_vLastCameraAngles.Init();
        m_vLastVehiclePYR.Init();
}
```

The above function will play an initial sound for starting up your vehicle, it will then create the vehicle model that you setup before, play a select or getting on animation and finally resets some data members. You still won't be able to ride your vehicle at this point but you are getting closer to getting everything setup. Next you will need to add a few more lines of code to functions that deal with the motion of the vehicle. Add the following line to CVehicleMgr::MoveLocalSolidObject( ):

```
// These guys don't handle movement.
case PPM_NORMAL:
case PPM_SNOWMOBILE:
case PPM_NEWVEHICLE:
        return false;
        break;
```

Add this line to CVehicleMgr::MoveVehicleObject():

```
// These guys don't handle movement.
case PPM_SNOWMOBILE:
case PPM_NEWVEHICLE:
```

You probably won't need to do any camera displacement so add the following line to CVehicleMgr::CalculateVehicleCameraDisplacment():

```
case PPM_NORMAL:
case PPM_SNOWMOBILE:
case PPM_NEWVEHICLE:
case PPM_LIGHTCYCLE:
        return;
```

```
        break;
```

Now you will need to make sure your vehicle is able to accelerate, brake and turn when the player presses those commands. You will need to make sure the commands are properly read. Add the following line to CVehicleMgr::UpdateControlFlags():

```
case PPM_NEWVEHICLE :
case PPM_SNOWMOBILE :
        UpdateSnowmobileControlFlags();
break;
```

Notice that your new vehicle is calling the function for updating the snowmobile control flags. Since the behavior you want is similar to the snowmobile and you will want to check the same control flags this is perfectly fine. If you decide you would want to have different behavior for your new vehicle then you would create a separate function to read in the control flags while riding your new vehicle.

Now you need to make sure the vehicle actually moves. Add the following line to CVehicleMgr::UpdateMotion():

```
case PPM_SNOWMOBILE :
case PPM_NEWVEHICLE :
        UpdateVehicleMotion();
break;
```

Add the following line to CVehicleMgr::UpdateVehicleGear():

```
case PPM_SNOWMOBILE :
case PPM_NEWVEHICLE :
        UpdateSnowmobileGear();
break;
```

And the following line to CVehicleMgr::UpdateFriction():

```
case PPM_SNOWMOBILE :
case PPM_NEWVEHICLE :
        UpdateVehicleFriction(SNOWMOBILE_SLIDE_TO_STOP_TIME);
break;
```

That takes care of updating the movement for your new vehicle. Now you'll want to make sure the rotation gets updated properly. Add the following code to CVehicleMgr::CalculateVehicleRotation():

```
case PPM_SNOWMOBILE:
        CalculateBankingVehicleRotation( vPlayerPYR, vPYR, fYawDelta );
        CalculateVehicleContourRotation( vPlayerPYR, vPYR );
        break;

case PPM_NEWVEHICLE:
        CalculateBankingVehicleRotation( vPlayerPYR, vPYR, fYawDelta );
        CalculateVehicleContourRotation( vPlayerPYR, vPYR );
        break;
```

That takes care of all the steps needed to get the physics working on your new vehicle. You may want to change some of the console variable values until you get the feel of the vehicle you want. You might also want to adjust how the vehicle banks, rotates and contours depending on what type of vehicle you are adding.

Player

The final steps in adding a new vehicle involve how the player interacts with the vehicle. The CPlayerObj also needs to change some behavior depending on what PlayerPhysicsModel it is in. Open \Game\ObjectDLL\ObjectShared\PlaeryObj.cpp and change the following lines of code.

Just like CVehicleMgr needs to change the physics model so does CPlayerObj. Add the following line to CPlayerObj::SetPhysicsModel():

```
case PPM_SNOWMOBILE:
case PPM_NEWVEHICLE:
case PPM_LIGHTCYCLE:
        SetVehiclePhysicsModel(eModel);
break;
```

This will let the player know they are actually riding a vehicle.

You will also need to make sure that other players, when in a multiplayer game, know that the player is riding a vehicle. Change the following line in CPlayerObj::SetPhysicsModel():

```
if (m_ePPhysicsModel == PPM_SNOWMOBILE)
```

To:

```
if( (m_ePPhysicsModel == PPM_SNOWMOBILE) || (m_ePPhysicsModel == PPM_NEWVEHICLE) )
```

This will set the USRFLG_PLAYER_SNOWMOBILE user flag on any player that is riding your vehicle. This is fine because this flag is just used to see if the player is on a vehicle, not just the snowmobile. If you need to distinguish a difference between the snowmobile and your new vehicle, you will need to add a new user flag (none is currently available) or change how this is handled by going through the SpecialFX message.

CPlayerObj also needs to keep track of where the vehicle is and let the AI know its on a vehicle. Add the following line to CPlayerObj::UpdateVehicleMovement():

```
case PPM_SNOWMOBILE:
case PPM_NEWVEHICLE:
```

CPlayerObj controls what animation the player should be in while riding vehicles so you'll have to make sure

Conclusion

You have now just added a new vehicle to NOLF2. Now that you are familiar with the vehicle systems, you should be able to change some values and create the vehicle you want. You will want to create new sounds, models and skins for your vehicle so it looks the way you want. Adding more vehicles should now be very simple for you and you could even create a mod with several different vehicles to choose form. Have fun!

Content Guide

Adding and Modifying Weapons in No One Lives Forever 2

Overview

This document will describe how to modify existing weapons and adding new ones to the game No One Lives Forever 2 (NOLF2). It will go over the necessary properties for the weapons, mods and ammo and how to add ammo and mods to a weapon. The main file used for the weapon, ammo and mod definitions is *weapons.txt* found in your NOLF2\Game\Attributes directory. This file will also take a look at *FX.txt* so you know how to create and modify the special FX for the weapons.

Weapons.txt

The first section you will see in weapons.txt is the WeaponOrder list. The WeaponOrder list allows you to specify the order of all weapons and gadgets used in the game and is also used to categorize the weapons into classes. The class numbers refer to the NextWeapon_XX command binding. The NextWeapon_XX command will cycle through all the weapons in the XX class. When adding or changing weapons you may want to reorganize the weapon classes so like weapons are grouped together. The last entries in the WeaponOrder list are the non-player weapons, weapons that are only used by AI.

After the WeaponOrder list is the AutoSwitch list. This is used to set a priority to weapons when auto switching. When the player picks up a weapon the AutoSwitch list is used to determine if they should switch to that weapon. Leave any weapons off the list that you feel the player would never want that game to automatically switch to.

MP_Weapons.txt

NOLF2 SinglePlayer and Cooperative game types strictly used weapons.txt for all of its weapon definitions. The other multiplayer game types, Deathmatch, TeamDeathmatch and DoomsDay also used an override file, mp_weapons.txt the allows you to override certain properties from weapons.txt for use in those game types. This allowed us to balance singleplayer and multiplayer game types separately. Only the properties that are going to be overridden need to be listed in the weapon definition in mp_weapons.txt. (see mp_weapons.txt for more info).

Ammo

The ammo section is where all of the ammo definitions are listed. The order of the ammo list is in order of creation and doesn't pertain to the weapons that use the ammo or the priority of the ammo. When creating a new ammo definition you must add it to the end of the already existing list with a tag that is one greater than the previous definition. If you wish to edit an existing ammo definition, you may change any of the properties, listed below, to the desired value. However, you cannot change the name of an ammo definition since that name is used to reference the ammo in the weapon definitions. If you wish to edit any property that is an ID of a localized string (stored in CRes.dll) or when adding a new ammo definition you must edit the string table in CRes.dll and rebuild the game code. (see the " Building the No One Lives Forever 2 Game" document on how to build the game.)

Ammo Properties

Name	(string)	This is the user friendly name that is used to reference the ammo in the weapon definitions and from within DEdit
NameId	(integer)	This is an ID that maps to a localized string for the name of the ammo (seen in game).
DescId	(integer)	This is an ID that maps to a localized string for the description of the ammo.
Type	(integer)	Specifies the type of ammo. 0 – Vector (bullet) 1 – Projectile (grenade) 3 – Gadget (code breaker)
Icon	(string)	File path and name of the interface icon fro the ammo that will be displayed when the ammo is selected.
MaxAmount	(integer)	Specifies the maximum amount of ammo that the player can carry at any given time.
SelectionAmount	(integer)	Specifies the amount of this ammo in a weapon when a weapon is picked up in game.
SpawnedAmount	(integer)	Specifies the amount of ammo that will be in an ammo box when this ammo spawns in an ammo box. (level designers can override this value).
InstDamage	(integer)	Specifies the amount of Instantaneous damage done by this type of ammo.
InstDamageType	(string)	Specifies the type of Instantaneous

		damage done by this type of ammo. (See valid DamageTypes in DamageTypes.txt).
AreaDamage	(integer)	Specifies the Max amount of Area damage done by this type of ammo (i.e., damage done if you are in the center of the damage).
AreaDamageType	(string)	Specifies the type of Area damage done by this type of ammo. (See valid DamageTypes in DamageTypes.txt).
AreaDamageRadius	(integer)	Specifies the radius of effect for the AreaDamage.
ProgDamage	(float)	Specifies the amount of Progressive damage done by this type of ammo. This is the amount of damage that is done per second.
ProgDamageType	(string)	Specifies the type of Progressive damage done by this type of ammo. (See valid DamageTypes in DamageTypes.txt).
ProgDamageDuration	(float)	Specifies how long the Progressive damage is applied to the victim.
ProgDamageRadius	(float)	Specifies the radius of effect for the Progressive damage (only used if ProgDamageLifetime is greater than 0.0).
ProgDamageLifetime	(float)	Specifies the time the progressive damage radius of effect lasts (for things like poison gas).
FireRecoilMult	(float)	This multiplier modifies the FireRecoilPitch value in a weapon firing this type of ammo (see **FireRecoilPitch** in Weapon definition below)
ImpactFXName	(string)	Specifies the impact special fx. The value of this property MUST correspond to the name of an ImpactFX specified in the FX.txt file.
UWImpactFXName	(string)	Specifies the under water impact special fx. The value of this property MUST correspond to the name of an ImpactFX specified in the FX.txt file.
ProjectileFXName	(string)	Specifies the a projectile special fx. The value of this property MUST correspond to the name of a ProjectileFX specified in the FX.txt file.
MoveableImpactOverrideFXName	(string)	Specifies an effect to play if the impact occurs on a moveable object, such as a model or a door. This way you can have effects that 'stick' but not stick to moveable objects so they won't hang in the air.
FireFXName	(string)	Specifies a fire special fx. The value of this property MUST correspond to the name of a FireFX specified in the FX.txt file.
TracerFXName	(string)	Specifies a tracer special fx. The value

		of this property MUST correspond to the name of a TracerFX specified in the FX.txt file.
WeaponAniOverride	(string)	Not used in NOLF2 and is unsupported
CanBeDeflected	(integer)	Ammo can be deflected by weapon. To deflect, the weapon must have a model string key "begindeflect" followed by a key "enddeflect".
DeflectSurfaceType	(string)	When ammo is deflected, the impact fx used will be based on this surface type. Surface types found in surface.txt.
CanAdjustInstDamage	(integer)	If set to 1 the InstDamage amount may be adjusted programatically, if set to 0 the value specified by InstDamage will always be used.
CanServerRestrict	(integer)	If set to 0 to not allow server options to restrict this item from the level. Default: 1.

Weapons

The weapon section is where all of the weapon definitions are listed. The order of the weapon list is in order of creation and doesn't pertain to the WeaponOrder list mentioned above. When creating a new weapon definition you must add it to the end of the already existing list with a tag that is one greater than the previous definition. If you wish to edit an existing weapon definition, you may change any of the properties, listed below, to the desired value. However, you cannot change the name of a weapon definition since that name is used in the WeaponOrder list and within DEdit. If you wish to edit any property that is an ID of a localized string (stored in CRes.dll) or when adding a new weapon definition you must edit the string table in CRes.dll and rebuild the game code. (see the " Building the No One Lives Forever 2 Game" document on how to build the game.)

Weapon Properties

Name	(string)	User friendly name used in WeaponOrder above and in DEdit.
NameId	(integer)	Id that maps to the weapon Localizable name (stored in CRes.dll)
DescriptionId	(integer)	Id that maps to the weapon localized description (stored in CRes.dll)
IsAmmoNoPickupId	(integer)	Id that maps to the string that is displayed for IsAmmo = 1 type weapons when the weapon can not be picked up (stored in CRes.dll)
ClientWeaponType	(integer)	Weapon type: 0 - none (NEVER use this number) 1 - vector (bullet type weapons) 2 - projectile (like crossbows)
AniType	(integer)	Classify the weapon into a style of animation when holding the weapon. Players and AI will hold their weapons depending on the animation type of the weapon. Values : 0 - rifle 1 - pistol 2 - melee 3 - Throw

		4 - Rifle2 5 - Rifle3 6 - Holster 7 - Place 8 - Gadget 9 - Throwing Stars
Icon	(string)	Filename of the texture used for an icon associated with the weapon. for Normal icons ".dtx" is appended to the name. for Unselected icons "U.dtx" is appended to the name. for Disabled icons "D.dtx" is appended to the name. for Silhouette icons "X.dtx" is appended to the name.
Pos	(vector)	The player-view weapon model pos (offset from player model in x, y, and z)
MuzzlePos	(vector)	The player-view weapon model muzzle pos (offset from the player model in x, y, and z). This is used to determine where to place muzzle fx.
BreachOffset	(vector)	The player-view weapon breach offset which is relative to the above model muzzle pos. This is used to determine where shell casings will be ejected from the player-view weapon.
MinPerturb	(integer)	The minimum amount of perturb on each vector or projectile fired by the weapon. A value in range from MinPerturb to MaxPerturb is added to the Up and Right components of the firing direction vector based on the current accuracy of the player/ai firing the weapon.
MaxPerturb	(integer)	The maximum amount of perturb on each vector or projectile fired by the weapon. A value in range from MinPerturb to MaxPerturb is added to the Up and Right components of the firing direction vector based on the current accuracy of the player/ai firing the weapon.
Range	(integer)	The range of the weapon. The weapon is ineffective if shooting at targets beyond the range.
Recoil	(vector)	The amount the weapon will recoil when fired.
VectorsPerRound	(integer)	The number of vectors (rays cast) per round when the weapon is fired. Used for things like a shotgun.
PVModel	(string)	File path and name of the player-view weapon model.
PVSkin0-N	(string)	Name of the player-view weapon skin(s) (NOTE: You can specify "Hands" if you want to use the player-view hands texture associated with the current player model

		style). There can be any number of skins: PVSkin0, PVSkin1, …, PVSkinN. The PVSkinX number relates to the texture indices in the piece info dialog in ModelEdit.
PVRenderStyle0-N	(string)	Name of the render style file ltb(s). There can be any number of render styles: PVRenderStyle0, PVRenderStyle1, ..., PVRenderStyleN. The PVRenderStyleX number relates to the renderstyle index in the piece info dialog in ModelEdit.
PVMuzzleFXName	(string)	Specifies the optional player-view muzzle fx for this weapon. The value of this property MUST correspond to a name specified in the MuzzleFX definitions in the Attributes\fx.txt file.
PVAttachFXName0-10	(string)	Specifies the optional player-view fx attachment for this weapon. The value of this property MUST correspond to an effect created with FXEd.
HHModel	(string)	File path and name of the hand-held (and powerup) weapon model.
HHSkin0-N	(string)	Name of the hand-held (and powerup) weapon skin(s). There can be any number of skins: HHSkin0, HHSkin1, ..., HHSkinN. The HHSkinX number relates to the texture indices in the piece info dialog in ModelEdit.
HHRendeStyle0-N	(string)	Name of the render style file ltb(s). There can be any number of render styles: RenderStyle0, HHRenderStyle1, ..., HHRenderStyleN. The HHRenderStyleX number relates to the renderstyle index in the piece info dialog in ModelEdit.
HHModelScale	(vector)	The scale of the hand-held (and powerup) weapon model.
HHMuzzleFXName	(string)	Specifies the optional hand-held muzzle fx for this weapon. The value of this property MUST correspond to a name specified in the MuzzleFX definitions in the Attributes\fx.txt file.
FireSnd	(string)	Name of the sound that is played when the weapon is fired.
AltFireSnd	(string)	Name of the sound that is played when the weapon is alt-fired. (**ALT Fire was not used in NOLF 2 and is unsupported**)
SilencedFireSnd	(string)	Name of the sound that is played when the weapon is fired with a silencer.
DryFireSnd	(string)	Name of the sound that is played when the weapon is dry fired.
ReloadSnd1-3	(string)	Name of the sound that is played when the weapon is reloaded. (mapped to SOUND_KEY 1 – SOUND_KEY 3)
SelectSnd	(string)	Name of the sound that is played when the weapon is selected. (mapped to SOUND_KEY 4)
DeselectSnd	(string)	Name of the sound that is played when the weapon is deselected. (mapped to SOUND_KEY 5)

MiscSound1-5	(string)	Optional miscellaneous sounds to play during weapon animations. (mapped to SOUND_KEY 10 – SOUND_KEY 14)
FireSndRadius	(integer)	The radius of volume falloff that fire sounds are played at
AIFireSndRadius	(integer)	The radius of volume that fire sounds make, w.r.t to AI's senses
WeaponSndRadius	(integer)	The radius of volume falloff that all other sounds, non-fire sounds, are played at
EnvironmentMap	(integer)	Specifies whether or not the weapon uses environment mapping.
InfiniteAmmo	(integer)	Specifies whether or not the weapon has unlimited ammo. (1 = true, 0 = false). Things like swords should have unlimited ammo.
ShotsPerClip	(integer)	The number of rounds the weapon can fire before it must be reloaded.
FireRecoilPitch	(float)	Specifies the amount the weapon should pitch (in degrees) when fired.
FireRecoilDelay	(float)	Specifies the time (in seconds) it should take for the weapon's pitch to get back to normal after recoiling from firing.
AmmoName#	(string)	Specifies the type of ammo available for this weapon. The value of this property MUST correspond to a name of an AMMO definition. If the weapon uses multiple ammo types, you may specify multiple AmmoName properties. For example: AmmoName0 = “9mm fmj” AmmoName1 = “9mm dum dum” Note: The first (or only) ammo type specified is considered the “default” ammo type. Every weapon MUST have at least one ammo type specified.
ModName#	(string)	Specifies the optional mods available for this weapon. The value of this property MUST correspond to the name of a MOD definition. If the weapon uses multiple mods, you may specify multiple ModName properties. For example: ModName0 = “silencer” ModName1 = “scope”
PVFXName#	(string)	Specifies the optional player-view fx available for this weapon. The value of this property MUST correspondd to a name specified in the PVFX definitions in the Attributes\fx.txt file. If the weapon uses multiple pv fx, you may specify multiple PVFXName properties.
AIWeaponType	(integer)	Classify the weapon for use by AI Values : 0 - None 1 - Ranged 2 - Melee 3 - Thrown
AIMinBurstInterval	(float)	Specifies the range of time (in seconds) an AI

		will wait in between firing a burst from a weapon. IE, the AI will shoot somewhere between Min/AIMaxBurstShots, then wait Min/AIMaxBurstInterval seconds, then shoot again.
AIMaxBurstInterval	(float)	
AIMinBurstShots	(integer)	Specifies the range of shots an AI will fire when it is firing a burst from a weapon. IE, the AI will shoot somewhere between Min/AIMaxBurstShots, then wait Min/AIMaxBurstInterval seconds, then shoot again.
AIMaxBurstShots	(integer)	
AIAnimatesReload	(integer)	Specifies whether or not an AI animates reloading this weapon. 1 = true, 0 = false)
LooksDangerous	(integer)	Specifies whether or not the weapon/gadget looks dangerous to AIs. (1 = true, 0 = false)
FireDelay	(integer)	Specifies the minimum delay in milliseconds between fire keys. The server will not allow shots to fire quicker than the FireDelay.
HideWhenEmpty	(integer)	Specifies whether or not the weapon/gadget should be hidden when out of ammo (also implies that the deselect animation shouldn't be played when auto-switching from this weapon when it runs out of ammo) (1 = true, 0 = false)
IsAmmo	(integer)	Specifies whether or not the weapon/gadget should be treated as ammo. In other words is the weapon placed/thrown (e.g., grenade, bear trap, etc.). If set to true the weapon's pick-up message won't be displayed when the weapon is picked up unless the weapon can't be picked up. If the weapon can't be picked up the IsAmmoNoPickupId string is displayed. This should be used for "thrown" weapons so the player doesn't see both the weapon pickup message AND the ammo pick up message. (1 = true, 0 = false)
HolsterAttachment	(string)	Specifies node and model to attch when weapon is holstered. For example: HolsterAttachment = "BACK AK47_holster"
HiddenPiece0-N	(string)	Names of Gadget model pieces to hide/show when the gadget is fired. There can be any number of pieces to hide/show.
FireAnimRateScale	(float)	Speeds up or slows down the rate of the Fire animation. By default the scale is 1.0 which will play the animation at normal speed, 2.0 will double the speed, and 0.5 will half the speed.
ReloadAnimRateScale	(float)	Speeds up or slows down the rate of the Reload animation. By default the scale is 1.0 which will play the animation at normal speed, 2.0 will double the speed, and 0.5 will half the speed.
PowerupFX	(string)	Name of the FxED created FX to be played when the item respawns. This FX will constantly loop as long as the item is waiting

		to be picked up.
RespawnWaitVisible	(integer)	(0 or 1) If 0 the item will become invisible once it is picked up and is waiting to respawn. Otherwise the item will remain visible.
RespawnWaitFX	(string)	Name of an FxED created FX to play once the item is picked up and is waiting to respawn.
RespawnWaitSkin0-N	(string)	Name of the skin files that will be associted with the mode once it is picked up and is waiting to respawn.
RespawnWaitRenderstyle0-N	(string)	Name of the renderstyle that will be associated with the model once it is pickedup and is waiting to respawn.
CanServerRestrict	(integer)	If set to 0 to not allow server options to restrict this item from the level. Default: 1.

Mods

The mod section is where all of the mod definitions are listed. The order of the mod list is in order of creation and doesn't pertain to the weapons that use the mod or the priority of the mod. When creating a new mod definition you must add it to the end of the already existing list with a tag that is one greater than the previous definition. If you wish to edit an existing mod definition, you may change any of the properties, listed below, to the desired value. However, you cannot change the name of a mod definition since that name is used in the weapon definitions and within DEdit. If you wish to edit any property that is an ID of a localized string (stored in CRes.dll) or when adding a new mod definition you must edit the string table in CRes.dll and rebuild the game code. (see the " Building the No One Lives Forever 2 Game" document on how to build the game.)

Name	(string)	User friendly name used in DEdit.
Socket	(string)	Name of the weapon model socket to attach the mod model to. You can add and edit sockets on a model within ModelEdit.
NameId	(integer)	Id that maps to the localized string for the name of the Mod.
DescriptionId	(integer)	Id that maps to the localized string for the description of the mod.
Icon	(string)	Name of the interface icon associated with the ammo type.
Type	(integer)	Specifies the type of mod. Valid values are: 0 – Silencer 1 – Scope
AttachModel	(string)	File path and name of the model attachment associated with the mod.
AttachSkin0-N	(string)	Name of the skin(s) associated with the mod attachment model. There can be any number of skins: AttachSkin0, AttachSkin1, …, AttachSkinN
AttachRenderStyle0-N	(string)	Name of the render style file ltb(s). There can be any number of render styles: AttachRenderStyle0, AttachRenderStyle1, …, AttachRenderStyleN
AttachScale	(vector)	Scale of the mod attachment model.
ZoomLevel	(integer)	Level the weapon can zoom to (Valid levels are 1, 2, and 3. 0 = can't zoom). Ignored for non-scope mods.

NightVision	(integer)	Does the mod provide night vision (1 = true, 0 = false). Ignored for non-scope mods. **(Night vision was not used in NOLF2 and is unsupported)**
Integrated	(integer)	Is the mod built into the weapon (1 = true, 0 = false). If this is true, any weapon that lists this mod in its list of mods will always have this mod.
Priority	(integer)	This is used for weapons that have more than one mod of the same type (e.g., two scopes). This field indicates which should be used if the weapon has both mods (the highest priority is used). NOTE: this is a number between 0 and 255, that should default to 0.
ZoomInSound	(string)	Name of the sound that is played when the weapon is zoomed in.
ZoomOutSound	(string)	Name of the sound that is played when the weapon is zoomed out.
PickUpSound	(string)	Name of the sound that is played when the mod is picked up (either absolute path to sound or name of soundbutes sound)
RespawnSound	(string)	Name of the sound that is played when the mod respawns in multiplayer (either absolute path to sound or name of the soundbutes.txt sound)
PowerupModel	(string)	Name of the powerup model associated with the mod. This is what players see in the game when picking up the mod.
PowerupSkin0-N	(string)	ame of the powerup skin(s) associated with the mod. There can be any number of skins: PowerupSkin0, PowerupSkin1, ..., PowerupSkinN
PowerupRenderStyle0-N	(string)	Name of the render style file ltb(s). There can be any number of render styles: PowerupRenderStyle0, PowerupRenderStyle1, ..., PowerupRenderStyleN
PowerupModelScale	(float)	The scaling factor used with the powerup model. (same model as attach model)
TintColor	(vector)	Color to tint the screen when mod is picked up (r, g, b)
TintTime	(float)	Duration in seconds of screen tint when gear is picked up
AISilencedFireSndRadius	(integer)	The radius of volume that silenced fire sounds make, w.r.t to AI's senses NOTE: this value only works with Type = 0 mods.
PowerupFX	(string)	Name of the FxED created FX to be played when the item respawns. This FX will constantly loop as long as the item is waiting to be picked up.
RespawnWaitVisible	(integer)	(0 or 1) If 0 the item will become invisible once it is picked up and is waiting to respawn. Otherwise the item will remain visible.
RespawnWaitFX	(string)	Name of an FxED created FX to play once the item is picked up and is waiting to

		respawn.
RespawnWaitSkin0-N	(string)	Name of the skin files that will be associted with the mode once it is picked up and is waiting to respawn.
RespawnWaitRenderStyle0-N	(string)	Name of the renderstyle that will be associated with the model once it is pickedup and is waiting to respawn.

Creating New Weapons

When adding a new weapon here are a few things to keep in mind. Your model will need a select, deselect, reload, idle_0 and fire animations. You may also have idle_1 and idle_2 for multiple idle animations, these animations are played less often and may be more pronounced. You can also have prefire and postfire animations that, if present, will play before and after the fire animation. These animations can be used to start and stop looping sounds, be used to show a ramp up and ramp down time for the weapon and may also be used to simulate holding an object to throw, like the grenades. In order for the weapon to actually fire a round it must contain a fire_key frame string on one of the keyframes of the model for one of the prefire, fire, or postfire animations. All the valid framestrings and what the are used for are listed below.

FIRE_KEY	Lets the weapon know when to fire.
SOUND_KEY (integer)	Tells the weapon to play a sound that is associated with the given number. Valid numbers and their mapping to the sound files in the weapon definition are: 1 – ReloadSnd 2 – ReloadSnd2 3 – ReloadSnd3 4 – SelectSnd 5 – DeselectSnd 6 – FireSnd 7 – DryFireSnd 8 – AltFireSnd 9 – SilencedFireSnd 10 – MiscSnd1 11 – MiscSnd2 12 – MiscSnd3 13 – MiscSnd4 14 – MiscSnd5
BUTE_SOUND_KEY (string)	Tells the weapon to play a buted sound of the given name. (see soundbutes.txt for more info on soundbutes)
LOOP_SOUND_KEY (integer)	Same as SOUND_KEY except the sound will begin to loop. To stop the looping sound use the frame string “LOOP_SOUND_KEY stop”. Usually you want the sound to begin looping during its prefire animation and stop during its postfire animation.
FX (string)	Play the specifed FxEd created fx.
FIREFX_KEY (string)	Fires the weapon and plays the specified FxEd create FX. Same as doing “FIRE_KEY; FIREFX_KEY (fx name)”
FLASHLIGHT (string)	Lets the weapon turn the flashlight on and off. Valid strings: ON OFF
DEFLECT (float)	When a weapon hits this key it specifies the

	time, in seconds, that this weapon can deflect hits from other weapons that are deflecting.
HIDE_PIECE_KEY (string)	Sets the specified piece name invisible.
SHOW_PIECE_KEY (string)	Sets the specified piece name visible again.
HIDE_PVATTACHFX (integer)	Hides specifed PVAttachFXName# on the weapon
SHOW_PVATTACHFX (integer)	Shows the specified PVAttachFXName#.
HIDE_PVATTACHMENT (integer)	Hides the specified PlayerViewAttachmentX (set on a per player model basis. See modelbutes.txt for more info)
SHOW_PVATTACHMENT (integer)	Shows the specified PlayerViewAttachmentX. (set on a per player model basis. See modelbutes.txt for more info)

Content Guide

Working with NOLF2 models

Basic model creation

All of the models in NOLF2 were modeled, textured and animated in Maya 4.0. We currently have working exporters for Maya3.0/4.0, and 3dStudioMax3.0/4.0. All of the basics of model creation mentioned here are specific to Maya, but the concepts apply to Max-created models.

Models and Polycounts

Models should be created using polygons. There are limits on the number of polygons of a particular model depending on the function and necessary detail level of that model. In NOLF2, the character models ranged from 1700 to 2700 triangles. This is a good range for most characters, but the limit could go as high as 4-5000 triangles in isolated situations. In addition to the main models, NOLF2 used hand-built level-of-detail (LOD) models for most of the gameplay characters (and all of the multiplayer models). The rule of thumb used was to create two duplicate models and reduce on to 60% and the other to 25% of the original model's polycount. LODs are not necessary, but are an optimization that will improve rendering speed when many models are onscreen simultaneously. More will be mentioned later about setting up LODs for a model.

Textures and RenderStyles

Textures can be created using any digital painting tool and should have dimensions that are a power of 2 pixels in length (…64,128,256, 512…). Model textures are generally kept square, but rectangular shaped textures are ok as long as the dimensions fill this requirement. A Photoshop plugin is included to allow creation of Jupiter's DTX texture format. In addition, plugins for Maya and Max are included to allow the use of DTX textures (Maya: IMFLithDTX.dll, MAX: MaxDTXLoader40.bmi). A model can theoretically have up to 16 textures, but each part of the model using a different texture needs to be separated into a unique piece. A model can be made up of as many as 64 separate pieces, but keeping as few as possible is desirable to avoid slowing model rendering down. Most NOLF2 characters have only two pieces, a head and a body, for this reason. In addition to simple textures, each piece can have a unique "render style". There are a lot of these that were created for very specific purposes, but the ones most

commonly used for models are: envmap.ltb (for adding an alpha masked environment map, like with super soldiers), glass.ltb (for creating gray-scale alpha masked transparency, like eyeglasses, and even Cate's eyelashes), transparent.ltb (for creating things like trees with alpha mask), furfins.ltb and furshell.ltb (basically another way of doing alpha masked transparency, used for things like Armstrong's beard). There isn't currently a way to duplicate the effects of these in Maya, but they're not too bad to set up by just checking them in the game. One model piece can have only one render style, and usually pieces are combined based on the way model's render styles are distributed.

Skeletons

After creating the model and textures, the model geometry is bound to a skeleton. In Maya, this consists of a hierarchy of one or more joints. All Maya models should be attached to skeletons using 'smooth bind' in the Skin menu. By default, the exporter will only export joints that are bound to geometry, or the parents of such joints. In some cases it is necessary have other joints in the hierarchy exported. This is done by creating a Boolean attribute called 'ForcedNode' on the joint and setting its value to 1. Every exported model must have at least one joint. Inanimate models, such as lamps, chairs, etc. will need only one joint. Most NOLF2 characters have 35. Again it's possible to use a lot of joints, but keeping the number as low as possible will improve performance. Model LODs can be made even more efficient by removing unneeded joints from the lower detail models. This can be done in Maya by using the 'Edit Smooth Skin > Remove Influence' function. A sample Maya4.0 scene has been provided to use as a starting point for further experimentation (Tools\Docs\Character_sample.mb). This file is exactly the same as the multiplayer version of the pilot model (\chars\models\multi\pilot.ltb). (Unfortunately, we have no sample MAX model scenes.)

Exporting models

To use the exporter plugin in Maya, place the file 'MayaModelExport40.mll' in the Maya bin\plugins\ folder and the file 'LithTechModelExportOptions.mel' in the Maya scripts\others\ folder. Make sure the plugin is loaded in Maya's Plugin Manager window.

The exporter plugin exports a text version of the Lithtech file format, '.lta'. It will also export a compressed version, '.ltc', to save disk space. The format should be specified in the filename supplied to the plugin. The exporter uses the command line format:

lithtechModelExport –all [-usePlaybackRange <*bool*> -append <*bool*> -ignoreBindPose <bool> -exportWorldNode <*bool*> -scale <*float*>] <animation name> <file name>

The optional flags are defined as follows:

- **-usePlaybackRange** – setting this to '1' will export only the visible range of an animation. '0' will export all keyframes in the scene.
- **-append** – setting this to '1' will append the animation to an existing file, allowing models to contain many animations. Setting it to '0' (the default) will overwrite the file if it exists.
- **-ignoreBindPose** – This option determines whether the bind pose of a model or the first frame of an animation is used as a base for deformation. This was set to '1' for nearly all NOLF2 models because of the child model workflow described below.
- **-exportWorldNode** – This allows multiple skeletons in a scene to be exported as one model if set to '1'. It adds a 'world node' as a parent to all exported skeletons. It was generally set to '0' for NOLF2 models.
- **-scale** – this will scale a model by the specified float value. This can be helpful since it can be tricky to scale animations in Maya.
- **animation name** – the name of an animation. When appending animations, if there is an existing animation of the same name, it will need to be deleted in ModelEdit before it can be used.

file name – this should contain the entire path to the desired file. The format should be either '.lta', or '.ltc".

The exporter will use these options to decide which skeleton(s) should be exported, and will include all visible geometry bound to it.

The concept of 'child models' was used for most characters in NOLF2. The child model is a model that is used solely to store animations (and animation specific data). When a child model is applied to another model, that model will share all the animation data from the child model. This allows continual edits to be made on models without needing to worry about re-exporting large numbers of animations. It also allows many different models to use the same animation set. The base character should be exported unanimated at its bind pose with the animation called 'base'. A child model is created by using the 'append' option in the exporter. Be very careful, since accidentally not setting the append flag will simply overwrite the file without warning. Also, when joint names are case-specific when appending and must match exactly the names of the model.

A Maya script that eases the export/model setup workflow described here was created by the NOLF2 animator. It creates an interface to save many of the settings described here and in the next section in a Maya scene. It was used for nearly all the models that were exported throughout production and is presented here as is… it worked for me - I hope it works for others. If nothing else it provides a starting point for others to understand how the lta format can be used. It is described mode fully in another section.

Model setup using ModelEdit

After exporting a character model, a few things need to be done to get it to appear in the game. These things are done in ModelEdit. This tool has been around for a few years and hasn't changed a lot, in spite of our workflow changing a lot. What that means is that there are a lot of options that probably haven't been used in a while and may be obsolete.

The new Lithtech file format exists in two forms. The exporter creates a user-friendly data file (lta or ltc), but this needs to be compiled as an ltb file for use in the engine. An lta file is a text file containing all the model and animation data. An ltc file is a compressed version of this, and is what was used for most NOLF2 models. ModelEdit only reads lta or ltc files, not ltb files. ModelEdit can be used to convert between ltc and lta files. In addition to this, a command line tool, LTC.exe, can be used to convert between lta and ltc files. For help using this tool, execute the command 'LTC'. ModelPacker.exe is used by ModelEdit to create ltb files.

There are several things that need to be set up in ModelEdit to get models working. As an example, a fully setup character, Game\Chars\Models\Character_Sample.lta, has been provided that is the exported version of the Maya scene Character_Sample.mb.

User Dims

The most basic thing that needs to be set on models is the User Dims. This is basically a bounding box for each model. To see this box in ModelEdit, select 'Options > Show User Dimensions'. By default the dims are a long box. This needs to be changed to more closely match the size of the model. This can be done either by adjusting the '+' and '-' buttons in the 'Animation Edit' section of ModelEdit or by selecting 'Animation > Dimensions…' and entering appropriate values. One thing to know about user dims is that the X value must always equal the Z value. If not the engine will pick one and apply it to both, producing strange results. Another thing to notice is that user dims can be set per animation. To avoid possible timing issues with changing dims, they are usually set the same for all animations on a character. The only exception in NOLF2 is the crouching animations for player characters. All human characters in NOLF2 have X=24, Y=53, Z=24.

Weight Sets

In order to render in the engine, character models need to have several 'weight sets' created. Weight Sets are used by the programmers to tell a character to play more than one animation at a time, such as the recoil animations or the upper/lower body blending in multiplayer. Select 'Model > Edit Weighted Animation Blending' in ModelEdit. Weight Sets are created one at a time by clicking the 'Add Set' button and entering the name. The following sets need to be added to every character model: Null, Upper, Lower, blink, twitch. To set the Weight Sets, select one or more nodes in the 'Nodes' window and set the appropriate value in the 'Current Node Weight' window. (When setting this number, don't press enter, as it will exit the dialogue and frustrate you).

- **Twitch** - select all the nodes in the node window and set the Weight to '2'. Setting the weight to '2' tells the engine to additively blend animations and in this case allows the twitch on dead bodies and recoil on shot enemies.
- **Blink** – select the Eyelid node (if one exists) and set its value to 2. This will cause the engine to play an animation called 'blink' if it exists constantly as long as the character is alive and blend it over whatever the character is doing. This could be used creatively…

- **Upper** – select all the nodes in the upper body of the character and set the weight to '1'. This tells the engine to replace the animation on these nodes with the desired upper body animation.
- **Lower** – select all the nodes in the lower body of the character and set the weight to '1'. Note that if there is a node controlling the translation of the model, it should be in the lower body.
- **Null** – leave all nodes set to zero.

Note that other than the Upper and Lower sets in multiplayer characters, all of these can be left at zero and the models will work fine. The sets just need to exist for the engine to be able to set up each character. Non-character models do not need weight sets.

Sockets

Sockets are created to define locations where other models or FX can be attached to characters. The most noticeable place where sockets are used is in the gun hands of character models. This is the only socket I can think of that is necessary for a character to function properly. This socket must be named 'RightHand', but can be attached to any node in the model (for example, the laser shooting super soldier has a RightHand socket attached to his Head node.) To create this socket in ModelEdit, select the desired node (usually the right hand) in the Nodes window. Then select 'Sockets > Add Socket…', and enter the name 'RightHand'. The socket will be created at the location and orientation of it's parent node. Usually this will need to be edited. This is most easily done by showing attachment models in ModelEdit. (Unfortunately, new sockets don't show attachments until the model is saved and re-opened.) Select 'Options > Show Sockets' and 'Options > Show Attachments'. Now double click the socket name (RightHand) in the 'Sockets' window. In the 'Attachment' field of this properties window, enter or browse to the location of an attachment model lta file (gun, flashlight…). Also in this window are entry boxes for orientation, translation, and scale values. These are very helpful when you know exactly what they need to be, but can be a little counterintuitive when starting from scratch since they are based on the orientation of the parent node. It's easier to use the 'Transform Edit' controls. The Red, Green, and Blue boxes correspond to the colored axes of the socket in the ModelEdit viewport. Translation and Rotation are usually all that is needed to position the attachment into place. Scale is usually only used if an attachment that is shared among several characters needs to be adjusted to fit onto larger (or smaller) body part. All the sockets that you create are listed in the 'Sockets' window in ModelEdit. Other important sockets that every character should have are LeftFoot and RightFoot. These are needed to create footprints and accurately located footstep sounds. Other than this the main uses for sockets are hats, glasses, holstered weapons. To see how these and other sockets are set up, examine the sample character. Most characters have about a dozen sockets even though many of them aren't used by all the characters. This is for consistency, so no one needs to remember which character gets which sockets. A socket name will not be valid unless it is appears on a list determined by game code. The full available list can be seen by opening DEdit, selecting an AI and opening the attachments property. One other thing… the exporter plug-in allows sockets to be created and exported from Maya. To do this create an object (usually a locator) and add a Boolean attribute called 'sockets' and set it to '1'. This object can be used to orient objects within Maya and may be easier to work with than ModelEdit's controls.

Importing Sockets and Weight Sets

Sockets and Weight Sets are two things that are commonly imported from similar existing models. This can be done by selecting 'File > Import…'. To import only Sockets and Weight Sets, check the boxes next to those options, and uncheck Animations. Then browse to a previously set up model with a matching skeleton and click 'Open'. This can save a lot of tedious work, but some sockets may still need to be adjusted depending on the situation. Among the other options, Animations can also be imported from one model to another (instead of using a child model). When importing Animations, you should always import User Dims and Translations simultaneously to avoid needing to reset this information. The 'UV Coords' is no longer used.

Child Models

Child models are added in ModelEdit by selecting 'Child Model > Add ChildModel…' and browsing to the desired lta or ltc file. The only requirement for a model to be a valid child model is that the hierarchy must match exactly that of the parent model. A model can have as many as 32 child models, some of which can be added in game code as needed. Generally, only 1-3 child models are needed, depending on the character, and they will be listed in the 'Child models' window in ModelEdit. If a child model has drastically different proportions than a character, you may notice the character distorting to match those proportions. This can be corrected by checking the box next to the distorted nodes in ModelEdit's Nodes window. This tells the engine to ignore the translation information from the child model for this node and use only the rotation. In most NOLF2 characters, all the nodes are checked except translation, movement, and most Face nodes. These nodes contain mostly translation animation, so ignoring translation will kill the animation. The Face nodes that should be checked are the eye nodes and the eyelid node, since they only contain rotation animation. Note that this only works if a character shares the exact face setup as the

child model. If this is not the case, all the face nodes should be checked.

Texture Setup

Setting up textures and render styles in ModelEdit is done in the Piece Info window. This is accessed by highlighting a model piece in the 'Pieces' window and selecing 'Piece > Piece Info…'. This can be the most confusing part of setting up a model. The main things to understand here are the Texture Indices and Render Style Index. All of the textures and render styles associated with a model are listed in an attribute file. Characters are in modelbutes.txt and props are in proptypes.txt. Here is an example of a Super Soldier's texture list:

```
Skin0          = "chars\skins\SSLtBody.dtx"
Skin1          = "chars\skins\SSLtHead.dtx"
Skin2          = "chars\skins\SSLtPack.dtx"
Skin3         = "chars\skins\shinyspot.dtx"
      RenderStyle0   = "RS\default.ltb"
       RenderStyle1   = "RS\envmap.ltb"
        RenderStyle2   = "RS\glass.ltb"
         RenderStyle3   = "RS\Glow.ltb"
```

The Texture Index refers to the Skin number in this list. The Render Style Index refers to the Render Style number in this list. For example the 'body' piece of the model will use Skin0 and RenderStyle0, so the indices will stay at the default values of zero. Some render styles use multiple textures. The only commonly used render style like this is envmap.ltb. An example of its use would be the Super Soldier's backpack. This uses RenderStyle1, Skin2 as the main texture map, and Skin3 as the environment map. To set this up, the 'Number of Textures' option needs to be set to '2'. Then Texture Index 0 should be set to '2' (for Skin2) and Texture Index 1 should be set to '3' (for Skin3'). The Render Style Index should be set to '1' (for RenderStyle1). There is also an option for 'Render Priority'. This is used to insure proper sorting of transparent models. All non-transparent models should have this set to '0'. Most models using alpha transparency render styles should have this set to '1' to insure that that piece will always be rendered properly when it is in front of an opaque piece. Complex model setups, like Armstrong's beard, can have a wide range of render priorities to sort the pieces properly.

Saving and Compiling

The last thing that needs to happen in ModelEdit is to save and compile the model. To Save and Compile, select 'File > Save and Compile…'. There are several options here, but only a couple that are ever actually used. One is the compression type. The default compression type is (RLE 16) which in the case of some animations, is too much compression. I generally compress unanimated models at '(RLE 16)' and compress any thing with subtle animations using '(RLE16)PlayerView'. This difference is that with (RLE16)PlayerView, everything is compressed to 16bit except translation which stays at 32bits. This basically results in smoother animations. The other commonly used option is 'Exclude Model Geometry'. This can be used for any model that is exclusively a child model and allows for really good data compression since the geometry is simply thrown out. The good news about the compile window is that all the settings are saved per model, so the only need to be set when first exported, if at all.

Other ModelEdit functions

Here are the other most commonly used ModelEdit functions.

Animation Editing

ModelEdit has a number of functions to allow simple editing of animation data. An animation cannot be edited if it is being shared from a child model. In the 'Animations' window, the animations are listed with a check mark in the box next to the name if they can be edited. Right clicking on the name of an animation will bring up a menu with options to rename, duplicate, or delete the animation. The animation time slider displays the keyframes of an animation as red (or green) ticks. For editing purposes, ModelEdit allows you to 'tag' a group of keyframes. Keyframes can be tagged individually by double clicking the red ticks in the time slider. This actually toggles the tagged state, so double clicking again will untag the keyframe. Tagged keyframes will appear as blue ticks. Groups of keyframes can be tagged by holding down the 'Shift' key and click-dragging the mouse across the group of red ticks. The tagged state of keyframes is not saved with the file. Right clicking the time slider opens a menu. Here are the useful options in this menu:

- **Delete Keyframe** – This will delete the current keyframe from the animation. It will not work on the first or last frame of the animation.
- **Delete Tagged Keyframes** – This allows multiple keyframes to be tagged and deleted simultaneously. Deleting unnecessary keyframes is a useful file size optimization.
- **Clip After Keyframe** – This will delete all the keyframes after the current keyframe.
- **Clip Before Keyframe** – This will delete all the keyframes before the current keyframe.
- **Insert Time Delay** – This will allow a time delay between two keyframes to be specified, and is used a lot in tweaking the timing of animations in cut-scenes.

More useful functions can be found in the 'Animation' menu:

- **Set Animation Rate** – This changes the keyframe rate of an animation. Most NOLF2 animations come from motion capture data, which has 30 frames per second. This function is used to reduce that rate. Most NOLF2 AI animations are reduced to 7.5 frames per second, while cut-scene animations are 10 frames per second. Tagged frames are protected from this function, so if there are rapid motions in the animation, tagging those frames will allow them to retain the full rate.
- **Set Animation Length** – This will scale the speed of an animation. During NOLF2 production, it was the easiest way to accurately set the timing of animations.
- **Set Animation Interpolation** – Interpolation is what the engine does to blend from one animation to another. This function allows the time of this blend to be set. The default is 200 ms, and that is good for most purposes. Looping animations and cut-scene animations usually have 0 interpolation.
- **Make Continuous -** This will duplicate the first frame of an animation and place it at the end to create a rough looping effect. This isn't perfect in most cases, but is a quick way to get a looping animation for testing purposes.
- **Reverse** – This will reverse the frames of an animation so that it plays backwards.
- **Create Single Frame** – This creates a one frame animation from the current frame of an existing animation.
- **Translation** – This is used to add a positional offset to an animation.
- **Rotate** – This is used to add a rotational offset to an animation.

Frame Strings

It is also possible to add commands to animations so that they are executed at a desired keyframe. This is done in ModelEdit by setting the animation to the desired keyframe and entering the command in the 'Frame String' text field (at the bottom of the ModelEdit interface). A keyframe that has a Frame String will appear as a green tick in the animation timeslider, as opposed to the default red tick. Multiple commands can be added to the same keyframe by separating the commands with a semicolon. There are many possible commands that can be used here – pretty much any command that can be sent to a character through triggers in the game will also work here. The most common ones are:

- **FOOTSTEP_KEY 1** – Right Footstep. This will play the appropriate sound effect when the right foot hits the ground and should be placed that point in the animation. It will also leave a footprint at the correct place in the snow.
- **FOOTSTEP_KEY 2** – Left Footstep. This should be placed when the left foot hits the ground.
- **DOOR** – Activate a door object. This should be placed at the keyframe where the door should start opening.
- **OPEN** – This command is used with smart objects to activate an object associated with the smart object. An example of this is the 'Desk' smart object where the OPEN command activates a chair to move when the animation pulls it from under the desk. For more information on smart objects see the AI docs.
- **FX *<fx name>*** - This will play an FX object at the desired keyframe. An example of this in action is the dissolve body FX (FX AIBodyRemove).
- **Attach *<socket name> <attachment name>*** – This will cause an attachment to be spawned at the specified socket and keyframe. An example is the cigarette appearing in an AI's hand at the appropriate time (Attach LeftHand cigarette).
- **Detach *<socket name>*** – This will remove an attachment from a character at the desired keyframe.

Importing LODs

Model LODs are another confusing part of model setup. Since they are not necessary, they may not be worth the trouble when experimenting. Nevertheless, here is how they can be imported using ModelEdit. LOD models should first be exported as a separate lta file, and the pieces should be named exactly as the pieces in the main file. LODs are imported on a piece-by-piece basis from this file. In ModelEdit, highlight the desired piece in the 'Pieces' window and select 'File > Import Custom LODs…'. Browse to the file containing the LOD pieces and click 'Open'. A window will open containing a list of all the pieces in that file. Select the appropriate piece and click 'OK'. Now open the '+' sign next to the piece name in the Pieces window. It should open to two '0.00' children. Here's the tricky part that will be frustrating. One of these is the original and the other is the LOD, but it can be difficult to guess which one. Usually the top 0.00 piece is the LOD, so highlight it and select 'Piece > Piece Info…'. The Texture and Render Style information should match the original Piece. In the LOD Properties section, set the Distance to a value other than 0. This will be the distance where the original model piece will swap with the new LOD. This process will need to be repeated for each piece of each LOD. Most of the characters in NOLF2 have two full LODs, one at 250 units, and on at 550.

Command String

The model's Command String is used to store various commands and settings used by the engine when setting up a model. In previous versions of Jupiter, there were many commonly used commands that were placed here, but many of those are now obsolete. The only thing the Command String is currently being used for is enabling shadows on a model. To access the Command String in ModelEdit, select 'Model > Command String…'. To turn in shadow casting simply enter "ShadowEnable" in the text field.

Piece Merging

A piece-merging feature was added to help optimize the number of geometry pieces in a model. It will merge all pieces that share a render style and texture. To use it highlight the desired pieces in the 'Pieces' window and select 'Piece > Merge Selected'. This allows modelers to keep models in separate pieces in the modeling software if desired and wait to combine them in the exported model for optimization.

Null LODs

It is possible to have model pieces disappear when models are a certain distance from the player. This is done in ModelEdit by highlighting the desired piece in the 'Pieces' window and selecting 'Piece > Create Null LOD…', then enter the desired distance. This was used as an optimization in Armstrong's beard, which mostly disappears at about 600 units.

Bute Files

All models in the game need to be defined in an attribute file. Characters are defined in 'modelbutes.txt', props are defined in 'proptypes.txt', and attachments are defined in 'attachments.txt'. There is a not a lot that needs to be understood about these files in order to add to them, but not following the format exactly can cause disastrous results. For this reason, they should be modified at your own risk, and never without first backup them up. Most new models are created by simply cutting and pasting new model listings from existing, similar models and changing the number, Name, the model filename, the Skin and RenderStyle lines. It is very important that each model has a unique number and name, and that each attribute file contains no gaps in numbers (example: a file that contains [Model56] and [Model58], but not [Model57] will crash the game. Also, a file that contains two [Model56] listings will crash the game). This is pretty important, but it's really all that needs to be understood to add most new models. Characters can be a little more complex for two reasons. One is the Skeleton section, which is used to define hit detection per skeleton and can be enormously painful to deal with. For this reason, it is recommended that experimenting with new characters should stick to the existing sample skeleton (Character_Sample.mb). A lot can be done with it without any changes. The other extra piece of information for characters is the 'DeathmatchModels' section. To add new multiplayer models, in addition to the model listing already described, the model must be added to this list using the 'Name' of the model listing (example 'Name36 = "NewModelName"'). This covers all the needed basics for model setup in NOLF2, but again modify at your own risk and be sure to backup the originals.

LtExport.mel Script

As mentioned in the exporter section, as MEL script was created to help streamline our workflow and give the ability to store a lot of settings in the Maya scene that would otherwise need to be set in ModelEdit every time a model is re-exported. It is intended to be an interface to the exporter plugin. This is 100% artist created tool and most of it was created as a means to learn MEL scripting. Because of this, it is unsupported and may not be the best way to do everything… but it works for us, and nearly all NOLF2 (and TRON) models were exported using this script as an interface. The script requires the file 'LTC.exe' to support the compressed ltc version of the lta file format. To use this script, place it in a user script directory, or Maya's Scripts\Others\ folder. Enter the command 'ltExport' to open the interface. Included is a brief description of options, settings, and known limitations in the script.

Edit Options

The 'Edit Options' section contains functions that attempt to mimic ModelEdit functionality:

- UserDims – This button creates a bounding box that can be scaled normally to set the dims. It correctly locks the X and Z values to avoid bad dims. The scale values will be saved on the skeleton so that they can always be exported correctly.
- Sockets – This opens a window that will give options to create and delete sockets and works very much like the ModelEdit dialogue.
- Piece Info – For texture setup, this works exactly like ModelEdit's Piece info window. Select a Piece and press this button and the window will open to allow settings to be made. It also has a place to enter the correct game texture path so that textures will show up in ModelEdit automatically (This may not be usable except during production).
- CommandString – This is exactly like the ModelEdit Command String window. Anything entered here will be saved with the skeleton.
- ChildModels – This allows child models to be specified. A list of child models is saved with the skeleton.
- NodeFlags – This mimics the 'Nodes' window of ModelEdit, allowing the node check box setting to be saved with the skeleton.

Other Setting Options

Here are the other available setting options:

- Append Animation – When checked on, this will cause the exporter to always append animations without warning. When off, a warning will come up to prevent accidentally overwriting files. This sets the '-append' flag of the exporter flag of the exporter plugin.
- Use TimeSlider range – This sets the '-usePlaybackRange' flag of the exporter plugin.
- ExportWeightSets – This will create (but not correctly set) the default set of weight sets needed to for characters to work in the game.
- Export Child Models – This was added to allow a model to have child models in Maya, but export without them if needed.
- CompileModel – This will automatically compile the model as an ltb file. When checked on, an option is exposed to 'Compile with no geometry'. This is used when exporting many animations to a child model.
- Dims – This provides a numerical input for setting User Dims.
- Scale – This sets the '-scale' flag of the exporter plugin.
- Interpolation – This sets the interpolation time of an animation. It is the same as using 'Set Animation Interpolation' in ModelEdit.
- ToolPath – This should provide the folder where the file 'LTC.exe' exists
- GamePath – This may not be usable, but shouldn't affect the working of the script.

In addition to these options, the script will export LODs for models by looking for a very specific naming convention in the Maya scene. I can't think of a better way to describe this setup than to point to the sample Maya character setup scene (Character_Sample.mb). It contains LOD pieces that have been hidden to prevent them from exporting. Unhiding them and exporting this scene using the ltExport Script will result in fully set up LODs.

Limitations

There are lots of issues that were never addressed due to lack of time. Here are a couple that you'll probably notice:

- There are a lot of 'Browse' buttons in the interface. Most of them don't work at all and the main File Browser will only work if you select an existing lta or ltc file. I never did figure out how to get file browsing to work correctly.
- The script will give an error if it is run when there are no skeletons in the scene.
- The script recognizes the first skeletal hierarchy in the scene to be the export skeleton. This is always the top joint listed in the outliner or the leftmost one in the hypergraph. Occasionally a hidden skeleton could be listed before the desired one and cause a lot of confusion.
- There is a weird bug that occurs when child models are specified and exported using this script… the normals are destroyed. Fortunately this can be remedied in ModelEdit by selecting 'Model > Generate Vertex Normals'. Because of this, unless there are several child models, I don't really use the child model part of the script much.
- The 'Node Flags' option will be made invalid if the skeleton is changed after setting the flags. The exporter won't notice this, and may cause an assortment of problems if invalid values are exported.

Content Guide

AI Placement in Dedit

All AI placed through Dedit are CAIHuman. (Yes, including robots, rats, and rabbits). In addition to placing CAIHumans, other AI objects are placed in Dedit that allow AI to understand the world. AI navigate the world by planning paths through connected AIVolumes. AI determine what objects or places of interest are nearby by searching for AINodes. AIVolumes and AINodes are invisible to the player.

Properties on the CAIHuman object determine the AI's look and behavior:

CAIHuman Parameters:

ModelTemplate	The model template from modelbutes.txt. The model template deteremines the ltb model file, skeleton, animation data file, skin textures, and renderstyles. The model template references an AI template in aibutes.txt. The AI template determines the AI's alignment, and sensory parameters.
Brain	The brain template from aibutes.txt. The brain determines a number of parameters that affect the AI's behavior.
GoalSet	The goalset defined in aigoals.txt. The goalset lists the goals that will control the AI's behavior.

AIButes.txt

AIButes.txt contains various sections defining parameters that affect AI behavior. The major sections are AIBrains, and AI Attribute Templates. Sections not described are insignificant, self-explanatory, or beyond the scope of this document (AIStimulus – tweak at your own risk!).

AIBrain

AIBrains specify a number of parameters that define an AI's behavior. Many AI may share the same brain. There are a number of required parameters, and some optional parameters. Some parameters found in AIbutes.txt were not used for NOLF2.

Required Parameters:

DodgeVectorCheckTime	How frequently we check to see if we need to dodge a vector
DodgeVectorCheckChance	The % chance we will check to see if we need to dodge a vector once this time is up
DodgeVectorDelayTime	After dodging a vector, the time we must wait before we can check again
DodgeVectorShuffleDist	How much space is needed to shuffle.
DodgeVectorRollDist	How much space is needed to roll.
DodgeVectorShuffleChance	The % chance the dodge will be a shuffle.
DodgeVectorRollChance	The % chance the dodge will be a roll.
DodgeVectorBlockChance	The % chance the dodge will be a block.
DodgeProjectileCheckTime	Not used for NOLF2.
DodgeProjectileCheckChance	Not used for NOLF2.
DodgeProjectileDelayTime	Not used for NOLF2.
AttackRangedRange	The min/max distance we must be at to attack with a ranged weapon (e.g. rifle)
AttackThrownRange	The min/max distance we must be at to attack with a thrown weapon (e.g. grenade)
AttackMeleeRange	The min/max distance we must be at to attack with a melee weapon (e.g. sword)
AttackChaseTime	After we lose sight of our target, how long do we wait before chasing
AttackChaseTimeRandom	An additional random amount of time to the above time
AttackPoseCrouchChance	When we enter the attack state, chance we will crouch and fire
AttackPoseCrouchTime	How long we stay crouched
AttackGrenadeThrowTime	How often we'll consider throwing a grenade
AttackGrenadeThrowChance	Chance we'll attempt to throw the grenade after this time expires
AttackCoverCheckTime	How often we check for the presence of cover
AttackCoverCheckChance	Chance we'll check once this time expires
AttackSoundTimeMin	Min amount of time since any AI played combat dialog before this AI can play combat dialog
AttackWhileMoving	Can the AI attack while moving?
AttackFromCoverCoverTime	How long we will stay covered for before uncovering

AttackFromCoverCoverTimeRandom	An additional random amount of time to the above time
AttackFromCoverUncoverTime	How long we will stay uncovered for before covering
AttackFromCoverUncoverTimeRandom	An additional random amount of time to the above time
AttackFromCoverReactionTime	If uncovered and shot at, the delay before we will go back to cover
AttackFromCoverMaxRetries	How many times we will attempt to go for new cover when cover becomes invalidated.
AttackFromVantageAttackTime	Time we will fire from a vantage node before moving on to the next one
AttackFromVantageAttackTimeRandom	An additional random amount of time to the above time
AttackFromViewChaseTime	How long we will we wait to chase at an attack from view node
ChaseExtraTime	Not used for NOLF2.
ChaseExtraTimeRandom	Not used for NOLF2.
ChaseSearchTime	Not used for NOLF2.
PatrolSoundTime	Time between chances of playing a patrolling sound
PatrolSoundTimeRandom	An additional random amount of time to the above time
PatrolSoundChance	The chance we'll play a patrol sound after the patrolsoundtime
DistressFacingThreshhold	Dot product (-1=180' to 1=0') at which target must be facing towards us to be considered threatening
DistressIncreaseRate	Rate at which distress increases when being threatened
DistressDecreaseRate	Rate at which distress decreases when not being threatened
DistressLevels	How many levels of distress to go through before panic
MajorAlarmThreshold	How high alarm level must be to start searching (threat assumed)
ImmediateAlarmThreshold	How high alarm level must be to know threat is immediate
TauntDelay	How long between checks to taunt
TauntChance	Chance the AI should taunt when it is able to
CrawlThreshhold	Not used for NOLF2.
CrawlChance	Not used for NOLF2.
CrawlLifetime	Not used for NOLF2.
CanLipSync	Can this AI lip sync?
DamageMask	Mask of accepted types of damage. Other types are ignored. "None" means affected by all damage types.

Optional Parameters:

Parameters that are not required for every AI are specified in the AIBrain with name = value pairs, in the form AIDataN = “SomeLabel=SomeValue”. For example:

```
AIData0 = "AccidentChance=0.1"
```

```
AIData1 = "DivergePaths=1.0"

AIData2 = "OneAnimCover=1.0"
```

AccidentChance	Percent chance of AI having an accident. Used for AIs falling on their faces when jumping over railings.
BreaksDoors	AI destroys doors when going through them. Used for SuperSoldiers.
CannotPathThruDoors	AI may not go through doors. Used to contain ambient life.
ChaseEndurance	Number of seconds running before AI needs to catch his breath. Used for police.
DeflectChance	Percent chance of deflecting a bullet. Used for ninjas with katanas.
DisappearDistMin	Minimum distance away a threat must be to trigger AI to disappear. Used for ninjas.
DisappearDistMax	Minimum distance away a threat can be to trigger AI to disappear. Used for ninjas.
DisappearFadeTime	Amount of time to fade translucency when disappearing. Used for ninjas.
DisposeLantern	AI must detach object from light socket when leaving an investigative state. Used for ninjas with search lanterns.
DivergePaths	AIs try to split up taking separate paths to the same destination.
DoNotCloseDoors	AI does not close doors after going through them.
DoNotDamageSelf	AI does not damage himself with his own weapons. Used to keep Volkov from injuring himself with rocket splash damage.
DoNotExposeHiding	AI does not expose a hidden player, in a hiding spot. Used for ambient life.
DoNotToggleLights	AI does not attempt to turn on/off lights.
DynMoveTimeMin	Minimum time AI stands still while attacking with a ranged weapon.
DynMoveTimeMax	Maximum time AI stands still while attacking with a ranged weapon.
DynMoveBackupDist	Distance an AI backs up while attacking with a ranged weapon.
DynMoveFlankPassDist	Distance past a target AI will travel when flanking.
DynMoveFlankWidthDist	Distance to the side of a target AI will travel when flanking.
IgnoreJunctions	AI cannot get lost in junctions.
LethalSpecialDamage	SpecialDamage will instantly kill AI. Used for ambient life, when hit with sleeping damage, electrical damage, etc.
LongJumpHeightMin	Minimum height of parabola when jumping long distances. Used for ninja lunge.
LongJumpHeightMax	Maximum height of parabola when jumping long distances. Used for ninja lunge.
LungeDistMin	Minimum distance away a target can be to trigger a lunge. Used for ninja.
LungeDistMax	Maximum distance away a target can be to trigger a lunge. Used for ninja.

LungeSpeed	Speed AI travels while lunging. Used for ninja.
LungeStopDistance	Distance in front of a target to end a lunge. Used for ninja.
GoalProximityDist	Distance to trigger the ProximityCommand goal.
MeleeKnockBackDist	Distance to push back a target with a melee attack.
MinPathWeight	Minimum weight of an AIVolume that AI will use when pathfinding. Used to restrict some AI to preferred volumes.
MusicMoodMin	Minimum music mood AI will set. Used to keep ambient life from affecting music.
MusicMoodMax	Maximum music mood AI will set. Used to keep ambient life from affecting music.
NoDynamicPathfinding	AI will not try to avoid dynamic obstacles.
OneAnimCover	AI uses one movement-encoded animation to attack from cover, rather than a series of animations.
PrimaryWeaponOnLeft	Primary weapon is on AI's left side, instead of the default right side.
ProneDistMin	Minimum distance from target AI must be to go prone.
ProneSlideDist	Distance AI slides forward when diving to the ground to go prone.
ProneTime	Amount of time AI stays prone.
ReappearDist	Distance away AI reappears after disappearing. Used for ninja.
RetreatTriggerDist	Distance from target to AI required to trigger AI to retreat. Used for ninja.
RetreatJumpDist	Distance AI jumps back when retreating. Used for ninja.
RetreatSpeed	Speed AI travels while jumping back when retreating. Used for ninja.
SleepTimeMin	Minimum time AI sleeps. Used for powered-down SuperSoldiers.
SleepTimeMax	Maximum time AI sleeps. Used for powered-down SuperSoldiers.

AI Attribute Templates

AI attribute templates define parameters for AI behavior associated with a model. Multiple models may share the same template.

Required Parameters:

Name	Name of template, referenced in modelbutes.txt as AIName.

RunSpeed	Speed AI moves when running with an animation that uses constant velocity (rather than movement encoding).
JumpSpeed	Speed AI moves while playing a jumpUp animation.
JumpOverSpeed	Speed AI moves while playing a jumpOver animation.
FallSpeed	Speed AI moves while falling.
CrawlSpeed	Not used in NOLF2.
WalkSpeed	Speed AI moves when walking with an animation that uses constant velocity (rather than movement encoding).
SwimSpeed	Not used in NOLF2.
FlySpeed	Not used in NOLF2.
Marksmanship	Not used in NOLF2.
Bravado	Not used in NOLF2.
SoundRadius	Sound radius used when AI plays dialogue sounds.
HitPoints	Amount of hit points AI starts with. Modified by difficulty.
Armor	Always 0 for NOLF2.
Accuracy	Percent chance a shot will hit AI's target.
AccuracyIncreaseRate	Rate AI regains accuracy after a change due to attacking from cover.
AccuracyDecreaseRate	Not used in NOLF2.
FullAccuracyRadius	Radius from AI where AI has perfect accuracy.
AccuracyMissPerturb	Amount of perturb on bullets that miss their target.
MaxMovementAccuracyPerturb	Maximum amount of perturb caused by aiming at a moving target.
MovementAccuracyPerturbTime	Amount of time to catch up when aiming at a moving target.
CanUseVolumeXXX	AI may pathfind through AIVolumes of type XXX (e.g. CanUseVolumeJumpUp). By default, AI may pathfind through all AIVolume types. Types of AIVolumes are described in the AIVolume section of this document.
CanXXX	TRUE if sense is turned on for AI, where XXX is a sense. (e.g. CanSeeEnemy).
XXXDistance	Distance that AI can sense XXX. (e.g. SeeEnemyDistance).

Senses referenced by XXX above include:

SeeEnemy	AI sees an enemy.
SeeEnemyLean	AI sees a leaning enemy.
SeeEnemyWeaponImpact	AI sees someone get shot by an enemy.
SeeEnemyDanger	AI sees something dangerous. Used for AIGoalLoveKitty.
SeeEnemyDisturbance	AI sees something disturbed by an enemy.
SeeEnemyLightDisturbance	AI sees a light disturbed by an enemy (turned on or off).

SeeEnemyFootprint	AI sees an enemy's footprint.
SeeAllyDeath	AI sees an ally's dead, or knocked out body.
SeeAllyDisturbance	AI sees an ally get disturbed.
HearEnemyWeaponFire	AI hears an enemy fire a weapon.
HearEnemyWeaponImpact	AI hears an enemy's weapon impact.
HearEnemyFootstep	AI hears an enemy's footsteps.
HearEnemyAlarm	AI hears an alarm sound, due to an enemy's presence.
HearEnemyDisturbance	AI hears an enemy make a disturbance.
HearAllyDeath	AI hears an ally die.
HearAllyPain	AI hears an ally shout in pain.
HearAllyWeaponFire	AI hears an ally fire a weapon.
HearAllyDisturbance	AI hears an ally get disturbed.
SeeAllyDistress	AI sees an ally in distress, going for backup.
SeeAllySpecialDamage	AI sees an ally affected by special damage (glued, in bear trap, on fire, etc)

AIVolumes

AIVolumes denote open space to an AI, and must be placed everywhere an AI may travel. AIVolumes are invisible to the player.

The restrictions and requirements for placing AIVolumes in general are:

- AIVolumes do not contain any physical obstacles (e.g. desks, walls, columns, etc).
- AIVolumes align exactly with adjacent AIVolumes.
- AIVolumes containing doors must be narrow volumes just containing the dimensions of the closed door. Only one door may be in an AIVolume.
- AIVolumes have minimum dimensions in XZ of 48x48.
- AIVolumes are generally 16 units tall, and 8 units off the ground. This is not a requirement.
- AIVolumes touching other AIVolumes must have a connection of at least 48 units.

There are several varieties of AIVolumes, used for different purposes. Below are descriptions of the purposes and parameters of the different types of AIVolumes. AIVolumes not listed below were not used on NOLF2.

AIVolume

AIVolume is the most basic volume type. All other AIVolume objects are derived from AIVolume. All AIVolumes have the parameters below at a minimum.

Lit	FALSE if volume is in the dark. Tells AI whether they need to turn on a light. Can be toggled through the commands "lit" and "unlit". Example: msg aivolume01 unlit
Region	AIRegion this AIVolume is part of. Regions are used for alarms and for searching.
LightSwitchNode	LightSwitch AINodeUseObject used to toggle the light state of this volume (lit/unlit).
PreferredPath	TRUE if this volume should be preferred over other for pathfinding. In NOLF2 this was used a couple of different ways. Only SuperSoliders and Robots can travel through AIVolume with PreferredPath set to true so they don't walk through spaces that are too small for them to fit. The AIBrain AIData MinPathWeight=1 takes care of this. We also used this to make the AI path finding look better in some situations. For instance in Japan, when a ninja was patrolling a section of street sometimes she would walk through a flower bed to get to her next patrol point. Setting the PreferredPath to True on the AIVolumes that were on the streets helped make the AI paths look better by making them walk on the streets when in a relaxed state..

AIVolumeAmbientLife

There is nothing special about AIVolumeAmbientLife. The volume types simply exists so that ambient life AI can be use a set of volumes that other AI do not use (e.g. so that a rat can run under a table, but a soldier cannot). AI are restricted from pathfinding through AIVolumeAmbientLife by adding this line to their AI Attribute Template in AIButes.txt:

```
CanUseVolumeAmbientLife = FALSE
```

AIVolumeJumpOver

AI use AIVolumeJumpOvers to pathfind and jump over obstacles (e.g. fences). This volume should connect to AIVolumes horizontally. AI jump from one edge of the volume to the opposite edge. The height of the parabola of the jump is determined by the height of the top of the volume.

AIVolumeJumpUp

AI use AIVolumeJumpUps to pathfind and jump up and down from things (e.g. rooftops). This volume should connect to AIVolumes above and below the jump destination. AIVolumeJumpUp has several additional parameters.

OnlyJumpDown	This volume may only be used to jump down

	from something (and not jump up onto something).
OnlyJumpWhenAlert	AI may only jump through this volume when alerted to a threat.
HasRailing	AI must play an animation to jump over a railing when jumping through this volume.

AIVolumeJunction

AI get lost when chasing a target, and running through an AIVolumeJunction while the target is out of sight. A junction connects up to four AIVolumes. AIVolumeJunctions have extra parameters determining the percent chance of different actions that can be taken once the junction is reached. The extra parameters are duplicated for each of the connecting volumes in four possible directions, North, South, East, and West.

Junction Actions:

Nothing	Run through junction and get lost.
Continue	Run through junction and get lost.
Peek	Peek into a connecting volume, and then run a different direction.
Search	Run into a volume and search its AIRegion.

AIVolumeLadder

AI use AIVolumeLadders to pathfind and climb on ladders. This volume should be centered on the ladder, and connect to AIVolumes above and below the ladder. AI play a special death animation when killed inside an AIVolumeLadder.

AIVolumeLedge

AI play a special death animation when killed inside an AIVolumeLedge. These volumes are typically used on balconies, rooftops, and cliffs. The angle of the Yaw determines which direction the AI will fall off the ledge.

AIVolumeStairs

AI use AIVolumeStairs to pathfind and walk up stairs. This volume should enclose the staircase, and connect to AIVolumes above and below the stairs. AI play a special death animation when killed inside an AIVolumeStairs. The angle of the Yaw determines which direction the AI will roll down the stairs when killed.

AIVolumePlayerInfo

AIVolumePlayerInfo is totally unrelated to the other AIVolumes. AIVolumePlayerInfo is not used for AI pathfinding. It is used to determine if the player is hidden, or attackable from an associated AINodeView.

AIVolumePlayerInfo has a number of unique parameters.

On	Volume is usable for hiding, and sense mask is enabled when On is TRUE. This volume can be messaged "On" and "Off" with commands.
SenseMask	SenseMask applied to the volume, that blocks out specified senses while AI are relaxed. Used in the HARM Villa to make enemies ignore Cate when she was in unrestricted areas.
Hiding	Player can use this volume to hide.
HidingCrouch	Player can use this volume to hide, if she is crouched.
ViewNodes	List of AINodeViews that are associated with this volume. If an AI cannot find a path to the player, and wants to attack, he may go to an AINodeView and attack from there.

AINodes

AINodes are objects that provide information to AI about opportunities at particular locations. AI must be able to find a path to an AINode, so all AINodes must be inside of AIVolumes. AINodes are invisible to the player.

There are several varieties of AINodes, used for different purposes. Below are descriptions of the purposes and parameters of the different types of AINodes. AINodes not listed below were not used on NOLF2.

AINode

AINode is the most basic node type. All other AINode objects are derived from AINode. All AINodes have the parameters below at a minimum.

Face	AI turns to face the direction of this node when arriving at this node.
Alignment	Only AI with the specified alignment may use this node. By default, Alignment is set to "None," which means all AI may use the node. An AI's alignment is specified in the AI Attribute Template in AIButes.txt.
StartDisabled	This node starts disabled. AI ignores disabled nodes. Nodes may be enabled / disabled through the commands "enable" and "disable".

AINodeBackup

AI runs to an AINodeBackup when he sees a threat and needs backup. The goal GetBackup sends an AI to an AINodeBackup. Other AI that are within the AINodeBackup's stimulus radius will respond to the AI when he arrives at the node, and follow him to the threat if they have the goal RespondToBackup.

AttractRadius	AI must be within this radius for the node to be valid to go to for backup.
ResetTime	Amount of time before any AI may use this node again.
StimulusRadius	AI within the stimulus radius may respond to the call for backup.
Movement	Movement animation AI uses to get to the node.
CryWolfCount	Number of times AI may call for backup at this node before allies think he is crying wolf. Count resets when allies see a threat
Command	Command to run on arrival to the node.

AINodeCover

AI run to an AINodeCover when a threat is present, and the AI activates goal Cover or AttackFromCover.

Duck	AI ducks at node and pops up to attack.
BlindFire	AI ducks at node, and fires blindly over an object.
1WayRoll	Not used in NOLF2.
2WayRoll	Not used in NOLF2.
1WayStep	AI stands at node, and steps out from behind something to attack. Node rotation determines the direction of the step.
2WayStep	AI stands at node, and steps out from behind something to attack. AI may step out from 2 directions. Node rotation determines the directions of the step.
IgnoreDir	Ignore the direction of a threat when determining if node is valid for cover.
Fov	Field of view that threat must be in to validate the node for cover.
Radius	AI must be within this radius for the node to be valid to go to for cover.
ThreatRadius	If threat enters the ThreatRadius, node is invalidated for cover.
Object	Object or AINodeUSeObject to activate on arrival at the node. (e.g. to flip a table to take cover behind).
Timeout	Amount of time AI can stay at the node.
HitpointsBoost	Amount to boost AI's hitpoints while at the node.

AINodeDeath

AI pathfind to an AINodeDeath before dying with a special animation from the DramaDeath goal.

AINodeExitLevel

AI walk to an AINodeExitLevel if they have the ExitLevel goal, and no other goals are active. This node is mainly used for reinforcements leaving the level if there are no open patrol routes of guard posts.

AINodeGoto

AI pathfind to an AINodeGoto when given a Goto goal with an AINodeGoto as the destination.

Command	Command AI runs when arriving at the AINodeGoto.

AINodeGuard

AI stays within the ReturnRadius of an AINodeGuard when he has the Guard goal, and is unaware of a threat.

ReturnRadius	AI stays within this radius when unaware of a threat. AI returns to this radius when giving up on a chase or search.
GuardedNodes	List of AINodeUseObject nodes AI may use while assigned to this AINodeGuard. Only the AI assigned to the AINodeGuard may use these nodes.

AINodePanic

AI run to AINodePanic nodes when they are in distress. AI plays a panic animation at the node.

Next	Next AINodePatrol on the patrol route.
Action	Animation to play at the node.
Command	Command to run on arrival to the node.

AINodePatrol

AI with the Patrol goal patrol along a route set by chaining AINodePatrol nodes.

AttractRadius	AI must be within this radius for the node to be valid to go to for backup.
ResetTime	Amount of time before any AI may use this node again.
StimulusRadius	AI within the stimulus radius may respond to the call for backup.
Movement	Movement animation AI uses to get to the node.
CryWolfCount	Number of times AI may call for backup at this node before allies think he is crying wolf. Count resets when allies see a threat
Command	Command to run on arrival to the node.

AINodeSearch

AI pathfind between AINodeSearch nodes when searching for a known threat. At each node, AI plays a searching animation.

ResetTime	Amount of time before any AI may visit this node again.
ShineFlashlight	Play a flashlight animation.
LookUnder	Play a look under animation.
LookOver	Play a look over animation.

LookUp	Play a look up animation.
LookLeft	Play a look left animation.
LookRight	Play a look right animation.
KnockOnDoor	Play a knock animation.
Alert	Play an alert animation.
SearchType	Invalidation test if any: Default, OneWay, or Corner. OneWay nodes are only valid if AI approaches facing the same direction the node points. Corner nodes are only valid if AI approaches facing the opposite direction from the direction the node points. If set to Corner then the Yaw should be pointed at the corner the AI is going to peak around when searching.

AINodeTalk

AINodeTalk is exactly the same as AINodeGuard, but AI stand at AINodeTalk nodes when having a conversation with another AI.

AINodeUseObject

AINodeUseObject nodes are special general-purpose nodes. When an AI arrives at an AINodeUseObject, he animates using a set of animations defined by the SmartObject specified on the node. The set of animations usually includes a transition in and out, some looping idles, and some intermittent fidgets.

Object	Optional object to activate on arrival to the node.
Radius	AI must be within this radius for the node to be a valid destination.
Movement	Movement animation used to travel to the node (e.g. Run or Walk).
SmartObject	SmartObject defining the purpose of the node. SmartObjects are defined in AIGoals.txt.
PreActivateCommand	Command to run on arrival to the node.
PostActivateCommand	Command to run when leaving the node.
ReactivationTime	Amount of time before any AI may use this node again.
FirstSound	Sound to play on arrival to the node.
FidgetSound	Sound to play while playing fidget animations.
OneWay	This node may only be used when approaching facing the direction matching the direction of the node. This was used to prevent SuperSoldiers from spinning around 180 degrees to destroy something on the wrong side.

AINodeVantage

AI run to an AINodeVantage to attack an enemy from a specific position. When an enemy's position validates an AINodeVantage, the goal AttackFromVantage sends the AI to the AINodeVantage to attack.

IgnoreDir	Ignore the direction of a threat when determining if node is valid.
Fov	Field of view that threat must be in to validate the node.
	AI must be within this radius for the node to be valid.

Radius	
ThreatRadius	If threat enters the ThreatRadius, node is invalidated.
Object	Object or AINodeUseObject to activate on arrival at the node. (e.g. open a hatch or window shutters).

AINodeVantageRoof

AINodeVantageRoof is exactly the same as AINodeVantage. Having a separate node type allows goals to use specific nodes. Ninjas use special logic in AttackFromVantageRoof to determine when to jump onto a roof to attack from an AINodeVantageRoof.

AINodeView

AI run to an AINodeView to attack an enemy that they cannot find a path to. If an AI cannot find a path to a threat, and the threat is in an AIVolumePlayerInfo, the goal AttackFromView sends the AI to an AINodeView associated with the AIVolumePlayerInfo.

Radius	AI must be within this radius for the node to be valid to go to it.

AIGoals

Goals control an AI's behavior. AI are given a goalset in Dedit. Goalsets are defined in AIGoals.txt. Goals may also be added and removed dynamically through commands. Goals compete for activation, and only one goal is active at a time. Each goal has an Importance value, which determines its priority over other goals. Goals may optionally have required SenseTriggers and/or Attractors. Attractors are types of nodes, or types of SmartObjects. The Importance of a goal is zero when SenseTrigger and/or Attractor requirements are not satisfied. Goals may optionally have parameters that can be set through name=value pairs in commands. Goals not listed below were not used on NOLF2.

Alarm

AI runs to activate an alarm when an enemy is seen, then runs back to position where enemy was seen and searches the AIRegion.

Importance	32
SenseTriggers	SeeEnemy
Attractors	AINodeUseObject with SmartObject Alarm

Animate

AI plays an animation on command. This goal is used to script animations.

Importance	100
SenseTriggers	None
Attractors	None

Parameters:

ANIM=animname	Name of animation to play, from ltb file.
LOOP=1 or 0	Infinitely loop the animation if TRUE.

AttackFromCover

AI runs to the nearest valid AINodeCover and attacks from there.

Importance	27
SenseTriggers	SeeEnemy
Attractors	AINodeCover

AttackFromVantage

AI runs to the nearest valid AINodeVantage and attacks from there.

Importance	27
SenseTriggers	SeeEnemy
Attractors	AINodeVantage

AttackFromRandomVantage

AI runs to a random valid AINodeVantage and attacks from there.

Importance	27
SenseTriggers	SeeEnemy
Attractors	AINodeVantage

AttackFromRoofVantage

AI runs to the nearest valid AINodeVantageRoof and attacks from there. AI will only activate this goal if no other AI are at AINodeVantageRoof nodes, and at least 3 AI are currently attacking the same target. Used for ninja.

Importance	28

SenseTriggers	SeeEnemy
Attractors	AINodeVantageRoof

AttackFromView

AI runs to the nearest valid AINodeView and attacks from there. AI will only activate this goal if he cannot find a path to his target, and his target is an AIVolumePlayerInfo that has associated AINodeViews.

Importance	27
SenseTriggers	SeeEnemy SeeEnemyLean HearEnemyWeaponFire HearEnemyWeaponImpact SeeEnemyWeaponImpact HearAllyDeath HearAllyPain HearAllyWeaponFire HearEnemyDisturbance SeeEnemyDisturbance
Attractors	AINodeView

Parameters:

SEEKPLAYER=1	AI will activate goal without first sensing anything if it receives the parameter SEEKPLAYER=1.

AttackMelee

AI attacks with a melee weapon, if he has one and target is within range set in AIBrain.

Importance	24
SenseTriggers	SeeEnemy
Attractors	None

AttackProne

AI drops to lie on stomach and attack with a ranged weapon. AI first checks for clear space in front of him. Only one AI may AttackProne at a time, and timing is coordinated between all AI to control frequency.

Importance	26
SenseTriggers	SeeEnemy
Attractors	None

AttackRanged

AI attacks with a ranged weapon, if he has one and target is within range set in AIBrain.

Importance	23
SenseTriggers	SeeEnemy
Attractors	None

AttackRangedDynamic

AI attacks with a ranged weapon, if he has one and target is within range set in AIBrain. Ai constantly moves between shots. Moves are randomly selected. Moves include strafing, flanking, and backing up.

Importance	23
SenseTriggers	SeeEnemy
Attractors	None

Chase

AI chases a target, and fires a weapon if allowed by the AIBrain. AI gets lost if chase path goes through an AIVolumeJunction, and target is not in sight. If target is lost, AI searches the AIRegion.

Importance	20
SenseTriggers	SeeEnemy (other senses are used to get AI to lose interest in searching).
Attractors	None

Parameters:

SEEKPLAYER=1	AI will activate goal without first sensing anything if it receives the parameter SEEKPLAYER=1.
NEVERGIVEUP=1	AI will never lose interest in chasing if it receives the parameter NEVERGIVEUP=1. Used for ninjas in the trailer park, arena combat level.
KEEPDISTANCE=1	AI will only come close enough to get target in firing range if it receives the parameter KEEPDISTANCE=1. Used for HARM Villa, where guards apprehend player rather than attacking.

CheckBody

AI checks the body of a dead or knocked out ally. Ally is woken if knocked out. If ally is dead, AI disposed of the body. AI searches the AIRegion after checking the body.

Importance	15
SenseTriggers	SeeAllyDeath (other senses are used to get AI to lose interest in searching).
Attractors	None

Cover

AI runs to the nearest valid AINodeCover and takes cover. This goal only activates if AI has a ranged weapon. AI takes cover if he was not previously aware of a threat, and hears a bullet hit nearby.

Importance	18
SenseTriggers	HearEnemyWeaponImpact
Attractors	AINodeCover

Destroy

AI walks to AINodeUseObject and animates destroying something. Animation may cause AI to fire a weapon, or attack with bare hands. Used for SuperSoldiers destroying environment.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject Attackable or Smashable

DisappearReappearEvasive

AI disappears in a cloud of smoke, and reappears elsewhere. AI only disappears if an enemy is aiming a weapon at her. Frequency of any AI disappearing is coordinated between all AI. Used for ninja.

Importance	31
SenseTriggers	SeeEnemy
Attractors	None

Parameters:

NOW=1	AI will disappear on command if it receives the parameter NOW=1.
OVERRIDEDIST=dist	Distance used to override ReappearDist set in AIBrain.
DELAY=secs	Amount of time before AI reappears.

Distress

AI puts up hands in distress. This goal only activates if the AI is unarmed and a weapon is aimed at him. Distressing may lead to panicking, running to an AINodePanic.

Importance	17
SenseTriggers	SeeEnemy
Attractors	None

DramaDeath

AI pathfinds to an AINodeDeath, and animates a dramatic death. This goal only activates when the AI dies. Used for deaths of Pierre and Volkov.

Importance	1000
SenseTriggers	None
Attractors	AINodeDeath

Parameters:

LAUNCH=1	Launch AI in a parabola to the AINodeDeath if the parameter LAUNCH=1 is set, rather than pathfinding.

DrawWeapon

AI draws a weapon (or weapons) when a target is sensed, if he has any weapons.

Importance	50
SenseTriggers	SeeEnemy SeeEnemyLean SeeEnemyFootprint HearEnemyWeaponFire HearEnemyWeaponImpact SeeEnemyWeaponImpact HearEnemyFootstep HearEnemyAlarm HearEnemyDisturbance SeeEnemyDisturbance SeeEnemyLightDisturbance SeeAllyDeath

	HearAllyDeath HearAllyPain HearAllyWeaponFire SeeAllyDisturbance HearAllyDisturbance SeeAllySpecialDamage
Attractors	None

EnjoyPoster

AI examines something on a wall. This was only used in the India Street level where the Police were hanging wanted posters.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject Examinable

ExitLevel

AI walks to an AINodeExitLevel, and deletes himself from the game. This is used to remove reinforcements who entered the level as the result of an alarm sounding. If the reinforcements cannot find any open guard nodes or patrol routes, they exit the level.

Importance	7
SenseTriggers	None
Attractors	AINodeExitLevel

Flee

AI runs to an AINodePanic and panics. This goal only activates if the AI is unarmed. If no panic node is found, AI runs away randomly.

Importance	16
SenseTriggers	SeeEnemy HearEnemyWeaponFire HearEnemyWeaponImpact SeeEnemyWeaponImpact SeeAllyDeath HearAllyDeath

	HearAllyPain HearAllyWeaponFire SeeAllyDisturbance
Attractors	AINodePanic

FollowFootprint

AI follows an enemy's footprints, then searches the AIRegion.

Importance	14
SenseTriggers	SeeEnemyFootprint
Attractors	None

GetBackup

AI runs to an AINodeBackup to call for help. If allies are within the stimulus radius of the AINodeBackup, and have the RespondToBackup goal, they will follow the AI back to where he last saw the enemy. If AI loses track of his target, he searches the AIRegion.

Importance	30
SenseTriggers	SeeEnemy SeeEnemyWeaponImpact
Attractors	AINodeBackup

Goto

AI pathfinds to a specified node on command.

Importance	8
SenseTriggers	None
Attractors	None

Parameters:

NODE=nodename	Name of node to pathfind to.
MOVEMENT=movement	Type of movement animation (e.g. Walk or Run).
AWARENESS=awareness	Awareness type of animation (e.g. Investigate).

Guard

AI stays within the radius of an AINodeGuard. The AINodeGuard may be assigned through a command, or the AI can find the nearest AINodeGuard. AI may wander within the radius, and activate goals

associated with AINodeUseObject nodes that are guarded by the AINodeGuard. If AI leaves the guard radius to chase an enemy, he will return to the radius when he relaxes.

Importance	5
SenseTriggers	None
Attractors	AINodeGuard

Parameters:

NODE=nodename	Name of node to assign as the AI's AINodeGuard.

HolsterWeapon

AI holsters weapon. This goal only activates if AI has a weapon in his left and/or right hand.

Importance	10
SenseTriggers	None
Attractors	None

Investigate

AI investigates a disturbance. AI pathfinds to a disturbance and looks around, then searches the AIRegion. The AI's movement (walking or running) depends how alarmed the AI is. Disturbances have different alarm levels, and multiple minor disturbances accumulate to equal a larger disturbance. If the disturbed object has an AINodeUseObject pointing at it, the AI may activate the object (e.g. close a drawer). If it is the first minor disturbance, or the AI cannot pathfind to the disturbance, he will just look in the direction of the disturbance.

Importance	13
SenseTriggers	HearEnemyFootstep HearEnemyWeaponFire HearEnemyWeaponImpact SeeEnemyWeaponImpact HearAllyDeath HearAllyPain HearAllyWeaponFire HearEnemyDisturbance SeeEnemyDisturbance SeeEnemyLightDisturbance SeeAllyDisturbance

	HearAllyDisturbance SeeAllySpecialDamage SeeEnemyLean SeeEnemy
Attractors	None

LoveKitty

AI walks to an AngryKitty proximity mine to pet it. If AI has been blown up before, he says something like “bad kitty” and does not walk over.

Importance	9
SenseTriggers	SeeEnemyDanger
Attractors	None

Lunge

AI lunges at a target and causes damage. Only one AI may lunge at a time. AI must have a path clear of static obstacles in front of her. Used for ninja.

Importance	26
SenseTriggers	SeeEnemy
Attractors	None

Menace

AI walks to an AINodeUseObject and places a menace animation. This goal is used to make AI appear agitated when they are actually unaware of a threat. Used in levels where the AI is intended to already know that the player is around.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject MenacePlace

MountedFlashlight

AI turns on/off flashlight mounted on rifle when entering/exiting a dark AIVolume.

Importance	0
SenseTriggers	None
Attractors	None

Patrol

AI patrols on route set between AINodePatrols.

Importance	2
SenseTriggers	None
Attractors	None

PlacePoster

AI places a poster at the location of an AINodeUseObject. Used for police.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject PostingPlace

ProximityCommand

AI runs a command when he comes within proximity of an enemy. Command is set on CAIHuman object in Dedit. Used for HARM guards in HARM villa, who apprehend the player rather than attacking.

Importance	1000
SenseTriggers	SeeEnemy
Attractors	None

Parameters:

RESET=1	Reset the command to run again if AI receives the parameter RESET=1.

PsychoChase

AI chases an enemy relentlessly. No stimulus is required to activate the goal. AI ignores junctions, and never gives up.

Importance	21
SenseTriggers	None
Attractors	None

RespondToAlarm

AI responds to an alarm. Alarms specify AIRegions where AI respond or go alert. If AI is in an alert AIRegion, he looks around. If he is in a respond AIRegion, he runs to the alarm. The alarm specifies a list of AIRegions to search after reaching the location of the alarm.

Importance	19
SenseTriggers	HearEnemyAlarm
Attractors	None

RespondToBackup

AI responds to an ally's call for backup. AI must be standing within the stimulus radius of an AINodeBackup that an AI with the goal GetBackup is running to. Responding AI follow the AI that called for backup back to the last place the enemy was seen, and search the region. If AI has called for backup too many times, without finding anything, allies believe he is crying wolf, and ignore his call for help.

Importance	19
SenseTriggers	SeeAllyDistress. AI loses interest in RespondToBackup if he senses SeeEnemy.
Attractors	None

Retreat

AI jumps back to retreat from an enemy. AI only retreats from enemies wielding melee weapons. Used for ninja.

Importance	25
SenseTriggers	SeeEnemy
Attractors	None

Ride

AI plays a riding animation when at an AINodeUseObject. Used for Pierre getting shot up on steam for the Underwater Base boss fight.

Importance	500
SenseTriggers	SeeEnemy
Attractors	AINodeUseObject with SmartObject Ride

Search

AI searches a list of AIRegions on command. Used for scripting AI to search, without first sensing a disturbance.

Importance	12
SenseTriggers	None
Attractors	None

Parameters:

REGION=list of AIRegions	Name of node to pathfind to.
MOVEMENT=movement	Type of movement animation (e.g. Walk or Run).

SerumDeath

AI dies instantly if hit with AntiSuperSoldierSerum while powered down. Any other damage type reduces hitpoints but does not kill AI. When AI reaches zero hitpoints, he powers down and regenerates full hitpoints, then powers up. Used for SuperSoldiers.

Importance	1000
SenseTriggers	None
Attractors	None

Sleep

AI sleeps at an AINodeUseObject. Visual senses are turned off while sleeping.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject Bed

Sniper

AI stands alert and fires at any enemy in view. If a node is not specified, AI will find the nearest node owned by his AINodeGuard. If AI is shot, he will choose a new node. Sniper nodes may be listed as view nodes of AIVolumePlayerInfos, so that AI chooses the best node to attack an enemy in specified areas.

Importance	29
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject Sniper

Parameters:

NODE=nodename	Name of node to go to, to fire from.

SpecialDamage

AI reacts to special damage types. Special damage includes sleeping, stun, bear trap, glue, laughing, slippery, bleeding, burn, choke, ASSS, poison, and electrocute damage types. AI plays a looping, incapacitating damage animation with a transition in and out. Some damage types knock the AI out, at which point weapons may be stolen. If the AI wakes up unarmed, he panics. Otherwise, he searches the AIRegion.

Importance	1000
SenseTriggers	Anything sensed interrupts the AI's search.
Attractors	None

Parameters:

PAUSE=1 or 0	AI stops decrementing the damage timer when PAUSE=1, and continues when PAUSE=0. This command parameter is used when a player picks up a knocked out AI.
SLEEPFOREVER=1	AI is put to sleep and never wakes up if it receives the parameter SLEEPFOREVER=1. This was used to knock out Cate in Japan co-op.

Talk

AI walks to an AINodeTalk to have a conversation with another AI. AI waits until all dialogue participants are in place. If AI is waiting at the node and notices a disturbance, he may investigate, and then return to the AINodeTalk.

Importance	6
SenseTriggers	None
Attractors	None

Parameters:

RETRIGGER=1	AI retriggers the dialogue object on arrival to the AINodeTalk if it receives the parameter RETRIGGER=1.
DISPOSABLE=1	If AI receives the parameter DISPOSABLE=1, the conversation will never occur if the AI is drawn away to investigate a disturbance.
GESTURE=animname	Name of animation to play while having a conversation. Used for scripting gesture animations from dialogue objects.
MOVEMENT=movement	Type of movement animation (e.g. Walk or Run).

Work

AI walks to an AINodeUseObject and does some work. Work may include anything a SmartObject is set up to do. Work includes desk work, smoking, urinating, stretching, etc.

Importance	4
SenseTriggers	None
Attractors	AINodeUseObject with SmartObject Work

SmartObjects

SmartObjects are defined in AIGoals.txt. A SmartObject is a template that announces an AINode's purpose to an AI, and describes the animation set to play while using the SmartObject. In general, the AI will play a single Action animation, or a looping Idle animation with optional intermittent Fidget animations.

NOLF2 had 87 SmartObjects covering a wide variety of objects and activities, such as desks, microscopes, urinals, places to hammer, smoke, and lean. This document will not enumerate each of the NOLF2 SmartObjects, but will describe the properties and commands that define a SmartObject.

Here are a couple examples of SmartObjects:

// A desk that AI can sit at and do work.

```
[SmartObject1]

Name  = "Desk"

Flag0 = "WorkItem"

Cmd0  = "HumanUseObject Activity=DeskWork Pose=Sit LoopTime=[30.0,60.0] FidgetFreq=
[8.0,10.0] LockNode=FALSE"

AddAnimsLTB0 = "\chars\models\anims\desk.ltb"
```

```
// A file cabinet that serves 2 purposes to AI:

// 1) AI may open and close the cabinet to do work in

//    its default state.

// 2) AI may notice that a player has opened the cabinet,

//    and put it into a disturbed state.
```

```
[SmartObject5]

Name  = "FileCabinet"

Flag0 = "WorkItem"

Cmd0  = "HumanUseObject Activity=FilingHigh LoopTime=[15.0,25.0] LockNode=FALSE"
```

```
Flag1 = "Disturbance"

Cmd1  = "HumanUseObject Action=CloseDrawer LockNode=FALSE"

StateDefault0 = "WorkItem"

StateDisturbed0 = "Disturbance"

AddAnimsLTB0 = "\chars\models\anims\filecabinet.ltb"
```

SmartObject Properties:

Name	Name of SmartObject.
Flag0-N	Type of SmartObject. A SmartObject may have any number of types. Types are listed in SmartObject Types table below.
Cmd0-N	Command for SmartObject type flag with matching index. Parameters for SmartObject commands are described in the table below.
StateDefault0-N	SmartObject types active in the Default state.
StateDisturbed0-N	SmartObject types active in the Disturbed state.
AddAnimsLTB0-N	Child ltb files containing extra animations that need to be loaded for the AI to interact with this SmartObject.

SmartObject Types:

Alarm	AI can sound an alarm at the node.
Attackable	AI can attack from the node.
Bed	AI can sleep at the node.
Coverable	AI can take cover at the node.
DamageType	SmartObject used to define animation resulting from taking special damage (glue, bear traps, sleeping, stun, slippery, poison, ASSS, bleeding, burn, choke, electrocute).
Disturbance	AI can investigate a disturbance at the node.
Examinable	AI can look at something on a wall.
LightSwitch	AI can turn on/off a light at the node.
MenacePlace	AI can play a menace at the node.
PostingPlace	AI can post something at the node. Used for police.
Ride	AI can ride something. Used for Pierre getting shot up by steam.
Smashable	AI can destroy something. Used for SuperSoldiers.
Sniper	AI can shoot someone from the node.
WorkItem	AI can do some work at the node. (e.g. sit at a desk)

SmartObject Command Parameters:

HumanUseObject	All SmartObject commands start with HumanUseObject, because they are setting parameters in the AIHumanStateUseObject.
Action	Action animation property from AnimationsHuman.txt.
Activity	Awareness animation property from AnimationsHuman.txt.

Mood	Mood animation property from AnimationsHuman.txt.
Pose	Posture animation property from AnimationsHuman.txt.
WeaponAction	WeaponAction animation property from AnimationsHuman.txt.
LoopTime	Minimum and maximum for random range of seconds to animate. [0, 0] means loop infinitely.
FidgetFreq	Minimum and maximum for random range of seconds between fidget animations.
LockNode	FALSE if AINode is re-usable after an AI has used this SmartObject.

AI Commands

Level Designers can modify AI behavior by sending commands to AINodes, AIVolumes, and AIs themselves.

Commands to AINodes:

ENABLE	Enable the AINode, so that AI may use it.
DISABLE	Disable the AINode, so that AI will ignore that it exists. If AI was currently at the node, AI will lose interest in whatever he was doing.

Commands to AIVolumes:

ENABLE	Enable the AIVolume, so that AI may use it for pathfinding.
DISABLE	Disable the AIVolume, so that AI will ignore that it exists, and may not pathfind through it.
LIT	Make the AIVolume appear lit to the AI.
BRIGHT	Make the AIVolume appear lit to the AI.
UNLIT	Make the AIVolume appear unlit to the AI.
DARK	Make the AIVolume appear unlit to the AI.

Commands to AIVolumePlayerInfos:

ON	Enable the AIVolumePlayerInfo, so that player may use its hiding properties, and SenseMask will be enabled.
OFF	Disable the AIVolumePlayerInfo, so that player may not use its hiding properties, and SenseMask will be disabled.

Commands to AI:

HIDDEN 1 or 0	AI is hidden if HIDDEN 1, and exposed if HIDDEN 0.
CINERACT animname	AI plays a specified animation once.
CINERACTLOOP animname	AI loops a specified animation.
TRACKLOOKAT	AI tracks player with his head.
TRACKAIMAT	AI tracks player with his head and torso.
TRACKNONE	AI disables head and torso tracking.
ALIGNMENT alignment	Set an AI's alignment. **Alignment is Case Sensitive!! Valid alignments are: GoodCharacter BadCharacter NeutralCharacter
DAMAGEMASK mask	Set an AI's damage mask to a mask listed in AIButes.txt, or to None.
GOAL_xxx name1=value1 name2=value2 ...	Set parameters on a goal that the AI already has, where xxx is the name of a goal. For example: GOAL_chase SEEKPLAYER=1 KEEPDISTANCE=1
GOALSET goalset	Change an AIs goalset. Goalset may be any goalset listed in AIGoals.txt.
ADDGOAL goal name1=value1 name2=value2 ...	Add a goal to an AI, and optionally set parameters on the new goals. For example: ADDGOAL goto NODE=AINodeGoto1 MOVEMENT=run
REMOVEGOAL goal	Remove a goal from an AI.
GOALSCRIPT goal1 name1=value1 name2=value2 ... goal2 name1=value1 name2=value2 ...	Add a sequence of goals to an AI. This is used to script an AI to do a series of actions. There can be any number of goals, and each goal can set any number of parameters. For example: GOALSCRIPT goto NODE=AINodeGoto1 MOVEMENT=run animate ANIM=dance LOOP=1

AI Debug Keys

If you build debug or release configurations, some helpful debugging features will be enabled while running the game. See the documentation on building the Nolf2 source code for more information on this topic.

F2	Spectator Mode: Player can fly the camera around the level freely without physics.
F5	AI Info: AI's names will float above their heads, and a panel of information will be displayed to the AI that the camera aims at. Information includes their current hitpoints, goal, and state. Hitting F5 multiple times displays different types of information.
F7	GodMode: Runs the cheats mpgod, mpkfa, mpmods
F11	Show AIVolumes and Paths: Render wireframes and names for AIVolumes, and draw AI paths. Hitting F11 multiple times cycles through 4 modes: Show AIVolumes Show AI Paths Show AIVolumes and Paths Show AIVolumePlayerInfos
SHIFT + F11	Show AINodes: Render wireframes and names for AINodes. Hitting SHIFT + F11 multiple times cycles through the different node types.

Content Guide

NOLF2 AI Gameplay Setup

This document describes how the nolf2 AI systems were used to setup basic gameplay. This is not an exhaustive description of every piece of the AI systems, but instead an overview of the most commonly used pieces, highlighting things that may not be obvious by looking at the code or Dedit properties.

GoalSets

In general, AI in nolf2 are assigned either the Patrol, Guard, or Reinforcement GoalSet. There are a total of 46 GoalSets defined in aigoals.txt, but most are variations of Patrol or Guard for special characters (e.g. Ninjas, SuperSoldiers, Roots, etc). Please see aigoals.txt for details of what Goals are in each GoalSet.

Patrol, Guard, and Reinforcement share identical combat behavior, but vary in their relaxed and investigative behaviors. Below are descriptions of the expected behaviors with each of these GoalSets.

Patrol GoalSet

- The Patrol GoalSet includes the DefaultRequired and DefaultBasic GoalSets to define the standard combat behavior.
- Patrol's relaxed behaviors are defined by the Goals Patrol and Work.
- AI will walk along a patrol path defined by LevelDesign through AINodePatrols.
- A patrolling AI owns his path, and no other AI can use it until he dies or otherwise relinquishes it.
- A patrol path can be assigned by specifying a node on the path in the AI's Initial Command: goal_patrol node=AINodePatrol9.
- If no node is specified, AI will find the nearest unowned AINodePatrol, and take ownership of that node's path.
- A patrolling AI may not own an AINodeGuard. Patrol and Guard are mutually exclusive.
- If a patrolling AI walks near an unowned work node (AINodeUseObject with SmartObject set to Work), he will stop patrolling to do some "work," then continue patrolling. Patrolling AI will ignore any work nodes that are owned by an AINodeGuard.
- A patrolling AI may animate an investigative walk rather than the routine patrol walk through the command: goal_patrol awareness=investigate. This is useful for levels where AI are intended to be eternally suspicious regardless of stimuli.
- Patrolling AI will turn off lights as they leave an AINodePatrol, and turn on lights at a destination AINodePatrol.

Guard

- The Guard GoalSet includes the DefaultRequired and DeafultBasic GoalSets to define the standard combat behavior.
- The Guard GoalSet includes the Sniper Goal.
- Guard's relaxed behaviors are defined by the Goals Guard, Work, and Menace.
- A guarding AI will stay within the return radius of his guard node. If he is drawn outside of this radius, he will walk to return to it.
- A guarding AI owns his guard node, and no other AI can use it until he dies or otherwise relinquishes it.
- An AINodeGuard can be assigned by specifying a node in the AI's Initial Command: goal_guard node=AINodeGuard22.
- If no node is specified, AI will find the nearest unowned AINodeGuard.
- A guarding AI may not own any AINodePatrols. Patrol and Guard are mutually exclusive.
- The guarding AI owns his guard node, and the guard node may own additional nodes. Owned nodes are AINodeUseObjects used for the Work, Menace, and Sniper Goals.
- An AI that owns a guard node will randomly select work nodes owned by his guard node, rather than always going to the next nearest work node. This AI may not use any unowned work nodes. The same applies to menace nodes.
- Menace nodes are used exactly like work nodes, but are used in levels where the AI is supposed to already be aware of the enemy's presence, and should appear permanently agitated, even when the AI system thinks the AI is relaxed. In other words, when the AI is not in combat he will run between menace nodes (AINodeUseObjects with SmartObject set to Menace), and play aggressive animations.
- When a guarding AI investigates a disturbance, he will stop at the edge of his guard radius and play an alert animation, rather than walking all the way to the disturbance.
- A Guarding AI will not go to search nodes outside of his guard radius.
- If the guarding AI has previously seen the player, heard an alarm, or been shot, he will ignore his guard radius and investigate anywhere. Once the AI returns to a relaxed state of awareness, he will start paying attention to his guard radius again.
- If the AI notices a disturbance outside of his guard radius, he will not speak. This prevents an AI from repeatedly saying "I hear you!," "What was that!," "I know you're there!," but never coming to investigate.

- If an AI currently owns a talk node, he will ignore his guard node. This is covered in more detail later in the section about the Talk Goal and conversations.

Reinforcement

- The Reinforcement GoalSet is used for AIs who spawned as reinforcements, usually from a command in an alarm.
- The Reinforcement GoalSet includes the DefaultRequired and DeafultBasic GoalSets to define the standard combat behavior.
- The Reinforcement GoalSet includes the Sniper Goal.
- Reinforcement's relaxed behaviors are defined by the Goals Guard, Work, ExitLevel, and Menace. (See the Guard GoalSet above for more info on Menace).
- A reinforcement AI will enter the level and respond to whatever is currently going on. Once the situation dies down, and the AI returns to a relaxed state of awareness, he will look for unclaimed guard and patrol nodes. If one is found, he will claim a node and activate the Guard or Patrol goal, and behave like an ordinary AI that is guarding or patrolling.
- Nodes may be unclaimed because the AI that previously owned them have died. In this case the reinforcements are replenishing the level to maintain some number of AI, and keep the level at approximately the same difficulty.
- Alternatively, the level may intentionally contain a surplus of nodes, so that the level will become harder once reinforcements are brought in.
- If no unclaimed nodes are found, the AI will look for an AINodeExitLevel. If an ExitLevel node is found, the AI will walk to the node and get removed from the level. These are normally placed in areas the player cannot get to, so it is not obvious the AI is getting instantly removed from the game.
- The AI will not look for an ExitLevel node until he has been in the level for at least 5 seconds. This restriction prevents an AI from spawning near an ExitLevel node, and immediately removing himself before noticing an alarm sounding, or other stimuli.

Scripting

Scripting was avoided as much as possible in nolf2, but there were situations where some amount of scripting was necessary to drive the story or vary the gameplay for specific levels. Here are the ways we accomplished scripted behaviors without breaking the autonomy of the AI.

- **Enable/Disable Node Commands**: The majority of scripting in nolf2 involved enabling and disabling nodes, or assigning new nodes with command, such as: goal_guard node=AINodeGuard45. This gives control over where AI are idling while still allowing some randomness and allows them to react normally to whatever situation they find themselves in.
- **GoalScripts**: GoalScripts supply an AI with a sequence of Goals to complete. When one Goal is satisfied (e.g. the AI has arrived at the destination node of a Goto Goal), the next Goal activates. GoalScripts may be interrupted if the AI notices a disturbance, and the GoalScript will continue once the AI returns to a relaxed state of awareness. GoalScripts are useful for making an AI go fro place-to-place and animate.
- **Search**: Normally AI search after activating other Goals (e.g. after investigating or chasing and losing the target). In cases where AI are spawned with the intention of immediately searching, they may be given the Search Goal, which is a scripted Search. AI with the Search Goal can be given a list of comma-separated regions to search: addgoal search region=airegion01,airegion02,airegion03. AI who search through normal means will only search their current region.
- **Patrol**: Another way to script searching behavior is to spawn AI and assign them patrol routes. The Patrol Goal may be instructed to play investigative walking animations: addgoal patrol awareness=investigate. In some nolf2 levels, we periodically spawn a patrolling AI to ensure the level will never be completely empty (and boring to the player), even when all AI have been killed leaving no one to sound alarms. An AI may be spawned, instructed to patrol, and then removed through a command on a destination patrol node.

Conversations

The Talk Goal allows AI to coordinate meeting up, having a conversation, and going on to do other things. The Talk Goal also handles responding to interruptions, and possibly returning to have the conversation later. There are basically two options for setting up AI to have a conversation:

Statically Positioned Conversation Participants:

- AI are placed at AINodeTalk nodes and given initial commands of: addgoal talk node=AINodeTalk04. AI will walk to Talk nodes and wait for a dialogue object to be triggered.
- If both AI are within the radii of their Talk nodes when the corresponding dialogue object is triggered, they will have their conversation.
- When the conversation ends, the dialogue object will run its finished and cleanup commands.
- An AI with a Talk Goal and Talk node will ignore his Guard and Patrol Goals. The Talk Goal overrides other relaxed goals, so the AI may still have a full GoalSet, such as Patrol or Guard.
- If an AI is at a Talk node, and notices a disturbance, he will investigate. Guard node radii will be ignored.
- If the dialogue object is triggered while one or both of the dialogue participants are not relaxed, the dialogue will not play, and the dialogue object's cleanup command will be run.
- If the dialogue object was not triggered while the AI was investigating, when the AI returns to a relaxed state of awareness, he returns to his Talk node and continues waiting for the dialogue to be triggered.
- The Talk Goal parameter disposable=1 tells the AI not to return to his Talk node if he was disturbed while waiting for the conversation. In this case, the AI will simply return to his normal relaxed behaviors and forget about having the dialogue.

Dynamically Positioned Conversation Participants:

- AI may be anywhere in the level, running various Goals. They do not need to be near their Talk nodes.
- When a dialogue object is triggered, commands are sent to the dialogue participants: addgoal talk node=AINodeTalk09 retrigger=1. If either AI are outside of their Talk node radii, the dialogue will not start. The parameter retrigger=1 tells the AI to retrigger the dialogue object once the AI arrive at their Talk nodes.
- Once both AI are in position, the conversation starts.
- When the conversation ends, the dialogue object will run its finished and cleanup commands.
- The above steps may be used to get AI to walk towards each other and start talking, or to make an AI stop working and start talking to someone.

A couple more general notes about the Talk Goal:

- While the Talk Goal is active, AI will face each other and talk while playing their basic idle animation (usually 3StGd).
- AI may be instructed to play gesture animations at key points in the conversation. From the dialogue object, send a command to the AI: goal_talk gesture=NameOfAnim.
- If a dialogue object is triggered, and one or more of the dialogue participants does not exist (probably because he is dead), the dialogue will not play and the dialogue object's cleanup command will run.

Goal Notes

Below are points of interest about various Goals:

Chase

- Once the Chase Goal is activated (by seeing the enemy) the AI will never lose interest in chasing, with the exceptions described below.
- An AI may lose interest in chasing if another goal activates that puts the AI in a state of awareness other than alert (so, relaxed or suspicious). Normally any Goals that activate in favor of Chase are other alert Goals, such as AttackRanged. The exceptions are Goals like SpecialDamage, which incapacitate the AI and then send him searching for his attacker. When an AI wakes up from SpecialDamage, he is investigative and loses interest in chasing.
- An AI may lose interest in chasing if he loses his target in a Junction volume, and gives up.
- Chase Goal Parameters:
  - **SeekPlayer=1**: This parameter instructs the AI to immediately start chasing a player, even if no stimuli have been sensed. This allows the AI to cheat in combat levels where the AI is intended to already know where the player is. The AI ignores Junction volumes while seeking the player. Once the player has been seen, the AI falls back to normal chasing behavior where he can get lost in Junction volumes.
  - **NeverGiveUp=1**: This parameter instructs the AI to cheat indefinitely. The AI will always know where the player is, and will never lose interest in chasing. This parameter was used for the nolf2 ninja trailer park level, where combat should never stop, but ninjas may get temporarily incapacitated when hit with SpecialDamage (e.g. poison).

AttackFromCover

- Cover nodes have an optional parameter Object. This Object may be an object like a table that can be flipped on its side to use for cover. If an AINodeUseObject also points to this object, and the UseObject node has a SmartObject set to something for cover, the SmartObject can specify what animation the AI should use when activating the Object. For example, SmartObject FlipTable tells an AI how to animate when flipping a table to use as cover. After flipping the table, the AI runs to the Cover node, and runs the normal AttackFromCover behavior.

AttackFromVantage

- AINodeVantage may specify an optional Object property. This object will be triggered On before the AI starts attacking from the Vantage node, and triggered Off after the AI attacks. This could be used to open and shut window shutters, for example. This Object was used in nolf2 to allow Pierre to open and shut steam hatches in the underwater base boss-fight.
- There are a couple Goals derived from AttackFromVantage that may be useful, or may be good templates for doing similar things.
  - **AttackFromRandomVantage** works exactly like AttackFromVantage, but does not require any stimuli to activate the Goal, and chooses valid Vantage nodes at random instead of choosing the nearest valid node. This Goal is used in nolf2 for the Pierre boos-fight in the underwater base, where Pierre pops out of random steam hatches to throw knives at the player.
  - **AttackFromRoofVantage** works exactly like AttackFromVantage, but uses AINodeVantageRoof attractor nodes instead of normal AINodeVantages. An AI may only activate AttackFromRoofVantage if no other AI already has the Goal active, and only if at least 3 AI are already attacking the same target. This Goal is used in nolf2 to control when ninja AI attack from rooftops.

AttackFromView

- An AI will attack from an AINodeView if the AI has sensed stimuli of the enemy's presence, and cannot find a path to the enemy, and the enemy is in an AIVolumePlayerInfo that points to a View node that the AI can find a path to.
- **SeekPlayer=1**: This parameter instructs an AI to attack from a View node without first sensing any stimuli.

Work

- AINodeUseObjects set to SmartObjects for work are used with the Work Goal. These nodes control most of nolf2's relaxed behaviors (smoking, working at desks, dancing, leaning, etc).
- There are various Goals derived from Work that are nearly identical Goals, but are attracted to different types of Smart Objects:
  - **Sleep**: AI sleeps in a bed or chair, or while standing. Visual senses are disabled while sleeping.
  - **PlacePoster** and **EnjoyPoster**: Used in nolf2's India levels for police placing posters and civilians looking at posters.
  - **Destroy**: Used for nolf2's SuperSoldiers to destroy things by smashing or shooting.
  - **Ride**: Used for nolf2's underwater base boss-fight to levitate Pierre on steam bursts.
- If a Work node has a ReactivationTime of zero, an AI may not use this node again until first using another Work node. Any ReactivationTime greater than zero will allow an AI to re-use the same node once the node is active again.
- An AI can be instructed to immediately stop working at a node by sending a Disable command to the node.
- If an AI is interrupted in the middle of working, he may optionally play an interrupt animation. For example, an AI that is smoking may abruptly throw down his cigarette instead of calmly putting it out. When the Work Goal deactivates, it will play an interrupt animation if it exists in animationsCharacterName.txt, otherwise the AI will play its normal out transition if it exists.
- If an AI is hit with SpecialDamage while working, he may optionally play a specific SpecialDamage animation rather than transition out of Work normally. For example, and AI may slump over and pass out at his desk, instead of standing up, pushing in his chair and then passing out. This animation actually plays in the SpecialDamage Goal.
- AI may play a specific death animation corresponding to his work. For example, slumping over a desk, or falling out of a chair.

SpecialDamage

- The SpecialDamage Goal handles damage types that are specific to nolf2, but may act as a template for similar functionality in other titles. It handles both instant and progressive damage, and includes the following damage types:
  - Sleeping
  - Stun
  - Bear_Trap
  - Glue
  - Laughing
  - Slippery
  - Bleeding
  - Burn
  - Choke
  - ASSS (Anti SuperSoldier Serum)
  - Poison
  - Electrocute
- Incapacitating animations and transition animations are defined through SmartObjects. Note that this goal uses SmartObjects to drive animation without the presence of an AINodeUseObject.
- Instead of playing the normal in-transition animation, the Goal may play an animation specific to the AI's current animation, as described previously in the Work Goal. This allows an AI to slump over on his desk when knocked-out. This also allows an AI to get knocked out while sleeping or prone without getting up first.
- AI may play a specific death animation corresponding to the damage. For example, dying instantly without moving while knocked out.

Content Guide

Working With AI

AI (Artificial Intelligence) refers specifically to any characters in your game that are not controlled by a human player. All characters not controlled by the player are controlled by the computer, and by design they must know how to interact with players and perform actions of their own. These actions include, but are not limited to: standing still, attacking, patrolling, talking, searching, eating, responding to alarms, smoking, and working. AIs must know when to perform these actions, how to perform these actions, and when they cannot peform these actions.

While the Jupiter engine code itself does contain any AI specific code, the NOLF2 game code supplied with the Jupiter engine contains a large number of classes and objects specifically for AI. This section focuses on how AI works using NOLF2 game code. Specifically this section explains how Level Developers can implement NOLF 2 AI in their own levels using the NOLF2 game code. The purpose of this section is to serve as a reference. Developers of unique games should create their own AI classes and objects, and use the NOLF2 AI game code only as a reference.

Regardless of how you choose to implement your own AI, making the tools necessary for level designers to implement and create AI interaction through DEdit creates a quick and easy way for your Level Designers to create levels (worlds) that are interactive and interesting to the player. If you choose to implement AI strictly through a programming interface (API), then it becomes far more difficult for your level designers to see and test how their AI characters operate and interact with the players in the game. Because of this, you are advised to make your AI classes and objects available to Level Designers through the DEdit application.

This section contains the following topics specific to implementing NOLF 2 AI through DEdit:

About the NOLF2 AI	Describes the fundamentals of how the AI system in NOLF2 operates, the base classes used, and the types of objects applied through DEdit.
Using Regions and Volumes	Describes how to specify where an AI can see and travel.
Using AI Brains and Goals	Describes how to specify actions for an AI under different conditions.
Using AI Senses	Describes how to use and modify the senses AIs use to precieve their environment.
Using AI Attributes	Describes how to use and modify the general attributes of AIs such as hit points, armor, jump speed, and others.
Scripting AI Goals	Describes how to create goal scripts to specify multiple actions for an AI.
Using AI Commands	Describes the commands available to send as player messages.
Debugging AIs	Provides instuctions for debugging AIs.

Creating Cinematic AIs	Describes general rules for creating AIs that talk in cut-scenes.
AI Tutorial	Provides a walk-through set of steps describing how to create a simple AI interaction in a level.

Top

Content Guide

About the NOLF2 AI

The NOLF2 AI is a complex system of components integrated thoroughly with the game code. Because the AI system is not entirely modular, you may find it difficult to extract and implement the system in your own games. A better approach for Developers attempting to implement their own AI is to use the NOLF2 AI game code as a reference. This section is intended for both Developers and Level Designers, and provides a general overview of the components used by the NOLF2 AI, and the objects used in DEdit to implement AI in a level.

Read the following sub-sections to get a general understanding of how the NOLF2 AI operates:

- About the Operation of NOLF2 AI
- About the NOLF2 AI Components
  - About DEdit AI Objects

About the Operation of NOLF2 AI

NOLF2 does not differentiate between NPC and combat AI. There are a handful of generic systems that work in combat and non-combat situations, for both friendly NPC and enemy behavior. The major systems are Pathfinding, Goals, SmartObjects, Stimulus and Senses.

AIs are supplied with a set of Goals that determine their actions. Actions are selected based on an AI's proximity to SmartObjects and on sensed stimuli. The NOLF2 pathfinding system is unique in that it not only finds a path, but also instructs an AI what type of movement is required. For example, a ninja may run from point A to point B, or may leap over cars, fences or trailers to get there.

Collaboration

Each of the AI systems is fairly simple on its own. The challenge comes in anticipating what will happen when all of the systems work together. What happens when an AI is working at his desk and he sees an ally run by on fire? What happens when a ninja spots the player and jumps from rooftop to rooftop to try to get to an alarm, or when she is suddenly electrocuted while in mid-air? These are the types of situations that arise in NOLF 2.

Many action games subscribe strictly to the "monster closet" game design philosophy, where AIs stand still waiting for the player to trigger them to rush out and attack. Most of the levels in NOLF 2 are populated

with AIs who are going about their business. The player is free to explore the level, and the AIs will respond to their observations of the world around them.

Relaxed AIs wander between SmartObjects where they take naps, use restrooms, do deskwork, or turn on a radio and dance a jig. If an AI is at his desk and notices someone turned off a light, he will get up and investigate, turn the light back on and go back to his desk. In a combat scenario, an AI may knock the desk over and take cover behind it, while blindly firing over the top at an enemy who barged into his office.

Alerted AI will chase the player, but there are always opportunities to elude the enemy. For example, the bear trap can stop the pursuers in their tracks, buying the player time to find a hiding spot. When the enemies finally free themselves, they will activate goals to alert allies and sound alarms to bring in reinforcements. As new AIs join the hunt, they will spread out to search the area, but if the player is not found, the aggressive goals will deactivate as the AIs concede defeat and return to their relaxed behaviors. The goal system provides AIs with lower priority behaviors to fall back to, giving the player the opportunity to recover from a dangerous situation.

Monolith Level Designers have filled the levels with SmartObjects that provide opportunities for the AI to showcase their behaviors. Many levels are packed with desks, drinking fountains, alarms, radios, microscopes, urinals, roofs to jump on, cars to jump over, beds, light switches and lots more. The AI dynamically decides to activate goals that are appropriate for their current situation, making gameplay unpredictable and replayable, adapting to however the player approaches the level.

The Goal-Based Approach

NOLF 2 uses a goal-based approach to AI design because it produces a living world with AIs that behave believably and have some purpose other than killing the player. This approach also creates a system that lets players use stealth. If players are careful, they can sneak around observing the enemy unnoticed. If players are discovered, they can get away.

The advantage of a goal-based system is that it gives the AIs' behavior consistency. Each individual AI does not need to be scripted to respond to different situations. Goals take care of responding appropriately to whatever situation an AI finds himself in. This gives depth to the gameplay, as the player has freedom to do whatever they want. For example, in one of the earlier missions, Cate is assisted by a Soviet pilot. If the player wants to tranquilize the pilot, they have the option to perform this action.

Other benefits of goal-based AI are that it's easier to code, and easier to globally modify behavior. Each goal is its own module, so each behavior can be tested individually. Once it works on its own, the goal can be added to an existing goal set, and instantly, every AI in the game that has this goal set has a new behavior. This is a lifesaver during bug-fixing beta periods, as bug fixes to AI behavior can usually be applied to the entire game without requiring any work from the over-loaded Level Designers.

Development and Testing

Once an AI is supplied with a full palette of goals (our average AI has 26 goals in its goal set), the AI may behave in unexpected ways. This unpredictability is great for gameplay, but can cause trouble during development and testing.

he flexibility of the AI goal system allows for a shared base of low-level functionality upon which we can add specific behavior and tendencies unique to each specific AI type. This works well to help keep things fresh as the game progresses. Instead of a new enemy AI being the same as the last but with a different model skin, facing a new AI type becomes a new obstacle for the player to overcome, requiring adaptive strategies and a degree of problem solving.

The most notable aspect of the NOLF 2 AI compared with many others in the genre is the AI's unscripted awareness of the world around them. They really know what is going on, and they interact with the environment and incorporate feedback into both their idle activities and combat strategies. This allows the AI to act and react as you would expect a real opponent to.

The major limiting factor in the level of complexity of the AI is development time. Every new behavior takes time to tweak and to perfect its interaction with other existing behaviors. Sometimes, problems do not surface immediately because different levels provide different opportunities for the AIs to showcase their behaviors.

The engine's performance also factors into the number of AIs that can be active at once because AIs need to share the processing resources for physics and visibility calculations. Each AI needs to do expensive physics calculations to find the floor while they are moving. Expensive line of sight calculations are also required to allow the AIs to determine what is visible to them.

Monolith has put a lot of effort into making its AIs believable. AIs can split up to encircle a target, alert their comrades to threats, even shout at each other to get out of the way if they can't get a clear shot at the player. They work at desks and drink coffee and go to the restroom. They even sometimes trip and fall on their faces when vaulting over a railing to chase the player. These behaviors help make enemies seem believable, but if it is a choice between what's realistic and what's fun, fun always wins. For example, early on Monolith had enemies carry out organized, efficient searches when they were alerted to the player's presence. The problem was that they always found the player, just like they would in real life. So Monolith made them slightly less competent. The AIs still search, which feels authentic, but instead of looking in the really likely hiding spots, they search NEAR the likely hiding spots, which makes it easy for you to observe them without getting caught. Monolith refers to this as "movie realism."

Bug Problems

All of the NOLF 2 human AIs respond to tranquilizers and tazers by passing out. This leads to some interesting problems in UNITY headquarters, where the player can tazer a friendly scientist, then activate him while he laid on the ground sleeping. He would turn his head and say "Hi Cate, Mildred said to say thank you for the flowers."

The addition of ambient life, such as rats and rabbits, sounded simple enough. Ambient creatures are simply AI with a very limited goal set. They wander around, and run in fear from humans. Even the simplest addition can cause unpredicted results. The first problem was that our interactive music system is tied to the collective AI state of awareness. When the player scared a rabbit, the music intensity would go up to its aggressive themes. This made scaring a rabbit a very dramatic event.

The NOLF 2 AIs are also aware of each other, and notice if someone else is alarmed. This gives some desirable behavior for free. If the player agitates a rabbit and it runs by the enemy, it causes a disturbance that may alert the enemy to the player's presence. The problem was that rabbits were getting alarmed by one AI, and running by another AI. This meant that even if the player had never even moved yet, the AI were already suspicious of an intruder.

About the NOLF2 AI Components

The NOLF2 AI aggregates many sub-components to implement its logic. The list below describes some of the sub-components used to implement AI logic:

CAITarget—This class manages tracking of the AIs current target. It updates the target's visibility, position, threat status, and other attributes.

CAIGoalMgr—This components is responsible for creating autonomous behavior in AIs. It maps sensory events into state transitions, and implements a higher level logic than what exists at the machine level.

CAISenseMgr—This component is responsible for managing all of the AIs senses. Each individual sense is implemented as a derivative of CAISense. The senses track the state of their stimulation and reaction times against a stimulus.

CAISense—This is the parent class for the CAISenseMgr.

CAISenseRecorder—This component handles remembering what sensory events the AI has already experienced.

CAnimationMgr—This is the parent class that handles animations of all AIs.

CAnimationContext—Part of the CAnimationMgr system specific to human AIs.

CAnimator—The parent class for all non-human animations.

CAIState—The parent class for all CAI states. The AI state is the most important logical aspect of the AI, and performs the bulk of decision making for the AI. Some examples of state are: attack, chase, check body, talk, and search. The state shares a lot of the similar Update and Handle methods as the AI, because the AI commonly defers these methods down to the state. For example, the state has a Pre/Post/Update, a call to update senses, a call to update animation, a call to update the dynamic music, etc. The state also handles similar events such as AI damage, trigger commands, model strings, and senses. AIs switch states in one of three ways: by being send a command by a level designer, by the goal subsystem, or by a message from another state. All state transitions occur by the AI receiving a text string with the name of the state and its parameters. State text string messages are handled by CAI::HandleCommand and CAIState::HandleNameValuePair.

CAIHumanStrategy—Most states share some common sub-behaviors. Sub-behaviors are encapsulated in strategies. The CAIHumanStrategy base class contains sub-states that present a common interface to the state. For example, the CAIHumanStrategyCover base class is used in CAIHumanStateAttackFromCover. The concrete base classes that actually get instantiated are CAIHumanStrategyCover1WayCorner, CAIHumanStrategyCoverBlind, and all of the other cover type base classes. The CAIState uses the same common interface to the base strategy. The term "Strategy" comes from design patterns.

Top

About DEdit AI Objects

The objects discussed in this topic are the components available through DEdit that allow level designers to implement the NOLF2 AI in their levels. Like all DEdit objects, each AI object has properties that allow you to set the specifics of the AI actions and reactions.

CAI obects

The CAIHuman class is the basis for all AIs in the NOLF2 game code. When you insert a CAI object into your level, you must specify a model, brain, and basic goalset for the AI. For additional information, see Using AI Brains and Goals.

AI volume objects

AI volumes allow AIs to pathfind, controlling where they can move. AIVolumes are used strictly for navigation, and do not control raycasting or what the AI can see. AIs never move into areas where AIVolumes do not exist unless they are killed, blasted, or otherwise moved against their will. For specific information about the different types of AIVolumes, see Specifying an AIVolume.

AI node objects

AI nodes are locations and areas within an AI volume that specify particular actions (and animations) an AI can perform based on their current goals. Some examples include: AINodeGuard, AINodePatrol, AINodeUseObject, and AINodeCover. For additional information, see Using AINodes and Goals.

AI regions

AI regions are a collection of AIVolume objects bound to a single AIRegion object. Each AIVolume must belong to one, and only one, AIRegion. The primary purpose for AIRegions is to for searching. AIRegions also control AI response to alarms, and allow you to set which AIRegions respond or become alert when an alarm gets sounded. Additionally, you can specify regions that responding AI search after they have responded to an alarm and visited all of the AINodeSearch nodes in the region the alarm exists in. For additional information, see Specifying an AI Region.

AI goals

Each character AI has a default goal set of "None" when they are initially placed in a level. You can change this goal, giving the character an initial goal equivalent to the action you want them to perform. You can set initial goals in the Commands parameter of the CAI class. During gameplay you can add or remove goals from AI by using the addgoal and removegoal commands. For additional information, see Using AI Brains and Goals.

AI commands

CAI objects respond to message commands just like any other object in the NOLF2 game code. The Using AI Commands section provides a detailed list of each of the commands an AI can receive. During gameplay you can send commands to AIs to perform a variety of operations, including: modifying attributes, changing alignment, moving to a location, activating, facing a specific direction, and many others. You can also use commands in AI goal scripting to create cinematic sequences and activate specific animations.

Top

Content Guide

Using Regions and Volumes

To control where your AI characters can move and navigate, you must use the AIRegion and AIVolume objects. Typically, you will create many AIVolume objects in your level. Each AIVolume is linked to an AIRegion, which controls the AIVolumes an AI can search, and also controls how they respond to alarms.

All AI characters must exist within an AIVolume or they cannot navigate or move within the level.

This section contains the following topics on regions and volumes:

Specifying an AI Region

The AIRegion node controls the set of AIVolumes an AI can search. All AIVolume nodes must be bound to an AIRegion. The placement of AIRegion nodes is not important, only the volumes that are bound to them.

Regions also allow you to specify which AIs respond to an alarm. When you place an alarm Prop object in your level, you can select the regions that go on alert, respond to the alarm, and which regions get searched after the AIs respond to the alarm.

Specifying an AIVolume

This section lists the properties and placement information for the AIVolume nodes available through DEdit using the NOLF 2 game code. For additional information about AIVolumes, see Placing AI Volumes in the AI Tutorial section.

One major point to make about AIVolumes in general is that NO AIVolume should ever overlap. You should always place AIVolumes, regardless of their type, directly on the border of another AIVolume. The only exception to this rule is the AIVolumePlayerInfo volume which can be placed anywhere in any other volume.

AIvolumes have the following general properties:

Property	Description
Lit	This Boolean value allows you to determine the lit or unlit status of a room when the level starts. AIs are still able to see the player when Lit is False, however they cannot see other objects in the volume.
Region	Allows you to specify the AIRegion node for a volume. All AIVolumes must be bound to a region.
PreferredPath	This Boolean value allows you to specify whether or not the AI prefers this volume as a path when patrolling. Robots only tread on preferred paths.
LightSwitchNode	This optional property allows you to point to a lightswitch prefab that lights or unlights the volume. You must specify the lightswitch prefab using the .node suffix (for example PrefabLightSwitch.node).

The following table describes the AIVolumes that currently exist in the NOLF2 game code:

Volume	Description
AIVolume	For all AI, the standard AI volume specifies a volume in which an AI can exist. AIVolumes must be directly next to one another for AIs to pass between them. AIVolumes must be bound to an AIRegion node to specify the region the AI can sense across. • **Minimum Size:** 48 X 16 X 48. Making doors smaller than 48 units wide is allowed. The recommended size for door volumes is 32 units deep and less than 48 units wide. • **Placement:** Place at least 16 units off the ground. In order for an AI to pass between volumes, the volumes must have an abutment of a minimum of 48 units. AIVolumes may overlap, but ideally you should place the borders directly on top of each other. AIVolume that overlap may cause errors, and AIVolume borders that are not in contact with each other results in a separation that AIs cannot pass through. For specific instructions on placeing AIVolumes, see Placing AI Volumes in the AI Tutorial section.
AIVolumeAmbientLife	Ambient life specifies a volume that animals such as rats and cockroaches can move in. Normal CAI that are human cannot move into AmbientLife volumes, so they are often used to send rats and rabbits into areas that humans cannot travel (such as under beds, and into conduits).
AIVolumeJumpOver	For Ninjas only, this volume allows an AI to jump over an obstacle or from ledge to ledge. • **Placement:** JumpOver volumes require 48 DEdit units on each side of the obstacle or ledge. The blue arrow must point along the path of the JumpOver volume. The AI reaches the height of its jump arc at the top middle of the volume.
AIVolumeJumpUp	For Ninjas and Soldiers only, this volume flags an AI of an area where they can jump straight up or down from a ledge or roof.

AIVolumeJunction Used in areas where an AI can take more than one path. AIs chasing the player can "lose" the player if the player passes through a junction. The AI decides which direction to go based on the settings of the junction.

- **VolumeNorth:** Specifies the volume that borders to the north of the junction in the Top view.
- **VolumeSouth:** Specifies the volume that borders to the south of the junction in the Top view.
- **VolumeWest:** Specifies the volume that borders to the west of the junction in the Top view.
- **VolumeEast:** Specifies the volume that borders to the east of the junction in the Top view.
- **Volume<direction>Actions:** The following four actions exist for each of the four directions listed above:
  - **Nothing—**The AI performs no action other than choosing another direction.
  - **Continue—**The AI continues through the volume (used for hallways).
  - **Search—**The AI performs a search in the adjoining volume, and searches through the rest of the AIRegion the volume is in. (You must place at least one search node in each region).
  - **Peak—**The AI performs a peaking animation and then selects another direction.
- **Volume<direction>Chance:** Determines the percentage chance that the AI selects a particular action when they enter the junction. Use decimal percentages to specify (for example 0.5 for 50%). You are allowed to have multiple actions for each possible direction.

AIVolumeLadder Used to flag AIs with climbing animations of a volume that they can climb.

AIVolumeLedge Used to flag an AI that it should fall from the side of a building or ledge. Point the blue arrow toward the side of the ledge to specify the direction the AI of the fall.

AIVolumeStairs Used to flag an AI that it should roll downward when it gets killed. This volume does not control the stair-stepping animation of an AI, which is preformed by default when a standard AIVolume node exists over sloped or staired world geometry.

- **Placement:** Place this volume 16 units above the stairs stretching down to the bottom of the last step (or edge of a slope). Point the blue arrow in the volume in the direction you want the AI to roll (typically the bottom stair or bottom of the slope).

AIVolumeTeleport Teleport volumes specify an area AIs can teleport from. You must specify a destination volume.

AIVolumePlayerInfo A subclass of AIInformationVolume, the AIVolumePlayerInfo is used to tell the AI when they can and cannot see the player. The ViewNodes property points to View nodes that allow the AI to see the player when they are inside the player volume. This allows the player to become hidden from an AI when they move into the volume, but still tells the AI that there is a location they can move where they can see the player. When the player moves into an AIVolumePlayerInfo volume, the AIs targeting the player or chasing the player immediatly check the ViewNodes parameter and move to the nodes where they can then view the player. If no nodes are specified in the ViewNodes property, then they AI simply ignores the character, or searches the areas when disturbed.

- **Placement:** Typically, PlayerInfo volumes fill an entire room from wall to wall and floor to cieling.
- **SenseMask:**
  - **None—**Does not apply a sense mask.
  - **EnemyPermitted—**Masks an enemy AIs senses so that a player can walk freely within the volume without being attacked (unless the player attacks the enemy).
- **On:** This Boolean value determines the On/Off state of the PlayerInfo volume.
- **Hiding:** When True, the player is hidden from the AI (unless the AI gets very close to the character). Turn Hiding on to simulate a dark room.
- **HidingCrouch:** When True, the player is hidden (see above) only when they are crouched.
- **ViewNodes:** Specifies the AINodeView nodes used by AIs when they detect the player using this AIVolumePlayerInfo volume. This allows AIs to move to a location (the AINodeView node) where they can see the player. For example, if you place the AIVolumePlayerInfo volume behind a pillar, then place the view node at an angle where the AI can see behind the pillar. When the player fires from within the AINodePlayerInfo volume, the AI automatically check the volume to see if there is a AINodeView node associated with it. If so, then the AI move to the AINodeView node to fire at the player. You must only point to AINodeView nodes, do not attempt to use this feature with AINodeVantage or AINodeCover nodes.

Top

Content Guide

Using AI Brains and Goals

When you place a CAI in a level, you must set the **ModelTemplate**, the **Brain**, the **GoalSet**, and the **Attachments** for the CAI to operate correctly. Each of these four properties need to match or the CAI may end up hovering in the air on fire, or not even appear in the game.

Another important property is AttributeOverrides, which allows you to modify an AIs senses. For more information about AttributeOverrides, see Using AI Senses.

All human AIs have a CAIBrain object (in code) that specifies a collection of behaviour-related attributes. In DEdit, the CAIBrain selection appears in the Brain property in the Properties tab for every CAIHuman you insert into your level.

Brains must mach a characters ModelTemplate because the ModelTemplate for each CAI can only perform a specific set of animations. If a brain does not match a ModelTemplate then errors result when the CAI attempts to perform actions its brain tells it to do, but that it has no animations to perform. Similary, **Attachments** for the CAI must match what the CAI is capable of carrying or the object can appear embedded in the model's body incorrectly. The left hand, for example, can typically carry a grenade, but does not carry a sword correctly.

Goals must also match, because not all ModelTemplates and Brains have the same animation sets or understanding of objects. Ninjas can jump, but soldiers do not have a jump animation. A SovietSoldier ModelTemplate with a Ninja brain and a NinjaPatrol goal may try to perform a smoke and teleport operation during combat, but the ModelTemplate does not have an animation for this. The results of mismatching Brains, Models, and Goals are unpredictable, but never satisfactory. Often ModelTemplates with mismatching goals or brains simply stand in place and do nothing.

The following topics explain how to use brains, goals, and AINodes to place NOLF 2 AI in a level and give them "life":

- Using CAI Brains
- Using CAI GoalSets
- Using AINodes and Goals
- Using AINodes and SmartObjects

Using CAI Brains

Each character AI requires a brain in order to function. Specific brains must match the models they are designed for. The default brain works for most human models, but to cause AIs to function according to the roles they are playing in the game, you must specify the brain that matches that role and model. Police, for instance, require the Police brain in order to act like police. Ninjas require the Ninja brain in order to sneak

around and teleport. Indian guards require the Tulwar brain in order to wave their swords in the air and attack as designed.

The following brains currently exist in the NOLF2 game code:

Brain	Definition
Default	The standard brain for all unspecified CAI classes.
Tulwar	The brain for Tulwar CAI classes.
Ninja	The brain for Ninja CAI classes.
EZNinja	A secondary brain for Ninja CAI classes.
Bystander	The brain for non-combatant bystander CAI classes.
SSoldier	The brain for Super-Soldier CAI classes.
PierreBoss	The brain for Pierre CAI classes.
Robot	The brain for Robot CAI classes.
ViewMaster	The brain for ViewMaster CAI classes.
Rat	The brain for Animal CAI classes.
Police	The brain for Police CAI classes.
Contact	The brain for Contact CAI classes.
Volkov	The brain for Volkov CAI classes.

Top

Using CAI GoalSets

The following table lists GoalSets that currently exist in the NOLF2 game code. These definitions exist in the aigoals.txt attributes file. Goals are listed in priority that the AI places on each goal. Some goals build on other goals, so examine the basic GoalSets listed in aigoals.txt for definitions on any goals not listed in this table.

Goal Name	Definition
None	Specifies no particular goalset. AI stands in place and only performs idle animations.
Test	Goal0 = "SpecialDamage" Goal1 = "Patrol"
EngineerTest	GoalSet0 = "Test"
Sniper	RequiredBrain0 = "Default" RequiredBrain1 = "ViewMaster" IncludeGoalSet0 = "DefaultRequired" Goal0 = "Sniper" Goal1 = "Guard" Goal2 = "DrawWeapon" Goal3 = "Alarm"
Patrol	RequiredBrain0 = "Default" RequiredBrain1 = "Tulwar" RequiredBrain2 = "Bystander" RequiredBrain3 = "ViewMaster" RequiredBrain4 = "Police" IncludeGoalSet0 = "DefaultRequired" IncludeGoalSet1 = "DefaultBasic" Goal0 = "Patrol" Goal1 = "Work"

- The Patrol GoalSet includes the DefaultRequired and DefaultBasic GoalSets to define the standard combat behavior.
- The Patrol's "relaxed" behaviors are defined by the Goals - Patrol and Work.
- AI patrols walk along a patrol path defined by LevelDesign through AINodePatrols.
- A patrolling AI owns his path, and no other AI can use it until the owning AI dies or relinquishes ownership.
- A patrol path can be assigned by specifying a node on the path in the AI's Initial Command: goal_patrol node=AINodePatrol9.
- If no node is specified, AI will find the nearest unowned AINodePatrol, and take ownership of that node's path.
- A patrolling AI may not own an AINodeGuard. Patrol and Guard are mutually exclusive.
- If a patrolling AI walks near an unowned work node (AINodeUseObject with SmartObject set to Work), he will stop patrolling to do some "work," then continue patrolling. Patrolling AI will ignore any work nodes that are owned by an AINodeGuard.
- A patrolling AI may animate an investigative walk rather than the routine patrol walk through the command: goal_patrol awareness=investigate. This is useful for levels where AI are intended to be eternally suspicious regardless of stimuli.
- Patrolling AI will turn off lights as they leave an AINodePatrol, and turn on lights at a destination AINodePatrol.

Guard

RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "DefaultBasic"
Goal0 = "Guard"
Goal1 = "Sniper"
Goal2 = "Work"
Goal3 = "Menace"

- The Guard GoalSet includes the DefaultRequired and DeafultBasic GoalSets to define the standard combat behavior.
- The Guard GoalSet includes the Sniper Goal.
- Guard's relaxed behaviors are defined by the Goals Guard, Work, and Menace.
- A guarding AI will stay within the return radius of his guard node. If he is drawn outside of this radius, he will walk to return to it.
- A guarding AI owns his guard node, and no other AI can use it until he dies or otherwise relinquishes it.
- An AINodeGuard can be assigned by specifying a node in the AI's Initial Command: goal_guard node=AINodeGuard22.
- If no node is specified, AI will find the nearest unowned AINodeGuard.
- A guarding AI may not own any AINodePatrols. Patrol and Guard are mutually exclusive.
- The guarding AI owns his guard node, and the guard node may own additional nodes. Owned nodes are AINodeUseObjects used for the Work, Menace, and Sniper Goals.
- An AI that owns a guard node will randomly select work nodes owned by his guard node, rather than always going to the next nearest work node. This AI may not use any unowned work nodes. The same applies to menace nodes.
- Menace nodes are used exactly like work nodes, but are used in levels where the AI is supposed to already be aware of an enemy presence, and should appear permanently agitated, even when the AI system thinks the AI is relaxed. When the AI is not in combat he runs between menace nodes (AINodeUseObjects with SmartObject set to Menace), and play aggressive animations.
- When a guarding AI investigates a disturbance, he stops at the edge of his guard radius and play an alert animation, rather than walking all the way to the disturbance.
- Guard AI do not go to search nodes outside of their guard radius.
- If the guarding AI has previously seen the player, heard an alarm, or been shot, then he ignores his guard radius and investigates anywhere. Once the AI returns to a relaxed state of awareness, he starts paying attention to his guard radius again.

- If the AI notices a disturbance outside of his guard radius, he does not speak. This prevents an AI from repeatedly saying “I hear you!,” “What was that!,” “I know you’re there!,” but never coming to investigate.
- If an AI currently owns a talk node, he ignores his guard node.

Reinforcement

RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "DefaultBasic"
Goal0 = "Patrol"
Goal1 = "Guard"
Goal2 = "ExitLevel"
Goal3 = "Sniper"
Goal4 = "Work"
Goal5 = "Menace"

- The Reinforcement GoalSet is used for AIs who spawned as reinforcements, usually from a command in an alarm.
- The Reinforcement GoalSet includes the DefaultRequired and DeafultBasic GoalSets to define the standard combat behavior.
- The Reinforcement GoalSet includes the Sniper Goal.
- Reinforcement’s relaxed behaviors are defined by the Goals Guard, Work, ExitLevel, and Menace. (See the Guard GoalSet above for more info on Menace).
- A reinforcement AI initially enters the level and respond to whatever is currently happening. Once the situation returns to normal, and the AI returns to a relaxed state of awareness, the AI looks for unclaimed guard and patrol nodes. If one is found, then the AI claims the node and activate the Guard or Patrol goal. The AI then behaves like an ordinary AI that is guarding or patrolling.
- Unclaimed that exist because the AI that previously owned has died are replenished by reinforcements. This maintains the same number of AIs in the level, and keeps the level at approximately the same difficulty.
- Alternatively, the level may intentionally contain a surplus of nodes, so that the level becomes harder once reinforcements are brought in.
- If no unclaimed nodes are found, then the AI looks for an AINodeExitLevel. If an ExitLevel node is found, then the AI walks to the node and gets removed from the level. AINodeExitLevel nodes are normally placed in areas the player cannot get to, preventing players from seeing AIs getting instantly removed from the game.
- AIs dol not look for an ExitLevel node until they have been in the level for at least 5 seconds. This restriction prevents an AI from spawning near an ExitLevel node, and immediately removing itself before noticing an alarm sounding, or other stimuli.

SuicidePatrol

RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "SuicideBasic"
Goal0 = "Patrol"
Goal1 = "Work"

SuicideGuard

RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "SuicideBasic"
Goal0 = "Guard"
Goal1 = "Work"

SleepGuard

RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"

RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "DefaultBasic"
Goal0 = "Sleep"
Goal1 = "Work"
Goal2 = "Guard"

NinjaPatrol
RequiredBrain0 = "Ninja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "NinjaBasic"
Goal0 = "Patrol"
Goal1 = "Work"

NinjaGuard
RequiredBrain0 = "Ninja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "NinjaBasic"
Goal0 = "Guard"
Goal1 = "Work"

NinjaReinforcement
RequiredBrain0 = "Ninja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "NinjaBasic"
Goal0 = "Patrol"
Goal1 = "Guard"
Goal2 = "ExitLevel"
Goal3 = "Work"

EZNinjaPatrol
RequiredBrain0 = "EZNinja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "EZNinjaBasic"
Goal0 = "Patrol"
Goal1 = "Work"

EZNinjaGuard
RequiredBrain0 = "EZNinja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "EZNinjaBasic"
Goal0 = "Guard"
Goal1 = "Work"

EZNinajReinforcement
RequiredBrain0 = "EZNinja"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "EZNinjaBasic"
Goal0 = "Patrol"
Goal1 = "Guard"
Goal2 = "ExitLevel"
Goal3 = "Work"

Whimp
RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"

Talk
RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
Goal0 = "Talk"

Tail
RequiredBrain0 = "Default"
RequiredBrain1 = "Tulwar"
RequiredBrain2 = "Bystander"
RequiredBrain3 = "ViewMaster"
RequiredBrain4 = "Police"
IncludeGoalSet0 = "DefaultRequired"
Goal0 = "Tail"

ScientistPatrol
RequiredBrain0 = "Default"
RequiredBrain1 = "Bystander"
RequiredBrain2 = "ViewMaster"
IncludeGoalSet0 = "DefaultRequired"
Goal0 = "GetBackup"
Goal1 = "Patrol"
Goal2 = "Work"
Goal3 = "Alarm"

ScientistGuard
RequiredBrain0 = "Default"
RequiredBrain1 = "Bystander"
RequiredBrain2 = "ViewMaster"
IncludeGoalSet0 = "DefaultRequired"
Goal0 = "GetBackup"
Goal1 = "Guard"
Goal2 = "Work"
Goal3 = "Alarm"

RobotPatrol
RequiredBrain0 = "Robot"
Goal0 = "Patrol"
Goal1 = "DrawWeapon"
Goal2 = "HolsterWeapon"
Goal3 = "Chase"
Goal4 = "AttackMelee"
Goal5 = "AttackRanged"
Goal6 = "AttackFromVantage"
Goal7 = "AttackFromView"
Goal8 = "RespondToAlarm"
Goal9 = "RespondToBackup"
Goal10 = "Investigate"
Goal11 = "SpecialDamage"

SSPatrol
RequiredBrain0 = "SSoldier"
IncludeGoalSet0 = "SSRequired"
IncludeGoalSet1 = "SSBasic"
Goal0 = "Patrol"

SSGuard
RequiredBrain0 = "SSoldier"
IncludeGoalSet0 = "SSRequired"
IncludeGoalSet1 = "SSBasic"
Goal0 = "Guard"

PierreBoss
RequiredBrain0 = "PierreBoss"
Goal0 = "DramaDeath"
Goal1 = "Ride"
Goal2 = "AttackFromRandomVantage"

MimeArena
RequiredBrain0 = "Default"
RequiredBrain1 = "Bystander"
RequiredBrain2 = "ViewMaster"
IncludeGoalSet0 = "DefaultRequired"
Goal0 = "Investigate"
Goal1 = "AttackMelee"
Goal2 = "AttackRangedDynamic"
Goal3 = "RespondToAlarm"
Goal4 = "AttackFromView"
Goal5 = "PsychoChase"

Mime
RequiredBrain0 = "Default"
RequiredBrain1 = "Bystander"
RequiredBrain2 = "ViewMaster"
IncludeGoalSet0 = "DefaultRequired"
IncludeGoalSet1 = "DefaultBasic"
Goal0 = "Guard"
Goal1 = "Sniper"
Goal2 = "Menace"
Goal3 = "Work"

RatGuard
RequiredBrain0 = "Rat"
Goal0 = "Guard"
Goal1 = "Work"
Goal2 = "Flee"

	Goal3 = "SpecialDamage"
RatPatrol	RequiredBrain0 = "Rat" Goal0 = "Patrol" Goal1 = "Flee" Goal2 = "SpecialDamage"
NonViolentPatrol	RequiredBrain0 = "Tulwar" IncludeGoalSet0 = "DefaultRequired" IncludeGoalSet1 = "NonViolentBasic" Goal0 = "Patrol" Goal1 = "Work" Goal2 = "ProximityCommand"
NonViolentGuard	RequiredBrain0 = "Tulwar" IncludeGoalSet0 = "DefaultRequired" IncludeGoalSet1 = "NonViolentBasic" Goal0 = "Guard" Goal1 = "Work" Goal2 = "ProximityCommand"
NonViolentReinforcement	RequiredBrain0 = "Tulwar" IncludeGoalSet0 = "DefaultRequired" IncludeGoalSet1 = "NonViolentBasic" Goal0 = "Patrol" Goal1 = "Guard" Goal2 = "ExitLevel" Goal3 = "Work" Goal4 = "ProximityCommand"
Volkov	RequiredBrain0 = "Volkov" Goal0 = "Patrol" Goal1 = "DrawWeapon" Goal2 = "HolsterWeapon" Goal3 = "Chase" Goal4 = "AttackMelee" Goal5 = "AttackRanged" Goal6 = "AttackFromVantage" Goal7 = "AttackFromView" Goal8 = "RespondToAlarm" Goal9 = "RespondToBackup" Goal10 = "Investigate" Goal11 = "DramaDeath"
PilotGuard	RequiredBrain0 = "Default" RequiredBrain1 = "Contact" Goal0 = "SpecialDamage" Goal1 = "Guard" Goal2 = "Work"
PilotPatrol	RequiredBrain0 = "Default" RequiredBrain1 = "Contact" Goal0 = "SpecialDamage" Goal1 = "Patrol" Goal2 = "Work"
AllyMelee	RequiredBrain0 = "Contact" IncludeGoalSet0 = "DefaultRequired" Goal0 = "DrawWeapon" Goal1 = "HolsterWeapon" Goal2 = "Investigate" Goal3 = "Chase" Goal4 = "AttackMelee" Goal5 = "Guard"
Iskano	RequiredBrain0 = "Ninja" IncludeGoalSet0 = "NinjaBasic" Goal0 = "Guard" Goal1 = "Work"
Kamal	RequiredBrain0 = "Default" Goal0 = "Guard"

```
Goal1 = "Work"
Goal2 = "Flee"
Goal3 = "SpecialDamage"
```

Top

Using AINodes and Goals

AINode objects tell AIs how to use the environment they exist in. Without an AINode, AIs simply stand in place and perform idle animations (depending on their brains and default goalsets). AINodes provide visibility and perception of objects and components in the game that AIs can use and manipulate. An AI with a patrol goal, for example, requires at least two AINodePatrol objects in order to know which points to patrol between.

As with all AI objects, placement is important. Some AINodes are capable of sizing, and specify an area. Some AI Nodes must be linked to other AI nodes. Most AI nodes need to exist within an AI volume for the AI to locate and use the node. Pay paticular attention to the placement of AI nodes within the AI volumes of your level. Do not embed your AI nodes inside world geometry or the AI is likely to ignore the geometry and either pass through it or fail to use the node.

The following AINode objects exist in the NOLF2 game code:

Node Type	Description
AINode	Specifies a generic AINode object primarily used to script an AI to go to different points in a level, or to create general areas where AIs can path and operate.
AINodeAssassinate	Specifies a location used by the AI to attack. This AINode was used in NOLF1 and is not currently supported in NOLF2.
AINodeBackup	Specifies a location where the AI runs to backup when they have a backup goal.
AINodeCover	Specifies a location where the AI can find cover, and gives them a firing direction in a directed cone. These nodes get used by AIs in the ATTACKFROMCOVER and COVER states. Additional AINodeCover information: • Firing Direction: The cone displayed in DEdit determines the area (field of fire) that the AI fires from. Rotate the node to face the cone in the direction the AI will fire. If IgnoreDir is TRUE then the AI does not use the firing area. • 1WayRoll/1WayStep and 2WayRoll/2WayStep all cause 1WayCorner and 2WayCorner behaviour. The properties are named incorrectly for backwards compatibility with NOLF1 levels. For NOLF2, the properties can safely be changed to 1WayCorner and 2WayCorner. • The AI must enter the radius of the cover node for the cover not to be valid. • If the AI's target gets within the threat radius of the cover node, then the AI executes the ThreatRadiusReaction. • When the AI arrives at a cover node, it sends a message containing the word TRIGGER to the object specified in the Object property. This allows you to flip over tables or other objects in the way by activating a keyframed animation.
AINodeDeath	Used for special AI death animations. AIs with a special AI death goal move to this location and play their death animations when they get killed. Do not use this node unless a special death animation exists for the AI you are using.
AINodeDisturbance	Specifies a disturbance that can be triggered on or off. AIs move to this node to investigate the distrubance.
AINodeExitLevel	Specifies a location where an AI with an exit goal can exit the level.
AINodeGoto	Specifies a location that an AI can move to. You can trigger any AI to move to an AINodeGoto object by sending them a moveto message.
	Similar to a patrol node, but specifies a location that an AI with a Guard goal can guard. AIs with the guard goal will not attack until the character enters the guard node region. You can also specify other

	AINodeUseObject nodes that the guard can use when using the AINodeGuard node. These are typically smart objects such as smoking sections, coffee pots, urinals, etc.
AINodeGuard	**Possession:** If more than one AINodeGuard exists in an area, the first guard to get a node takes possession of all the GuardedNodes specified by that node AINodeGuard node. Make sure each guard AINodeGuard node specifies unique GuardedNodes or only the first guard will be able to use them. This is known as "possession" and keeps guards from using objects that belong to other guards.
AINodeObstruct	Specifies an obstruction that an AI can use for cover. This is similar to the cover node and allows you to specify a field of fire for the AI.
AINodePanic	Specifies a location for AIs with a panic goal to move to when they panic.
AINodePatrol	Specifies a patrol location and path for an AI with a patrol goal. Requires two or more patrol nodes linked together. Make sure patrol lines pass through AI volumes. If you specify a patrol through world geometry, the AI may pass through the world geometry.
AINodeSearch	Specifies an location where an AI with a search goal can move to and search. When AIs with a search goal detect a disturbance, they move to all of the search nodes in their region and check each one. You can specify a command to reset their goals once they have visited all of the search nodes.
AINodeSensing	Specifies an location where you can send commands to other objects based on an AIs sense perception of the player.
AINodeTail	Specifies an location for AIs with a tail goal to acquire an object to tail.
AINodeTalk	Specifies an location for AIs with a talk goal to go to and wait to talk.
AINodeUseObject	Specifies a smart object for the AI to use (such as a chair, lightswitch or urinal). You must place the object in the appropriate location near the world geometry or prefab that the CAI is intended to use. For more information about the AINodeUseObject node, see Using AIUseObjectNodes and SmartObjects in the next topic.
AINodeVantage	Specifies an area where the AI can see the character and acquire a target. These nodes get used by AIs in the ATTACKFROMVANTAGE state. See AINodeCover for a description of the properties. This node is similar to AINodeCover except it does not give a hit point bonus.
AINodeVantageRoof	Specifies an area where the AI can look down on the character. See the description of AINodeVantage for additional information.
AINodeView	Specifies an area where the AI can see the character. This node gets used in contjuction with the AIVolumePlayerInfo node. See Using Regions and Volumes for a description of the AIVolumePlayerInfo node.

GoalSet Operation

The following information describes how some of the more specific goals used by the NOLF2 AI operate.

Chase—Once the Chase Goal is activated (by seeing the enemy) the AI will never lose interest in chasing, with the following exceptions:

- An AI may lose interest in chasing if another goal activates that puts the AI in a state of awareness other than alert (so, relaxed or suspicious). Normally any Goals that activate in favor of Chase are other alert Goals, such as AttackRanged. The exceptions are Goals like SpecialDamage, which incapacitate the AI and then send him searching for his attacker. When an AI wakes up from SpecialDamage, he is investigative and loses interest in chasing.
- An AI may lose interest in chasing if he loses his target in a Junction volume, and gives up.
- Chase Goal Parameters:
  - **SeekPlayer=1:** This parameter instructs the AI to immediately start chasing a player, even if no stimuli have been sensed. This allows the AI to cheat in combat levels where the AI is intended to already know where the player is. The AI ignores Junction volumes while seeking the player. Once the player has been seen, the AI falls back to normal chasing behavior where he can get lost in Junction volumes.
  - **NeverGiveUp=1:** This parameter instructs the AI to cheat indefinitely. The AI will always know where the player is, and will never lose interest in chasing. This parameter was used for the nolf2 ninja trailer park level, where combat should never stop, but ninjas may get

temporarily incapacitated when hit with SpecialDamage (e.g. poison).

AttackFromCover—Cover nodes have an optional parameter Object. This Object may be an object like a table that can be flipped on its side to use for cover. If an AINodeUseObject also points to this object, and the UseObject node has a SmartObject set to something for cover, the SmartObject can specify what animation the AI should use when activating the Object. For example, SmartObject FlipTable tells an AI how to animate when flipping a table to use as cover. After flipping the table, the AI runs to the Cover node, and runs the normal AttackFromCover behavior.

AttackFromVantage—AINodeVantage may specify an optional Object property. This object gets triggered On before the AI starts attacking from the Vantage node, and triggered Off after the AI attacks. This allows you to perform various operations when the AI uses the AINodeVantage node, for example you can open and shut window shutters. This Object was used in NOLF2 to allow Pierre to open and shut steam hatches in the underwater base boss-fight.

- You may find the following goals, derived from **AttackFromVantage**, useful for providing AIs with additional operational capacity.
  - **AttackFromRandomVantage** works exactly like AttackFromVantage, but does not require any stimuli to activate the Goal, and chooses valid Vantage nodes at random instead of choosing the nearest valid node. This Goal is used in NOLF2 for the Pierre boos-fight in the underwater base, where Pierre pops out of random steam hatches to throw knives at the player.
  - **AttackFromRoofVantage** works exactly like AttackFromVantage, but uses AINodeVantageRoof attractor nodes instead of normal AINodeVantages. An AI may only activate AttackFromRoofVantage if no other AI already has the Goal active, and only if at least 3 AI are already attacking the same target. This Goal is used in NOLF2 to control when ninja AI attack from rooftops.

AttackFromView—An AI will attack from an AINodeView if the AI has sensed stimuli of the enemy's presence, and cannot find a path to the enemy, and the enemy is in an AIVolumePlayerInfo that points to a View node that the AI can find a path to.

- **SeekPlayer=1:** This parameter instructs an AI to attack from a View node without first sensing any stimuli.

Work—AINodeUseObjects set to SmartObjects for work are used with the Work Goal. These nodes control most of relaxed behaviors of the NOLF2 AI (such as smoking, working at desks, dancing, leaning, etc).

- There are various Goals derived from Work that are nearly identical, but are attracted to different types of Smart Objects:
  - **Sleep:** AI sleeps in a bed or chair, or while standing. Visual senses are disabled while sleeping.
  - **PlacePoster and EnjoyPoster:** Used in the India levels for police placing posters and civilians looking at posters.
  - **Destroy:** Used for SuperSoldiers to destroy things by smashing or shooting.
  - **Ride:** Used for the underwater base boss-fight to levitate Pierre on steam bursts.
- If a Work node has a ReactivationTime of zero, an AI may not use this node again until first using another Work node. Any ReactivationTime greater than zero will allow an AI to re-use the same node once the node is active again.
- An AI can be instructed to immediately stop working at a node by sending a Disable command to the node.
- If an AI is interrupted in the middle of working, he may optionally play an interrupt animation. For example, an AI that is smoking may abruptly throw down his cigarette instead of calmly putting it out. When the Work Goal deactivates, it will play an interrupt animation if it exists in animationsCharacterName.txt, otherwise the AI will play its normal out transition if it exists.
- If an AI is hit with SpecialDamage while working, he may optionally play a specific SpecialDamage animation rather than transition out of Work normally. For example, and AI may slump over and pass out at his desk, instead of standing up, pushing in his chair and then passing out. This animation actually plays in the SpecialDamage Goal.

- AI may play a specific death animation corresponding to his work. For example, slumping over a desk, or falling out of a chair.

SpecialDamage—The SpecialDamage Goal handles damage types that are specific to nolf2, but may act as a template for similar functionality in other titles. It handles both instant and progressive damage, and includes the following damage types:

- Sleeping
- Stun
- Bear_Trap
- Glue
- Laughing
- Slippery
- Bleeding
- Burn
- Choke
- ASSS (Anti SuperSoldier Serum)
- Poison
- Electrocute

Incapacitating animations and transition animations are defined through SmartObjects. Note that this goal uses SmartObjects to drive animation without the presence of an AINodeUseObject.

Instead of playing the normal in-transition animation, the Goal may play an animation specific to the AI's current animation, as described previously in the Work Goal. This allows an AI to slump over on his desk when knocked-out. This also allows an AI to get knocked out while sleeping or prone without getting up first.

AI may play a specific death animation corresponding to the damage. For example, dying instantly without moving while knocked out.

Top

Using AIUseObjectNodes and SmartObjects

Each ModelTemplate in the game has a set of animations specific to SmartObjects. SmartObjects are a selected property of **AINodeUseObject** nodes that allow an AI to use an object in the game. These objects allow you to tell an AI of a area where they can perform a specific action, such as smoking, browsing, and admiring the scenery. They also allow you to tell an AI about the location of an object they can use, such as a light switch, urinal, file cabinet, or alarm switch. There are many SmartObjects currently listed in the **AINodeUseObject** node, and not all are valid or functioning.

The following list briefly describes each of the SmartObjects you can use in the game, and in some cases gives instructions for the placement of the object. To research the precise placement of objects, you are advised to look in the NOLF 2 levels developed by Monolith. Needless to say, many SmartObjects require the CAI to have an animation for the action to perform, so not all ModelTemplates you select can use all SmartObjects. Additionally, AIs may also require an Attachment object to perform the action with, or an object prop may need to exist in the game for the AI to use. Many SmartObjects were designed to trigger or control specific animation sequences, so to understand the specifics of how they are used, you must look at how they are implemented in the NOLF 2 levels.

LightSwitch

Place the AIUseObjectNode - LightSwitch node near a lightswitch object to allow an AI to activate or deactivate a lightswitch. To get the lightswitch to work you must also toggle the volumes near the lightswitch from Lit TRUE to Lit FALSE when the lights are turned on or

off. You may also want to place and toggle a PlayerInformation node in this area so that he player can hide in areas where the lights are off. This allows you to create dark areas the player can hide in until the AI turns the lights on.

Desk

Place the AIUseObjectNode - Desk node near a desk to enable AIs to perform a "working at desk" animation.

DeskDrawer

Place the AIUseObjectNode - DeskDrawer node near a desk drawer to enable AIs to open and browse through a desk drawer.

Alarm

Place the AIUseObjectNode - Alarm node near an alarm box prop to enable AIs to sound an alarm.

SmokingSection

Place the AIUseObjectNode - SmokingSection in an area you want AIs to perform their smoking animations.

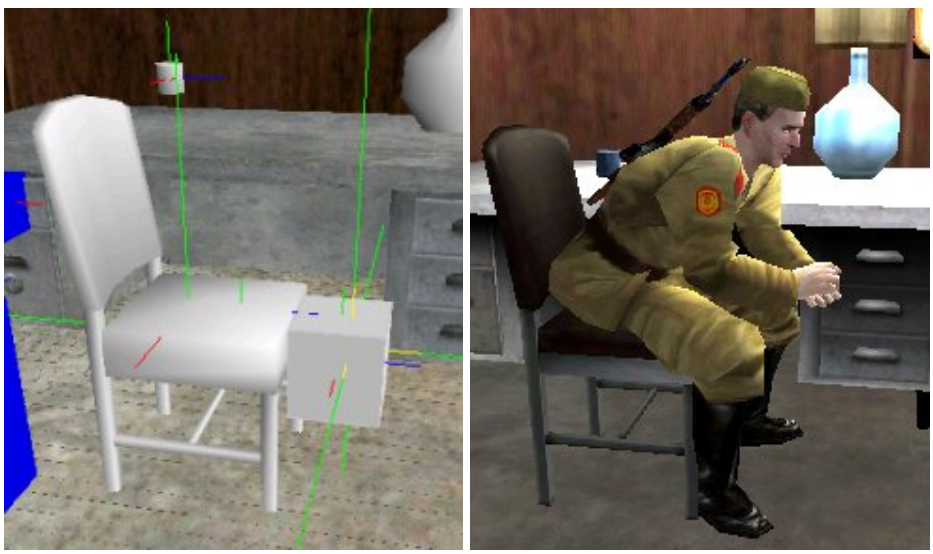
FileCabinet

Place the AIUseObjectNode - FileCabinet node in front of a file cabinet prop with a drawer that an AI can open and browse through.

Chair

Place the AIUseObjectNode - Chair node in front of a chair you want the AI to sit in.

The placement of this node requires that the blue line points away from the chair, and the green light points up. The top edge of the node is level with the top of the chair. As with all nodes, you may need to adjust the distance from the object several times before you get the desired effect. Incorrectly placed nodes can result in the ModelTemplate overlapping with the prop or world geometry. The following image illustrates the use of the Chair node:



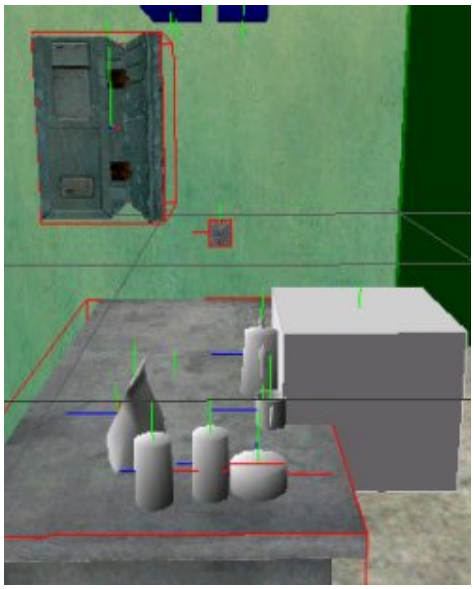
Urinal

Place the AIUseObjectNode - Urinal node in front of a urinal you want the AI to use.

CoffeePot

Place the AIUseObjectNode - CoffePot node near a table where you want the AI to play a drinking coffee animation. The AI does not pour the coffee, but picks up a cup from the table and drinks from it.

The placement of this node requires that the blue line points to the table where the AI will drink. The following image illustrates the use of the CoffeePot node:




Hammer

Place the AIUseObjectNode - Hammer node pointing to a location where you want an AI to play a hammering animation.

HammerLow

Place the AIUseObjectNode - HammerLow node pointing to a location where you want an AI to play a bend and hammer animation.

Wrench

Place the AIUseObjectNode - Wrench node pointing to a location where you want an AI to play a using wrench animation.

WrenchLow

Place the AIUseObjectNode - WrenchLow node pointing to a location where you want an AI to play a bend and use wrench animation.

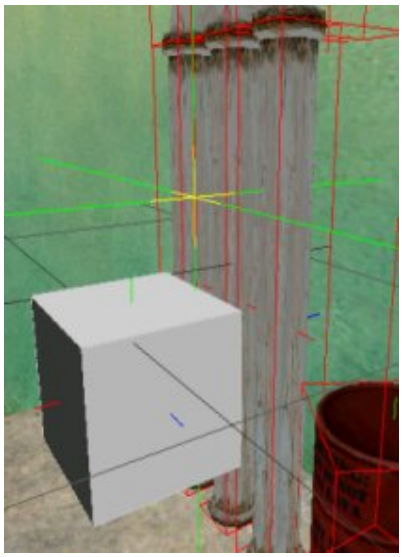
LeanBack

Place the AIUseObjectNode - LeanBack node near a wall the AI can lean against, or anywhere you want the AI to play a lean back animation.

LeanBar

Place the AIUseObjectNode - LeanBar node where you want the AI to play a leaning against a bar animation.

The placement of this node requires that the blue line points in the direction the AI will face, and the node is located at the appropriate distance from the object the AI will lean against. The following image illustrates the use of the LeanBar node:




LeanLeft

Place the AIUseObjectNode - LeanLeft node where you want an AI to play a leaning to the left animation.

LeanRight

Place the AIUseObjectNode - LeanRight node where you want an AI to play a leaning to the right animation.

Lazyboy

Place the AIUseObjectNode - Lazyboy node where you want an AI to play an a sit back and move feet up animation.

PosterSpot

Place the AIUseObjectNode - PosterSpot node where you want an AI to play an identify poster spot animation.

Poster

Place the AIUseObjectNode - Poster node where you want an AI to play a placing poster animation.

StandControlPanel

Place the AIUseObjectNode - StandControlPanel node where you want an AI to play a stand and use control panel animation.

Research

Place the AIUseObjectNode - Research node where you want an AI to play a research animation.

ResearchMicroscope

Place the AIUseObjectNode - ResearchMicroscope node where you want an AI to play a look through microscope animation.

Fire

Place the AIUseObjectNode - Fire node where you want an AI to play an on fire animation.

NinjaIdle

Place the AIUseObjectNode - NinjaIdel node where you want an ninja to play its idle animations.

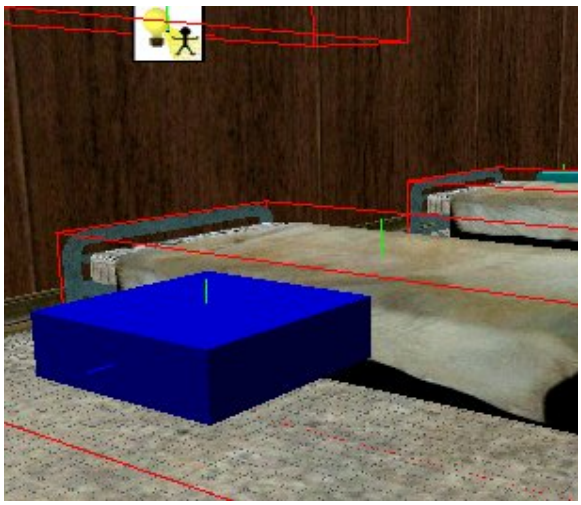
FileCabinetD

Place the AIUseObjectNode - FileCabinetD node where you want an AI to play its search through the files in a file cabinet animation. This node is primarily used for file cabinets the AI should check when a disturbance occurs. The file cabinet pointed to by this node only gets checked when the AI detects a disturbance.

BedTime

Place the AIUseObjectNode - BedTime node next to a bunk or a bed to get an AI to lay down and sleep in the bed. AIs using this node remain asleep only for a random amount of time, and they get up and go about their business after the sleep time passes. To get them to remain asleep until they are disturbed use BedTimeInfinite.

The placement of this node requires that the red line points to the head of the bed, the blue line points away from the bed, and the green light points up. The following image illustrates the use of the BedTime node and also applies to the BedTimeInfinite node:



BedTimeInfinite

BedTimeInfinite is identicale to BedTime except that rather than waking up and going about their business, AIs using this node remain asleep until they are disturbed by an alarm, the presence of the player, or any other disturbance.

SmashLeft

Place the AIUseObjectNode - SmashLeft node where you want a SuperSoldier AI to smash something to its left.

SmashRight

Place the AIUseObjectNode - SmashRight node where you want a SuperSoldier AI to smash something to its right.

AttackProp

Place the AIUseObjectNode - AttackProp node where you want an AI to attack a prop once. Works with all AI.

AttackPropRepeatedly

Place the AIUseObjectNode - AttackPropRepeatedly node where you want an AI to attack a prop until it is destroyed or until the AI gets disturbed by other stimuli. Works with all AI.

Steam

Place the AIUseObjectNode - Steam node where you want Pierre to play his "ride steam" animation.

LazyboyInfinite

Place the AIUseObjectNode - LazyboyInfinite node where you want an AI to play a sitting in a lazy boy chair animation. This is identical to the LazyBoy node except the AI will remains in the chair until disturbed.

ChairInfinite

Place the AIUseObjectNode - ChairInfinite node where you want an AI to play a sitting in a chiar animation. This is identical to the Chair node except the AI remains in the chair until

disturbed.

SniperStand

Place the AIUseObjectNode - SniperStand node where you want an AI to take a standing sniper position.

SniperCrouch

Place the AIUseObjectNode - SniperCrouch node where you want an AI to take a crouched sniper position.

CoverFlipTable

Use the AIUseObjectNode - CoverFlipTable node in conjunction with a cover node. The cover node sends a trigger to a WorldModel to flip a table when the AIReaches the node. See the NOLF2 levels for examples of implementation.

CoverFlipDesk

The AIUseObjectNode - CoverFlipDesk node works in a similar manner to the CoverFlipTable node, except a desk prop gets flipped instead of a table. See the NOLF2 levels for examples of implementation.

Entertain

Place the AIUseObjectNode - Entertain node is designed to work in conjunction with the Dance node and is intended to allow AIs to play their entertainment animation when a radio is turned on.

Dance

Place the AIUseObjectNode - Dance node where you want an AI to dance.

Barrel

Place the AIUseObjectNode - Barrel node is similar to the CoverFlipTable node. It works with a Cover node to flip a barrel the AI can use for cover.

VendingMachine

Place the AIUseObjectNode - VendingMachine node where you want an AI to play its use and kick vending machine animation. See the vending machine prefab for examples of implementation.

HitWall

Place the AIUseObjectNode - HitWall node where you want an AI to pound on the wall in a panic. This was used in NOLF2 where the AI were trapped and pounding on the door as water rushed in to drown them.

OutdoorUrinal

Place the AIUseObjectNode - OutdoorUrinal node where you want an AI to urinate without a urinal.

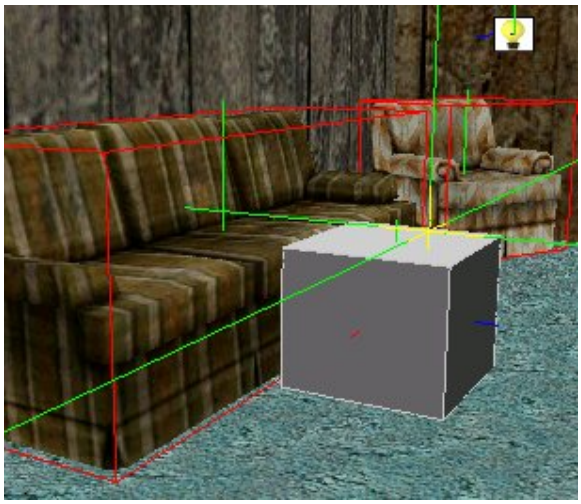
WashHands

Place the AIUseObjectNode - WashHands object where you want an AI to play its washing hands animation. This animation should be used in conjunction with a sink prop.

SmokingChair

Place the AIUseObjectNode - SmokingChair node near a chair you want the AI to sit and smoke in.

The placement of this node requires that the blue line points away from the chair, and the green light points up. The following image illustrates the use of the SmokingChair node:



SitControlPanel

Place the AIUseObjectNode - SitControlPanel node where you want an AI to sit and push buttons.

Typewriter

Place the AIUseObjectNode - Typewriter node where you want an AI to play its typing animation. This works best in conjunction with a desk and typewriter prop.

SpeakerPhone

Place the AIUseObjectNode - SpeakerPhone node where you want an AI to press the button on a speakerphone.

AlarmBox

Place the AIUseObjectNode - AlarmBox node in front of alarm boxes that require an open box animation prior to pressing the button to activate an alarm.

MenaceInfinite

Place the AIUseObjectNode - MenaceInfinite node where you want an AI to play its menace animation until it is disturbed.

MenaceNonInfinite

Place the AIUseObjectNode - MenaceNonInfinite node where you want an AI to play its menace animation for a specified period of time.

NinjaMenace

Place the AIUseObjectNode - NinjaMenace node where you want a Ninja to play its menace animation.

AdmireScenery

Place the AIUseObjectNode - AdmireScenery node where you want an AI to play its looking around animation.

StandSleep

Place the AIUseObjectNode - StandSleep node where you want an AI with a sleep goal to stand and doze off for a specified period of time.

StandSleepInfinite

Place the AIUseObjectNode - StandSleepInfinite node where you want an AI with a sleep goal to stand and doze off until they are disturbed.

RatIdle

Place the AIUseObjectNode - RatIdle animation where you want a rat to play its idle look around animation.

Browse

Place the AIUseObjectNode - Browse node where you want an AI to play its browsing through something animation.

ArmstrongStruggle

Place the AIUseObjectNode - ArmstrongStruggle node where you want Armstrong to play his struggle to lift something massive animation.

ResearcherCabinet

Place the AIUseObjectNode - ResearcherCabinet node where you want a scientist to play his

research cabinet animation.

DrinkingFountain

Place the AIUseObjectNode - DrinkingFountain node near a drinking fountain prop you want an AI to drink from.

PushButton

Place the AIUseObjectNode - PushButton node where you want an AI to play its push button animation.

PierreStop1

The AIUseObjectNode - PierreStop1 node is used for the Pierre on a unicycle animation.

PierreStop2

The AIUseObjectNode - PierreStop2 node is used for the Pierre on a unicycle animation.

CoverBarrel

Place the AIUseObjectNode - CoverBarrel node where you want an AI to flip a barrel and use it for cover.

BrokenUrinal

Place the AIUseObjectNode - BrokenUrinal node where you want an AI to play its broken urinal animation.

LeanRail

Place the AIUseObjectNode - LeanRail node where you want an AI to play its leaning against a rail animation.

MimeEntertain

Place the AIUseObjectNode - MimeEntertain node where you want a Mime to play its entertain/torture animation.

Top

Using AI Senses

Each character AI in the NOLF2 game code has senses that allow them to percieve the environment bounded within the AIVolumes belonging to the AIRegion they are spawned it. This section defines each of an AI's senses, and explains how to change them using the AttributeOverrides property.

This section contains the following AISenses topics:

About AI Senses

Each AI in the NOLF2 game code refers to an attribute template in the AIButes.txt file. These attributes templates specify whether each sense is on off. Any sense not listed in the template is off by default. For each sense that is on, a distance is specified that sets the range for the AI to be triggered by the sense.

Sense Stimuli

Events that stimulate AI senses are listed at the bottom of the AIButes.txt file in the Attributes folder. Each stimulus takes a number of required or optional parameters. The following table describes each of the stimuli that an AI can receive.

Stimulus	Description
Name	Name of the stimulus. The name must match an enumeration in the C++ game code.
Sense	The AIs sense that the stimulus triggers. Must match one of the sense listed in the Customizing AI Senses table in the following topic.
RequiredAlignment	LIKE or HATE. This stimulus triggers the AIs senses that LIKE or HATE the source of the stimulus. TOLERATE is assumed, so AIs with neutral alignments will be triggered by either LIKE or HATE.
Distance	Radius from the source of the stimulus that triggers the AIs response. AIs sense the stimulus when the AIs sense distance overlaps with the stimulus distance. This is a base distance which you can further modify in code (for example you can modify the distance depending on the material the AI is walking on). If the distance should be handled entirely in code, then set the distance to 1.00.
Duration	Time in seconds that the stimulus exists. Zero means specifies that the stimulus last forever, or until deleted in the code.
AlarmLevel	Priority setting for how alarming this stimulus is. Set this value between zero and four, with four being the most alarming.
ReactionDelay	Optional parameter specifying the time in seconds that the AI delays before reacting to a fully stimulated sense. The default time is zero.
StimulationIncreaseRateAlert	Optional parameter specifying the rate the stimulation gradually increases when the AI is alert. The default value, 100.00, forces immediate full stimulation.
StimulationIncreaseRateUnalert	Optional parameter specifying the stimulation gradually increases when the AI is not alert. The default value, 100.00, forces immediate full stimulation.
StimulationDecreaseRateAlert	Optional parameter specifying the rate the AIs stimulation gradually decreases when they are alert. The default value, 0.00, specifies the stimulation never decreases.
StimulationDecreaseRateUnalert	Optional parameter specifying the rate the AIs stimulation gradually decreases when they are not alert. The default value, 0.00, specifies the stimulation never decreases.
StimulationThreshold	Optional parameter specifying the range describing the amount of stimulation needed for partial and full stimulation. The default value, [1.00, 1.00] specifies that no partial stimulation exists.

FalseStimulationLimit	Optional parameter specifying the number of false stimulations before an AIs sense is forced to full stimulation. False stimulations occur when an AIs sense reaches partial stimulation, and then falls back to zero stimulation.
MaxResponders	Optional parameter specifying the maximum number of AIs that can respond to this stimulus.

Currently Existing NOLF2 Stimuli

Stimulus	Description
EnemyVisible	Trigger Sense: SeeEnemy. AIs constantly emanate this stimulus while they are alive.
EnemyFootprintVisible	Trigger Sense: SeeEnemyFootprint. Footprints of AIs constantly emanate this stimulus as long as they exist.
EnemyWeaponFireSound	Trigger Sense: HearEnemyWeaponFire. AIs create this stimulus when they fire their weapons.
EnemyWeaponImpactSound	Trigger Sense: HearEnemyWeaponFire. This stimulus is created when an AIs weapon fire hits something.
EnemyWeaponImpactVisible	Trigger Sense: SeeEnemyWeaponImpact. This stimulus is created when an AIs weaopn fire hits another AI.
EnemyFootstepSound	Trigger Sense: HearEnemyFootstep. This stimulus is created when an AIs movement animation triggers a Model String.
EnemyDisturbanceSound	Trigger Sense: HearEnemyDisturbance. This stimulus is created when an AI uses the coin.
AllyDeathVisible	Trigger Sense: SeeAllyDeath. AIs constantly emanate this stimulus while they are dead.
AllyDeathSound	Trigger Sense: HearAllyDeath. AIs create this stimulus when they are dying.
AllyPainSound	Trigger Sense: HearAllyPain. AIs create this stimulus when they are in pain.
AllyWeaponFireSound	Trigger Sense: HearAllyWeaponFire. AIs create this stimulus when they fire their weapons.

Customizing AI Senses

You can customize AI senses through commands and through the AttributeOverrides property of the CAIHuman node in DEdit. When you change an AIs senses, DEdit shows a green radius indicating the perception range of the AI. Use this radius indicator to adjust the AIs senses to the appropriate level.

The following table describes each of the senses in the CAIHuman AttributesOverrides property:

Sense	Description
Senses	Setting senses to TRUE sets all senses to true. Setting sense to FALSE sets all senses to FALSE except those senses you override with a TRUE value.
CanSeeEnemy	See a live enemy (TRUE or FALSE)
SeeEnemyDistance	Distance radius value
CanSeeAllyDeath	See ally die (TRUE or FALSE)
SeeAllyDistance	Distance radius value

CanHearAllyDeath	Hear an ally cry out when dying (TRUE or FALSE)
HearAllyDeathDistance	Distance radius value
CanHearAllyPain	Hear ally cry out when damaged (TRUE or FALSE)
HearAllyPainDistance	Distance radius value
CanSeeEnemyFlashlight	See enemy flashlight (TRUE or FALSE)
SeeEnemyFlashlightDistance	Distance radius value
CanSeeEnemyFootprint	See enemy footprint (TRUE or FALSE)
SeeEnemyFootprintDistance	Distance radius value
CanHearEnemyFootstep	Hear enemy footsteps (TRUE or FALSE)
HearEnemyFootstepDistance	Distance radius value
CanHearEnemyDisturbance	Hear enemy move when not moving quietly (TRUE or FALSE)
HearEnemyDisturbanceDistance	Distance radius value
CanHearEnemyWeaponImpact	Hear enemy weapon impact with world geometry (TRUE or FALSE)
HearEnemyWeaponImpactDistance	Distance radius value
CanHearAllyWeaponFire	Hear ally fire a weapon (TRUE or FALSE)
HearAllyWeaponFireDistance	Distance radius value

Testing AI Senses

Once you have changed an AIs senses, they operate differently, and you will want to test the AI to ensure they perform the actions in the game you want them to perform. For instance, you may want to increase an AIs ability to see the enemy at a distance so they can perform well as a sniper. At the same time you don't want the AI to be capable of seeing too far or they will shoot the character before it is logical for them to see the character. In order to test this, you will have to run the game and act as a target to see when the AI fires at you.

For information about consol commands and hotkeys you can use when testing AI, see Debugging AIs.

Top

Scripting AI Goals

Scripting was avoided as much as possible in NOLF2, but there were situations where some amount of scripting was necessary to drive the story or vary the gameplay for specific levels. The following list describes a few methods for accomplishing scripted behaviors without breaking the autonomy of the AI.

- **Enable/Disable Node Commands**—The majority of scripting in NOLF2 involved enabling and

disabling nodes, or assigning new nodes with command, such as: goal_guard node=AINodeGuard45. This gives control over where AI are idling while still allowing some randomness and allows them to react normally to whatever situation they find themselves in.

- **GoalScripts**—GoalScripts supply an AI with a sequence of Goals to complete. When one Goal is satisfied (e.g. the AI has arrived at the destination node of a Goto Goal), the next Goal activates. GoalScripts may be interrupted if the AI notices a disturbance, and the GoalScript will continue once the AI returns to a relaxed state of awareness. GoalScripts are useful for making an AI go fro place-to-place and animate. You can find examples of existing GoalScripts in the Attributes\aigoals.txt file.
- **Search**—Normally AI search after activating other Goals (e.g. after investigating or chasing and losing the target). In cases where AI are spawned with the intention of immediately searching, they may be given the Search Goal, which is a scripted Search. AI with the Search Goal can be given a list of comma-separated regions to search: addgoal search region=airegion01,airegion02,airegion03. AI who search through normal means will only search their current region.
- **Patrol**—Another way to script searching behavior is to spawn AI and assign them patrol routes. The Patrol Goal may be instructed to play investigative walking animations: addgoal patrol awareness=investigate. In some nolf2 levels, we periodically spawn a patrolling AI to ensure the level will never be completely empty (and boring to the player), even when all AI have been killed leaving no one to sound alarms. An AI may be spawned, instructed to patrol, and then removed through a command on a destination patrol node.

Top

Content Guide

Using AI Attributes

AI attributes include what an AI carries, what their default weapons are, how tough they are, and what search items they carry. This section describes each of the attributes you can modify, and provides some general guidlines for performing the modification.

Setting General AttributeOverrides

General AttributeOverrides are located in the AttributeOverrides property of the CAIHuman object. The following table describes each of the attributes and the values you can use to modify them. Values not listed in this table that appear in DEdit are not supported and were not used in the creation of NOLF 2.

All the attributes listed below are optional. If you leave the values blank, then they get filled in by the aibutes.txt attributes file dependent upon the ModelTemplate you have selected. The sample values listed do not indicate an average, but only represent one possible value taken from `AttributeTemplate1` of the aibutes.txt file located in the Attributes folder. These values indicate the standard attribute values for a SovietSoldier AI.

Ideally, to globally change attributes for all AIs of a specific ModelTemplate, you should edit the aibutes.txt file and create a new entry for the character.

Attribute	Description
SoundRadius	Specifies the default sound radius projected by the AI when they speak. Sample value = 1500
HitPoints	Specifies the number of hitpoints the AI starts with. Range is zero to infinity. Initial values depend on difficulty settings. Sample value = 25
Armor	Specifies the starting armor value for the AI. Range is zero to infinity. Initial values depend on difficulty settings. NOLF 2 does not use armor settings, so initial values in NOLF 2 are always zero. Sample value = 0
Accuracy	Specifies the accuracy of the AI when they fire at a target. Range is zero to one (indicating percent of the time the AI hits the target). Initial values are dependent upon difficulty. If the AI is not trying to hit the target, then they intentionally try to miss, however if the target moves in front of the weapon they may still be hit. Sample value = 0.5
FullAccuracyRadius	Specifies radius within which the AI never tries to miss. Range is zero to infinity. Initial values depend on difficulty settings. Sample value = 256
AccuracyMissPerturb	Specifies how far the AI aims to the left or right when intentionally trying to miss. Range is zero to infinity. Sample value = 64
MaxMovementAccuracyPerturb	Specifies the maximum range the AI is off target when firing at a moving target. Range is zero to infinity. Sample value = 10
MovementAccuracyPerturbTime	Specifies the amount of time required for an AIs aim to catch up with a moving target. Range is zero to infinity. Sample value = 3
JumpSpeed	Specifies how fast AI moves when movement for animation is of type Jump (in animationshuman.txt). Range is zero to infinity. Sample value = 280
WalkSpeed	Specifies how fast AI moves when movement for animation is of type Walk (in animationshuman.txt). Range is zero to infinity. Sample value = 60
SwimSpeed	Specifies how fast AI moves when movement for animation is of type Swim (in animationshuman.txt). Range is zero to infinity. Sample value = 0
RunSpeed	Specifies how fast AI moves when movement for animation is of type Run (in animationshuman.txt). Range is zero to infinity. Sample value = 700

Top

Setting Attachments

Each AI in the game is capable of taking a number of attachments. The Attachments property in the CAIHuman object allows you to select an attachment you can attach to a model socket.

Note: Selecting the wrong attachment for a model results in errors such as the model hanging in the air on fire. If this occurs, then it is likely that the attachment you

have selected is not designed for the ModelTemplate.

Top

Setting SearchProperties

The SearchProperties property in each CAI object allows you to control what the player finds when they search the body of a dead AI. The RandomItemSet list gives you a selection of body types, with each body type containing a variety of items selected randomly when the body gets searched. Rats, for instance, should be set to "None" so players do not find usable items when searching rat bodies. Soviet soldiers should be set to "Bodies Soviet."

The randomly selected search items are located in the searchitems.txt file of the Attributes folder. You can edit this file, along with the ClientRes.rc and ClientRes.h files to add new gear items.

You can also create specific items to appear when the player searches an AIs body. These items are typically weapons, gear, intelligence, or key items that you want a player to find when they search a particular AI.

To create a specific search item

1. Place a gear or weapon item anywhere in your level.
2. In the gear or weapon item, set Template to FALSE.
3. In the SearchProperties for any CAI, type the name of the gear or weapon item template in the SpecificItems box.

Note: Even with Respawn set to TRUE, you may need to create specific unique gear items for each AI in order to get them to appear. Item Respawn in the Single Player game may not work properly.

Top

Setting GeneralInventory

The GeneralInventory property of CAI is not used in NOLF2.

Top

Using AI Commands

This section describes commands you can send as messages to AIs to instruct them to perform specific actions. For additional information about sending commands from DEdit, see Using Object Commands in the Working With Objects section.

Command	Description
ACTIVATE	Activates an inactive AI.
ADDGOAL <goal>	Adds a goal to the AI's goalset.
ALIGNMENT <key>	Changes the AI's alignment.
ALWAYSACTIVATE <1 or 0>	Sets the AI to always active, or allows them to become activated by an Activate message.
ARMORMOD <mod>	Changes an AIs armor value.
CANDAMAGE <bool>	Specifies whether or not the AI can take damage.
CANTALK <1 or 0>	Specifies whether or not the AI can talk to the player.
CINERACT <Animation Name>	Used to specify an animation for the AI to play in cinematics. The animation gets played once.
CINERACTLOOP <Animation Name>	Used to specify an animation for the AI to play in cinematics. The animation gets looped.
DAMAGEMASK <key>	Specifies an integer value indicating what cannot damage the AI. Typically the value indicates a damage type. Damage types not listed will damage the AI as normal.
DEBUG <level>	Sets the debug level for an AI.
DESTROY	Destroyes the AI as if it were killed.
DETACH <attach pos>	Detaches an attachment from the AI at a specified attachment position.
FACEDIR <X,Y,Z>	Causes the AI to face in a specific direction.
FACEOBJECT <object>	Causes the AI to face a specific object.
FACEPOS <X,Y,Z>	Causes the AI to face in a specific direction.
FACETARGET	Causes the AI to face its current target.
FOVBIAS <fov>	Alters the AI's field of view.
GADGET <ammo id>	Adds a gadget to the AI's inventory.
GOALSCRIPT <script>	Specifies a goalscript for the AI to follow.
GOALSET <goalset>	Specifies a goalset for the AI to follow.
GRAVITY <1 or 0>	Toggles gravity for the AI.
HEALTHMOD <mod>	Modfies the AI's health.
HIDDEN <bool>	Specifies an AI as hidden or non-hidden. When Hidden an AI is still visible, but becomes intangible.
IGNOREVOLUMES <1-or-0>	Specifies whether or not the AI should ignore AI volumes and path through walls.
MOVE <X,Y,Z>	Specifies an X,Y,Z set of world coordinates the AI must move to.
PLAYSOUND <sound name>	Plays a specified sound at the position of the AI.
PRESERVEACTIVATECMDS <activateon or activateoff>	Activate On / Activate Off. Specifies whether the AI should continue accepting activation commands.
REMOVE	Cleanly removes an AI from the level and from memory.
REMOVEGOAL <goal>	Removes the specified goal from the AI's goalset.
SENSES <1 or 0>	Toggles the AI's senses on or off.
SOLID <bool>	Specifies an AI as solid or intangible.
TARGET <object>	Specifies a target object for the AI to attack.
TELEPORT <teleport point> |	Teleports the AI to the specified TeleportPoint node, or to a specific X,Y,Z

<vector>	coordinate.
TRACKLOOKAT	Specifies to use head tracking to look at a target.
TRACKNONE	Specifies no head tracking.
VISIBLE <bool>	Specifies an AI as visible or invisible.
ChaseSeekPlayer <bool>	Boolean value specifying the AI should chase the player.
NeverGiveUp <bool>	Boolean value specifying the AI should never give up when chasing the player.

Top

Content Guide

Debugging AIs

This section provides some general information and helpful hints on debugging AIs after you have placed them in a level.

About Debugging AI

Debugging AI is a process that often involves a great deal of testing. Placing AINodeUseObject nodes, for example, may require several attempts before you get the AI to use the target object in an appropriate manner.

AIVolume placement is another task that requires testing. Simply placing AIVolumes over an entire room results in AIs walking across furniture and other objects. At the same time, placing AINodeUseObjects that target objects outside of an AIVolume will cause an AI to play their animations in the wrong location, or fail to use the node at all.

AIVolumes near doors should leave an unvolumed area for the doors to open. This prevents AIs from walking through them or attempting to fire through the doors. Another important factor is the placement of the door volume. The WorldModel controlling the door should always be placed in its own volume to specify to the AI where to open the door. Failure to put WoldModels in their own limited AIVolumes results in the AI opening the door from the edge of the AIVolume the WorldModel is contained in. For example, if a RotatingSwitch for a door is in one large AIVolume, the AI can activate the switch from the edge of the volume rather from the location of the switch. For an illustration of door volume placement, see AIVolumes and Doors in the AI Tutorial.

Consol Commands

The following table lists commands that you can enter in the console to assist you in debugging AIs:

Command	Description
serv AISenseInfo 1	Supplies debugging information about an AI's sensory information.
serv AIAccuracyInfo 1	Supplies debugging information about an AI's accuracy.
trigger <name> "debug <level>"	Activates general debugging information where <name> specifies the name of an AI, and <level> specifies the debug level.
stimulus <stimulus type> [distance multiplier]	Sets a stimulus from the world. For a list of Stimulus types, see the Currently Existing NOLF2 Stimuli table. For example, `stimulus enemyFootstepSound` creates an enemy footstep sound for the AI to hear. You can add a distance multiplier to this command, for example: `stimulus enemyFootstepSound 5` specifies an enemy footstep sound at five times the distance.
renderstimulus <1 or 0>	Rrenders spheres in the game that represent stimuli in the world. This allows you to visualize the stimuli that AIs are reacting to. When this command is activated, the ten closest stimuli to the player get rendered as red spheres. The spheres size indicates the distance of the stimulus, although the maximum size is capped at one hundred DEdit units to keep them viewable.
aishowjunctions <1 or 0>	This command must be placed in your s_autoexec.cfg file. It allows you to see how the AI is responding to junctions.

Top

Mode and Setting Keys

The following table lists debug keys you can press during runtime to assist you in debugging AIs:

Key	Description
F11	Toggles through display of AI volumes, AI paths, and AI volumes and paths.
SHIFT + F11	Toggles thorugh display of AINodes.
F7	Gives the player all gear and equipment.
F5	Displays AI names. Point the crosshairs at any particular AI to display its goals and scripts.
F2	Activates clipping mode. In this mode you are undetectable and intangible. You cannot interact with the world, but you can view AI performing their actions without risk of disturbing them.
F1	Activates ghost mode. In this mode you are simply invisible. AI will not fire at you, but they can hear you and become disturbed. You can also interact with props, objects, and WorldModels in this mode.

Top

Content Guide

Creating Cinematic AIs

The Talk Goal allows AI to coordinate meeting up, having a conversation, and going on to do other things. The Talk Goal also handles responding to interruptions, and possibly returning to have the conversation later. There are basically two options for setting up AI to have a conversation: statically positioned, and dynamically positioned.

Statically Positioned Conversation Participants

- AI are placed at AINodeTalk nodes and given initial commands of: addgoal talk node=AINodeTalk04. AI will walk to Talk nodes and wait for a dialogue object to be triggered.
- If both AI are within the radii of their Talk nodes when the corresponding dialogue object is triggered, they will have their conversation.
- When the conversation ends, the dialogue object will run its finished and cleanup commands.
- An AI with a Talk Goal and Talk node will ignore his Guard and Patrol Goals. The Talk Goal overrides other relaxed goals, so the AI may still have a full GoalSet, such as Patrol or Guard.
- If an AI is at a Talk node, and notices a disturbance, he will investigate. Guard node radii will be ignored.
- If the dialogue object is triggered while one or both of the dialogue participants are not relaxed, the dialogue will not play, and the dialogue object's cleanup command will be run.
- If the dialogue object was not triggered while the AI was investigating, when the AI returns to a relaxed state of awareness, he returns to his Talk node and continues waiting for the dialogue to be triggered.
- The Talk Goal parameter disposable=1 tells the AI not to return to his Talk node if he was disturbed while waiting for the conversation. In this case, the AI will simply return to his normal relaxed behaviors and forget about having the dialogue.

Dynamically Positioned Conversation Participants

- AI may be anywhere in the level, running various Goals. They do not need to be near their Talk nodes.
- When a dialogue object is triggered, commands are sent to the dialogue participants: addgoal talk node=AINodeTalk09 retrigger=1. If either AI are outside of their Talk node radii, the dialogue will not start. The parameter retrigger=1 tells the AI to retrigger the dialogue object once the AI arrive at their Talk nodes.
- Once both AI are in position, the conversation starts.
- When the conversation ends, the dialogue object will run its finished and cleanup commands.
- The above steps may be used to get AI to walk towards each other and start talking, or to make an AI stop working and start talking to someone.

General Notes about the Talk Goal

- While the Talk Goal is active, AI face each other and talk while playing their basic idle animation (usually 3StGd).
- AI may be instructed to play gesture animations at key points in the conversation. From the dialogue object, send a command to the AI: goal_talk gesture=NameOfAnim.
- If a dialogue object is triggered, and one or more of the dialogue participants does not exist (probably because he is dead), the dialogue will not play and the dialogue object's cleanup command will run.

Top

Content Guide

—Tutorial—
Using NOLF 2 AI

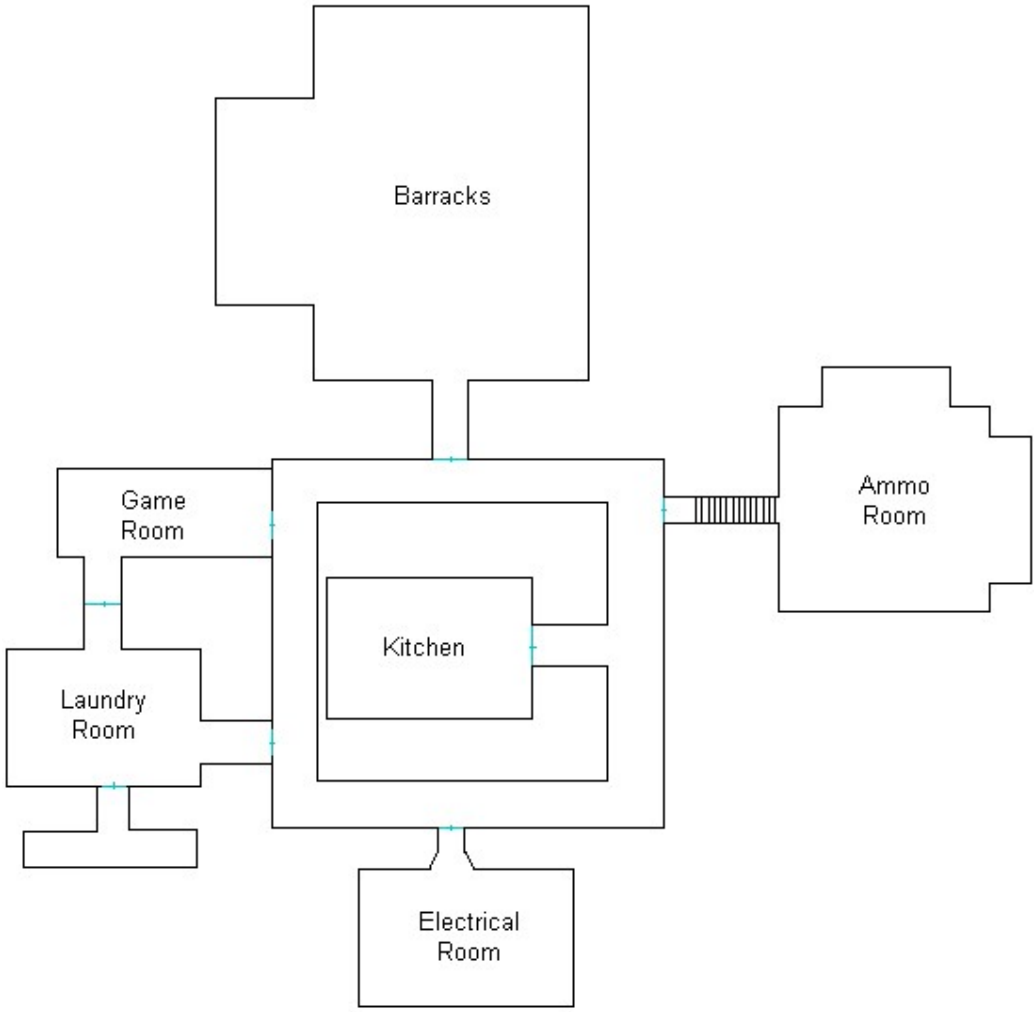
This section walks through the creation of AI interaction using the Tut_AI file. Tut_AI.ltc was created by LithTech as a tutorial showing how you can place and use NOLF 2 AI in a simple sample level. The Tut_AI.ltc file is located in the following directory:

\%install%\LithTech\LT_Jupiter_Src\Development\TO2
\Game\Worlds\RetailSinglePlayer\Tut_AI.ltc

Read the following sections to learn how to place character AI, AIVolumes, and AINodes:

About the Tut_AI Sample Level

The Tut_AI sample level is a simple six room level designed to show how to set up and use the NOLF2 AI. The following image shows the layout of the level.



When playing the Tut_AI, the goal of the game is to blow up the two bombable pallets in the Ammo room. The bombs are located in the utility cabinet in the Electrical room. The Kitchen (the starting area) is a safe location that AI cannot enter. Gameplay is set so that additional guards are spawned after some of the guards are killed (this uses an EventCounter object). In addition, guards are also spawned when the alarms in the Ammo room and the Barracks are sounded, and when the pallets are destroyed.

The creaky floorboard trigger outside the Laundry room also sounds an alarm to attract the Laundry room guard. The Laundry room alarm does not set off an audible alarm, but rather simply attracts the Laundry room guard to investigate the sound of the floorboard.

This tutorial walks you through the steps involved in adding AI to a level. There are many objects you must place in your level to get the NOLF2 AI to operate appropriately. The following list describes some general steps for adding AI object nodes:

General AI implementation steps

1. Place AIRegion nodes in each of the locations you wish to separate.
2. Place AIVolumes in the areas you want AIs to see, pathfind, and travel. Bind all volumes to a specific AIRegion when you place them. Make sure the AIVolumes are close enough together or AIs cannot path between the volumes.
3. Place CAI nodes in AIVolumes where you want an AI to start the level.
4. Place CAI template nodes near where you want to place their spawners.
5. Place Spawner objects where you want AIs from templates to appear when you trigger the Spawner.
6. Place AINodes in locations where you want an AI to perform an action or operate in a specific manner. Make sure each region has at least one AINodeSearch node.
7. Place Alarm props in locations where you want AIs to respond when the alarm is activated.

8. Place triggers or alarm switches in locations where AIs can activate them, and set them to trigger the appropriate Alarm prop object.

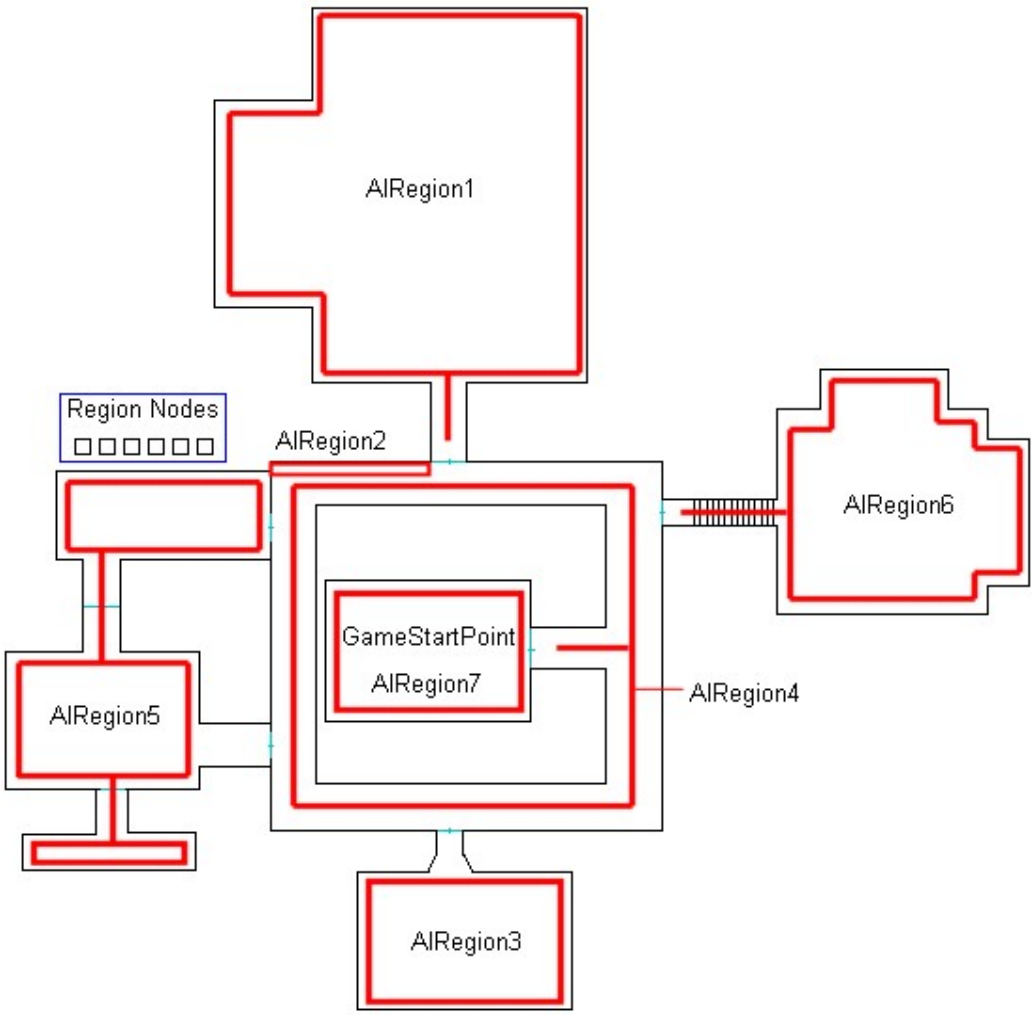
Top

Placing AIRegions

Tut_ai.ltc contains six AI region nodes. The only area without connecting AIVolumes is the kitchen where the character starts the level. This gives the character one safe area to collect gear and recuperate.

Aside from grouping the AIVolumes, the regions also control the response of the AIs to the alarms in the level.

The following images shows the placement of regions in Tut_AI.lta:



The area marked "Region Nodes" in the previous image contains the AIRegion nodes for each of the regions.

In DEdit, click the Nodes tab, expand the GamePlay folder, then expand the Character AI folder. Each set of character AI are grouped by region. This allows you to view the CAI in any AIRegion by selecting the desired folder.

The following list describes the AI content in each region:

AIRegion1 This region is the barracks area containing six soviet soldiers. Each guard in this region is named "BarracksGuard<number>" where <number> indicates the unique ID name for the guard. Three of them are set to SleepGuard, two are set to Guard, and one is set to Patrol. The guard on patrol may be attracted to the nodes in the hallway, and sometimes takes that patrol path in the hallway if the HallGuard in AIRegion4 is not using it.

AIRegion2 This region contains one rat patrol.

AIRegion3 This region is the electrical room. It contains one soviet soldier. The soldier, named "ElectricalGuard," is set to guard.

AIRegion4 This region is the hallway. It contains one soviet soldier. The soldier, named "HallGuard," is set to patrol around the hall. This soldier has use nodes for smoking, and may be attracted to the use nodes in other regions. It is possible that this guard will leave the hallway.

AIRegion5 This region is the laundry room. It contains one soviet soldier. The soldier, named "LaundryGuard," is set to guard. There are three smoking nodes in this region, two on the couch and one on the chair. There is also a vending machine that the guard can kick and curse at.

AIRegion6 This region is the Ammo room. It contains two soviet soldiers using winter models. The soldiers, named "AmmoGuard<number>" are set to guard. There is also a smoking node and a pour coffee node in this room.

AIRegion7 This region is the kitchen and starting area. The AIVolume in this region does not extend to the door, so the AI outside this region cannot enter.

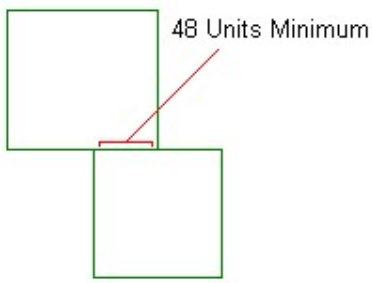
Top

Placing AIVolumes

Each AIRegion has a group of AIVolumes bound to it. AIVolumes control the navigation of an AI, and are required to allow an AI to pathfind from location to location. The AIRegions only control search areas and alarm response. AIVolumes allow the AI to pathfind and move through the areas of neighboring volumes.

After you have placed the AIRegion nodes for your level, have a careful look at your level design and plan the placement of your AIVolumes. Typically, you should use a generic AIVolume node for most areas. Special AIVolumes, such as AmbientLife, JumpOver, JumpUp, Junction, Ladder, Stair, and Teleport all allow for specific actions to occur. The generic AIVolume simply allows the AI to pathfind and move through the volumes. CAI objects placed in an area without an AIVolume cannot operate and simply stand in place, even if they are shot.

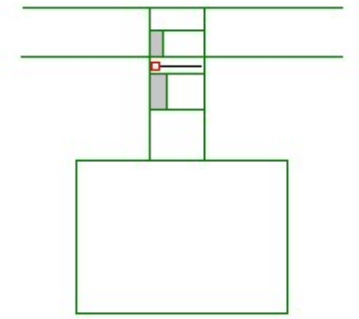
It is important to place AIVolumes directly against each other or AIs cannot across them. Additionally, AIVolumes placed against each other must have a 48 unit abutment (connection) between them or AIs cannot travel across them. The following image illustrates the required abutment distance:



AIVolumes and Doors

Doors create a special problem for AIVolumes. The AI perceives the WorldModel controlling the door

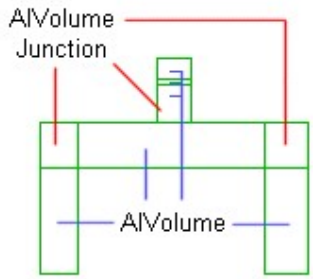
(such as a RotatingWorldModel object) to exist in the volume where the door exists. This causes AIs to play their opening door animations wherever they encounter an AIVolume containing a WorldModel with AIActivate set to TRUE. The opening door animation plays where the AI first enters the volume containing the door, and if the door is some distance from the start of the AIVolume, then the animation then occurs some distance from the door. This results in the appearance of the AI opening the door where the door does not exist. To avoid this you must place WorldModels with AIActivate set to TRUE in their own AIVolume. The following image illustrates how to set the AIVolumes surrounding a door that opens into a hallway:



The small red square indicates the RotatingWorldModel object controlling the door, and the two gray areas indicate locations where there are no AIVolumes. The areas with no volumes keeps the AI from attempting to walk through the door when it is open from either direction. Make sure that each the abutment areas between the volumes are 48 DEdit units wide.

AIVolumeJunction Volumes

The AIVolumeJunction volumes allow AIs to lose track of the character. Like all AIVolume nodes, they require a region, and must border other AIVolume nodes. Typically junctions are placed at corners. The following image shows three junctions used in a hallway.



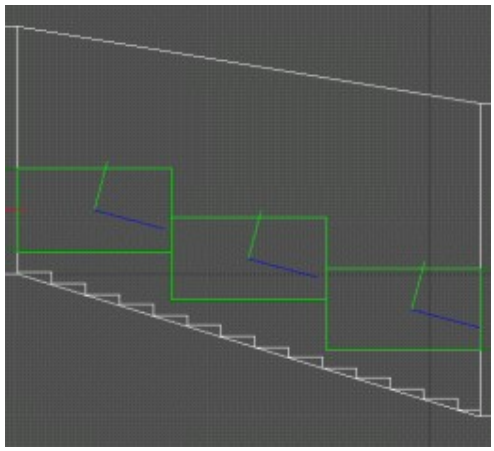
Junctions have the following effects and requirements:

- If an AI enters a junction and he cannot see his target, and has not seen his target past the junction, then he gets lost.
- If an AI gets lost in a junction, then he randomly chooses a connecting volume, and randomly chooses an action from those specified in the junction's parameters.
- As the players stealth skill increases, the odds decrease for a lost AI at a junction to choose the volume the player took.
- If an AI enters a junction and he can see his target, then the junction gets ignored and chase continues as normal.
- Junctions must have at least two connected volumes on their borders.
- Junctions must be at least 48 X 48 DEdit units in size.
- When the AI gives up a chase, he automatically searches all the search nodes in the region where the chase ended.

Notice in the hallway area of the tutorial level, each corner of the hallway has junction. Refer to the settings in these junctions for an example of how to set the properties of an AIVolumeJunction volume. These specify the bordering volumes and what actions the AI should perform when they enter the junction.

AIVolumeStairs Volumes

The stairs leading down to the Ammo room (AIRegion6) illustrate how to place stair volumes. The only purpose for stair volumes is to indicate to the AI which direction their body should roll when they are killed. Without a stair volume the AIs corpse simply falls in place on the stairs and remains there. The following image shows the placement of the AIVolumeStairs volumes used outside the Ammo room in Tut_AI:



Notice the blue arrows all point down towards the bottom of the staircase. This indicates the direction the AI's body falls when killed.

Top

Setting a Rat Patrol

There are two rat patrols in the AI tutorial. Each rat patrol uses an AIVolumeAmbientLife volume to path and travel through. Like human AIs, rat animal AIs can also follow a path. In order to patrol, the you must set patrol nodes for AIs with a patrol goal to follow.

In general, the rats don't do much except add ambience to the game. They follow their patrol paths, and stop when anything suspicious comes nearby. They can sometimes startle the guards, and likewise can be startled by the guards. Sometimes rats can become startled enough to panic and leave their set patrol routes. When this occurs, they wander around the level in a panic until they calm down and return to their normal behaviour and patrols.

To set a rat patrol

1. Place an AIRegion to bind the rat volumes to.
2. Place AIVolumeAmbientLife nodes where you want the rat to live. Bind all AIVolumes to the AIRegion you created in the previous step.
3. Place AINodePatrol nodes in the volumes you created for the rat, and bind each node to the next in the path.
4. Place a CAIHuman node in the volume, and select Rat for the ModelTemplate, Rat for the brain, and RatPatrol for the goal.
5. You may also want to set the SearchProperties - RandomItemSet to None for the Rat so that players are not prompted to search rat bodies for items.

Top

Setting a SovietSoldier Patrol

There are two SovietSoldier patrols in Tut_AI. One is in the hallway surrounding the kitchen area, and the other is in the barracks. Unlike Guards, Patrols do not own specific AINodeUseObject nodes. You can simply assign a guard a patrol route and they automatically use whatever AINodeUseObject nodes they are attracted to near their patrol route. When they finish using the node, they automatically return to their patrol.

The Hall guard has a smoking node right outside the kitchen area that he often uses near the start of the game. This places the guard in a good position for easy assassination by the player, and if the player is careful they can simply kill the guard and move his body into the kitchen where it will never be found.

Bad Example

As an example of what NOT to do, the unassigned reinforcement guard (HallGuard2) starts in the south west corner of the hall surrounding the kitchen, and may take the patrol route from the first guard once that guard has been killed. It is also possible that he will simply find a random search node and station himself there. Typically you should not place reinforcements within a level when they do not have a guard or patrol node to take. AIs with a goal who cannot find a node to perform that goal may perform undesirable random actions. HallGuard2 is an example of a guard with an unassigned node, and you can watch his actions to see the effects of placing an AI without a node to take possession of.

To set a SovietSoldier patrol

1. Place an AIRegion to bind the AIVolume nodes to, or if you have already created a region, then you can bind the new AIVolumes to that region when you add them in the next step.
2. Place AIVolume nodes where you want the soldier to live. Bind all AIVolumes to the AIRegion you created in the previous step.
3. Place AINodePatrol nodes in the volumes you created for the soldier, and bind each node to the next in the path. If you do not fully end the path, then the soldier will turn and retrace his steps when he reaches the last node in the path.
4. Place a CAIHuman node in the volume, and select SovietSoldier for the ModelTemplate, Default for the brain, and Patrol for the goal.
5. In the Commands for the guard, place the following command string:

goal_patrol node=<patrol node>

For example, the following command string sets the HallGuard to take the HallPatrol3 patrol node:

goal_patrol node=HallPatrol3

Top

Setting Initial Goals

When you place a CAIHuman node in a level, you do not need to set an initial goal. However, if you simply place all AI without initial goals, then they will migrate to Guard and Patrol nodes randomly. Typically they will take the closes Guard or Patrol node, but this may not always be the case. To prevent your AI from migrating all over the level when the level first starts, you should assign each of the CAI you initially place to a specific Guard or Patrol node, preventing them from taking possession of a distant node and having to travel to that location.

To set an initial goal

1. Using DEdit, place the AI in the level by pressing CTRL + K and selecting CAIHuman. The CAIHuman node appears at the current marker.
2. In the Commands parameter specify the initial command for the AI telling them which Guard or Patrol node to take possession of. The following list describes some of the commands you can use:
   - `goal_guard node=<Guard Node Name>`
   - `goal_patrol node=<Patrol Node Name>`
   - `goal_goto node=<Goto Node Name>`

For example, AmmoGuard1 in the Ammo room has an initial command of `goal_guard node=AmmoGuard01` setting the guard to take possession of the AmmoGuard01 AINodeGuard node. When the level starts, AmmoGuard1 moves to the AmmoGuard01 node and takes possession of the node and all the AINodeUseObject nodes specified in the GuardedNodes parameter of the AmmoGuard01 node.

Guards do not share SmartObjects, so each guard only uses its specified GuardedNodes. No other guard uses them. In contrast, patrols (who do not take possession of smart objects) may use any node not possessed by a guard if they are attracted to them on a patrol route.

Top

Creating an Alarm

Alarms are prop objects. Typically when you place them you set them to Visible FALSE so that they do not appear in the game. The purpose of alarm objects is to summon or alert guards in a specific region, and to specify the regions searched by the guards who respond to the alarm.

Alarms are commonly activated from WorldModel objects such as sliding switches, or from PlayerTrigger objects you place in areas to simulate creaky floorboards or electric eyes. When you trigger an alarm (using the ACTIVATE command) all of the guards you specified in the RespondRegions property come running to the location of the Alarm prop object and proceed to search the AIRegion the alarm exists in. Guards in AIRegions you specified in the AlertRegions section go on alert (stopping whatever AINodeUseObject actions they were performing and actively looking around). When the AI who have responded to the alarm complete their search of the AIRegion where the alarm exists, they will then proceed to search any additional regions you have specified in the SearchRegions property of the Alarm.

The alarm in the Ammo room (Region 6) is a good example of how alarms work. The sliding switch in the south east section of the Ammo room is set for both AIs and characters to use. When the sliding switch is pressed, it sends the following commands:

```
msg BackupSpawn1 SPAWN;
msg SniperSpawn (SPAWN Snipe1);
msg AmmoAlarmBell TRIGGER;
delay 1.5 (msg AmmoAlarm ACTIVATE);
```

The sequence of commands listed above spawns two backup guards, turns on the AmmoAlarmBell SoundFX node, and then activates the AmmoAlarm to summon the guards from AIRegion2, AIRegion4, and AIRegion5. AIRegion4 is also specified in the SearchRegions parameter of the alarm, setting the guards to search the hallway after they have completed their search of the Ammo room.

The Commands parameter for the CAI spawned guards has a `goal_goto node=<node name>` command. This instructs them to go to a specific location when they get spawned. If no alarm is active when they are spawned (for example, they are spawned from the EventCounter object) then their initial action is to go to the specified node, and then use their reinforcement goal to find an unassigned guard or patrol node to take possession of. If an alarm is activated after they are spawned, and they are in a region set to respond to that alarm, then they respond to the alarm first before going to the specified node.

Note: Keep in mind that alarms and the SoundFX object are two separate components. When you activate or deactivate an alarm, make sure you send a message to the SoundFX making the alarm sound or the sound simply continues playing in its loop.

Top

Creating Places to Hide

The AIVolumePlayerInfo volumes allow you to create places where you can mask an AIs sense of the player. The primary use for this node is to create places where the character can hide from the AI under the circumstances you specify. Tut_AI contains three AIVolumePlayerInfo volumes where the character can hide. The first two are in the Ammo room (AIRegion6) behind the ammo pallets. The third AIVolumePlayerInfo volume is located in the Barracks (AIRegion1) behind the file cabinet.

Notice that all the AIVolumePlayerInfo volumes in Tut_AI point to View nodes where AIs can see the character when they are hidden. This informs the AIs that once they are disturbed by getting shot from someone in these locations, they can move to the View nodes to get a view of the character. Setting AIVolumePlayerInfo volumes and View nodes in this way gives the AI the appearance of intelligence because they do not just stand around getting hit. Instead they move to a position where they can see who is firing at them.

Setting Hiding to TRUE causes AI to treat the player as if they were hidden regardless of the position the player is in. Setting HidingCrouch to TRUE causes AI to treat the player as hidden only when the player crouches.

AIVolumePlayerInfo volumes also control the "hiding" eye icon that appears when the player hides. A short timer appears to indicate when the player becomes fully hidden.

Top

Specifying a Smoking Area

Smoking areas simply give AIs a region to play their smoking animation. Smoking guards are still alert to disturbances, but it takes them a second to put down their cigarette and draw their weapon giving the player a little extra time to attack them before the AI can respond. AIs with a patrol goal can use any AINodeUseObject - SmokingSection node they come across, but AIs with a Guard goal can only use AINodeUseObject - SmokingSection nodes that have been assigned to them in the GuardedNodes parameter of their AINodeGuard node.

To specify a smoking area

1. Simply place an AINodeUseObject node where you want the AI to smoke, and then set the SmartObject parameter to smoking section.
2. If you want a specific AI guard to use the node, then in the AINodeGuard object for the desired guard, point the GuardedNodes parameter to the AINodeUseObject node where you want the AI to smoke.

Note: Never specify the same GuardedNode node for more than one guard. For example, if GuardOne has been specified an AINodeUseObject node, then do not specify the same node in the GuardedNodes parameter for GuardTwo. Doing so may result in both AI guards using the node at the same time, causing model overlap.

Top

Specifying a Place to Sleep

AIs with a sleep goal (typically SleepGuard although you can add a Sleep goal) are attracted to AINodeUseObject nodes with the following SmartObjects:

- BedTime
- BedTimeInfinite
- StandSleep
- StandSleepInfinite

A SmartObject specifying infinite does not mean the AI never wakes up, but rather indicates that the AI remains in the sleep state until disturbed. Sleeping AIs are not disturbed by visual stimulus and can only detect auditory stimulus. This allows a stealthy player to move near a sleeping AI without disturbing them.

Tut_AI contains three AIs with sleep goals in the Barracks room (AIRegion1). These three AIs have a SleepGuard goal and the AINodeGuard nodes they are assigned to each have a AINodeUseObject - BedTimeInfinite node set as one of their GuardedNodes. Since their primary goal is to sleep, and the AINodeUseObject nodes are set to BedTimeInfinite, the first action they perform when the game starts is to lay down and go to sleep. Until the character enters the room and disturbs them (or until an alarm goes off) they remain in the sleep state.

To specify a place to sleep

1. In the GoalSet for the character, specify SleepGuard. Alternatively you can use the ADDGOAL command to add a sleep goal to any AI, although if they do not have a usable AINodeUseObject with one of the four sleep SmartObjects, then they will simply remain awake.
2. Place an AINodeUseObject node near a bed, chair, or in a location you want the AI to sleep. In the SmartObject for the node, specify one of the four sleep objects. For more information about the placement of the node, see Using AIUseObjectNodes and SmartObjects.
3. For AIs with a SleepGuard GoalSet, in the AINodeGuard object they are assigned to, open BindToObject and attach the AINodeUseObject node you want them to use when they sleep.
4. For AIs with a goal other than SleepGuard (AI you have added a sleep goal to), simply place the AINodeUseObject with the sleeping SmartObject in an area where they can be attracted to it.

Top

Specifying a Place to Sit

AIs with a work goal are attracted to all AINodeUseObject nodes, although AIs with a Guard goal will only use those objects specified in their GuardedNodes list. Guards and Patrols attracted to AINodeUseObjects that have a sitting animation require templates that possess a sitting animation.

A SmartObject specifying infinite does not mean the AI never gets up, but rather indicates that the AI remains sitting until disturbed.

Tut_AI contains three AIs with sleep goals in the Barracks room (AIRegion1). These three AIs have a SleepGuard goal and the AINodeGuard nodes they are assigned to each have a AINodeUseObject - BedTimeInfinite node set as one of their GuardedNodes. Since their primary goal is to sleep, and the AINodeUseObject nodes are set to BedTimeInfinite, the first action they perform when the game starts is to lay down and go to sleep. Until the character enters the room and disturbs them (or until an alarm goes off) they remain in the sleep state.

To specify a place to sit

1. Place an AINodeUseObject node near a chair. In the SmartObject for the node, specify one of the sitting objects. For more information about the placement of the node, see Using AIUseObjectNodes and SmartObjects.
2. For AIs with a Guard GoalSet, in the AINodeGuard object they are assigned to, open BindToObject and attach the AINodeUseObject node you want them to use when they sit.
3. For AIs with a goal other than Guard, simply place the AINodeUseObject with the sitting SmartObject in an area where they can be attracted to it.

Top

Spawning New AI

There are several factors involved with the spawning of new AI into your level. First, you want to limit the number of AI in your levels to keep memory costs to a minimum. Second, you may want to spawn new AI when alarms get activated so that there is an appropriate response to the alarm. Third, you may want to repopulate your level to keep the game from becoming too easy, or to simulate repopulation after the character has been gone for a time and returned to a certain area.

Creating a Place to Spawn

You want to limit the size of your level while still simulating that it is part of a larger world. AIs should be capable of summoning help from outside the level, but in the game code there is nothing outside the level. To simulate a larger world you need to create areas where the player cannot go and doors they cannot pass through in order to have a safe place to spawn AI. You can also spawn them in areas where you know the player is not currently at, but if you have a multiplayer game, it may be difficult to ensure that a player is not in a location when new AIs are spawned.

To create a safe place for AIs to spawn, create a door somewhere in your level that opens into a room. Set the door so that it can only be triggered by AIs and not the character. When an AI summons backup, you can safely spawn the backup in the blocked room and then send the AIs messages to move to a specific node.

Tut_AI has a spawning room just outside the Laundry room (AIRegion5). The door to this room (the metal door in the laundry room) is set so that only AIs can open it. In the event that the player attempts to get through the door when an AI opens it, a blocker brush placed in the doorway prevents the player from entering the room. Blocker brushes do not effect AIs, only players.

The Spawner nodes placed in this room are set to spawn the two template CAIs located outside of the world geometry near the spawning room. The following general steps explain how to spawn an AI:

To spawn an AI

1. Create and place an CAIHuman anywhere in the level.
   You may want to place CAI node outside of the world geometry to help you keep track of the fact that this CAI is a template and does not appear in the game until spawned.
2. Set the parameters for the CAI, and ensure the Template parameter is TRUE.
3. Place a Spawner node where you want the CAI to appear when it spawns.
4. In the Spawner, set Target to exact name of the CAI you want it to spawn.
   For example, setting Target to BackupTemplateSniper tells the Spawner to spawn the BackupTemplateSniper CAI when it is triggered.
5. To activate the spawner, send the following command from any object capable of using messages:

```
msg <Spawner Name> SPAWN
```

```
-or-
msg <Spawner Name> (Spawn <Name>)
```

For example `msg SniperSpawn (SPAWN Snipe1)` tells the Spawner object named SniperSpawn to spawn a CAI named Snipe1. To spawn without a name, simply use `msg SniperSpawn SPAWN`.

6. When the message gets received, the Target template CAI gets spawned at the location of the Spawner node. Any initial commands contained in the CAI are immediately processed. AIs with an initial GoalSet of Reinforcement immediately search for an unassigned Guard or Patrol node to take possession of.

Controlled Repopulation

To keep the level difficulty somewhat the same, and to simulate guards coming on duty to relieve other guards, Tut_AI uses an event counter to trigger respawning. Each AI has a DamageProperties parameter, and every AI in the Tut_AI has a DeathCommand set to send `msg EventCounter0 (inc 1)` when the AI gets killed. The EventCounter object sends spawning commands when it reaches specific values. For example, when it reaches a value of three, it sends the following command:

```
msg BackupSpawn1 SPAWN
```

This spawns one new AI. When the EventCounter reaches a value of five, it sends the following command:

```
msg SniperSpawn SPAWN
```

This spawns one new AI. When the EventCounter reaches a value of eight, it sends the following commands:

```
msg BackupSpawn1 SPAWN;
msg SniperSpawn SPAWN
```

This spawns two new AI. When the EventCounter reaches a value of nine, it sends the following command:

```
msg EventCounter0 (DEC 9)
```

This resets the EventCounter to zero, and the process starts all over again as new AI get killed. Eventually it is possible to kill all of the AI in the level without triggering any respawning, however, because exploding ammo crates and alarm switches also spawn new AI, it takes the player quite some time to reach this point. In most cases the level should end before all AI are killed and no further AI are being spawned.

Top

Using Variables

You can create and use integer variables through the command syntax of any object capable of sending command messages. Variables allow you to set and track conditions in your game. For example, Tut_AI creates and uses a BombCount variable to unlock the exit trigger that ends the level. The two bombable targets (ammo pallets) in the Ammo room both increment and test the BombCount variable to determine when to unlock the ExitTrigger trigger in the Kitchen room.

To create and use variables

You can create a variable in any command message. The BombCount variable is initially created in the GameStartPoint object for the level. The command to create the variable uses the following syntax:

```
int BombCount 0
```

This creates and initializes the BombCount variable to zero. The command objects (CommandObject0 and CommandObject00) both increment and test this variable using the following syntax:

```
Add BombCount 1
IF (BombCount == 2) THEN (msg ExitTrigger UNLOCK)
```

When the BombCount variable equals 2, then the ExitTrigger gets unlocked. This keeps the player from leaving the level until they have completed the object of destroying both ammo pallets.

Debugging Variables

When using variables you may find it useful to search for objects using commands with a specific variable name. To get a list of all commands with a specific variable name, you can use the Object Search dialog box.

To Debug Variables

1. On the View menu, move to Debugging and then click Object Search.
2. In the Find box, type the name of the variable, and then click the Search button.
3. In the resulting list, double click the line containing the object to select the object.

Variables and Command Syntax

For additional information about variables and command syntax, see the **Command Architecture** document in the following folder of your Jupiter installation directory:

LT_Jupiter_Src\Specs\GameSystems\Commands.doc

Top

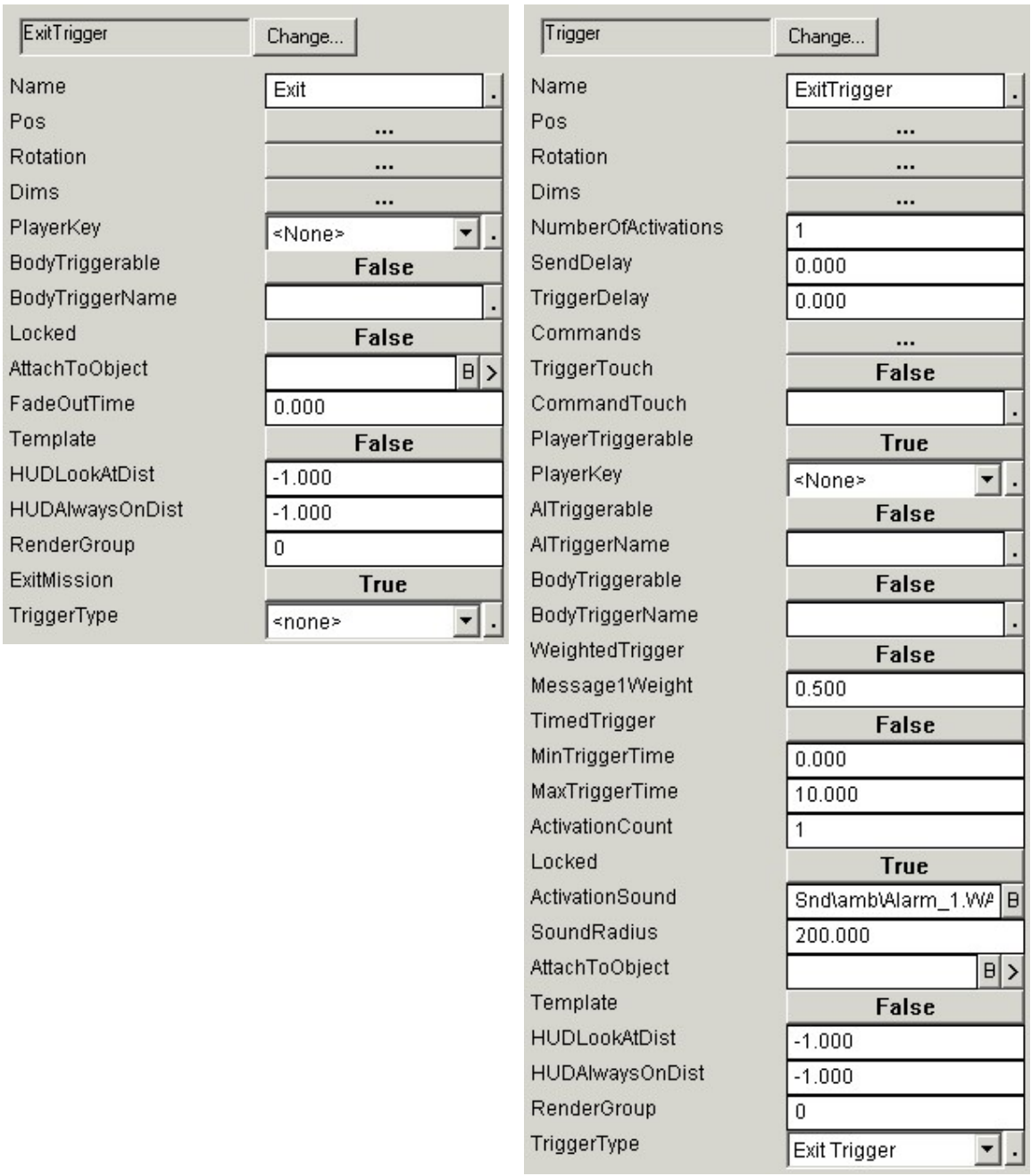
Exiting a Level

Simply exiting a level requires two trigger objects: a PlayerTrigger to send a TRIGGER message, and an ExitTrigger to exit the level. Tut_AI uses a locked Trigger node to trigger an ExitTrigger. When the player destroys both of the bombable ammo crates in the Ammo room (AIRegion6) the Trigger in the kitchen gets unlocked. To exit the level after the ammunition is destroyed, all the player needs to do is return to the kitchen.

The following images show the properties for the two trigger objects. The Trigger object sends the following message to the ExitTrigger object:

mgs Exit TRIGGER

ExitTrigger Object **Trigger Object**



The ExitMission property, when set to TRUE, exits the level to the first level of the next mission or ends the game if the current level is the last level in a mission. When set to FALSE, the ExitTrigger simply ends the level.

Top
